

System Design Interview Mastery

A Production-Grade Guide for Senior, Staff & Backend Engineers

14 Modules • 27 Classic Design Problems • Cheat Sheets & Mock Interviews
Foundations • Networking • Caching • Databases • Messaging • Storage
Microservices • Reliability • Security • Performance • Distributed Systems
Observability • System Design Problems • Interview Strategy

Table of Contents

System Design Interview Mastery

Module 1 — Foundations

- 1.1 Latency vs Throughput
- 1.2 Horizontal vs Vertical Scaling
- 1.3 The Core "-ilities": Availability, Reliability, Scalability, Durability
- 1.4 Consistency (and the Consistency Spectrum)
- 1.5 CAP Theorem and PACELC
- 1.6 SLA, SLO, SLI
- 1.7 Load Patterns & Read-heavy vs Write-heavy Systems
- 1.8 Stateless vs Stateful Services

Module 2 — Networking

- 2.1 TCP, UDP, and QUIC
- 2.2 HTTP/1.1 → HTTP/2 → HTTP/3, and HTTPS
- 2.3 DNS
- 2.4 CDN
- 2.5 Load Balancer, Reverse Proxy, API Gateway (and NAT)
- 2.6 WebSocket
- 2.7 gRPC

Module 3 — Caching

- 3.1 Why Caching Works (and the Cache Hierarchy)
- 3.2 Redis vs Memcached
- 3.3 Caching Patterns: Cache-Aside, Write-Through, Write-Back, Write-Around
- 3.4 Cache Invalidation
- 3.5 Cache Stampede (Thundering Herd)
- 3.6 Hot Keys
- 3.7 Distributed Cache Architecture

Module 4 — Database Design

- 4.1 SQL vs NoSQL
- 4.2 Replication: Leader-Follower, Multi-Leader, Leaderless (+ Read Replicas)
- 4.3 Partitioning & Sharding
- 4.4 Indexing Strategy
- 4.5 Schema Design (Normalization vs Denormalization)
- 4.6 Connection Pooling
- 4.7 CQRS (High Level)

Module 5 — Messaging & Event-Driven Architecture

- 5.1 Core Model: Queues, Topics, Producers, Consumers, Consumer Groups
- 5.2 Kafka (Deep Dive)
- 5.3 RabbitMQ, ActiveMQ, Amazon SQS (and choosing among them)
- 5.4 Ordering
- 5.5 Delivery Semantics: At-Most-Once, At-Least-Once, Exactly-Once
- 5.6 Retries, Dead Letter Queues, Backpressure
- 5.7 Event-Driven Architecture (EDA)

Module 6 — Storage

- 6.1 Object Storage (S3, GCS, Azure Blob)
- 6.2 Block Storage (EBS, Persistent Disk, SAN)
- 6.3 File Storage (NFS, EFS, Filestore)
- 6.4 "Blob Storage" and Handling Large Binary Data in Databases
- 6.5 Distributed File Systems (GFS/HDFS lineage, Colossus, Ceph)
- 6.6 Storage Trade-offs (the decision table)

Module 7 — Microservices & Resilience Patterns

- 7.1 Monolith vs Microservices
- 7.2 Service Discovery
- 7.3 API Gateway in a Microservice Architecture
- 7.4 Resilience: Timeout, Retry, Circuit Breaker, Bulkhead
- 7.5 Idempotency
- 7.6 Distributed Transactions: 2PC vs Saga
- 7.7 Outbox Pattern (and the Dual-Write Problem)

Module 8 — Reliability & High Availability

- 8.1 High Availability & Redundancy
- 8.2 Failover, Health Checks, Heartbeats
- 8.3 Disaster Recovery (DR)
- 8.4 Consensus & Raft (Interview Level)
- 8.5 Split Brain

Module 9 — Security

- 9.1 Authentication vs Authorization
- 9.2 Session Management vs JWT (Stateful vs Stateless Auth)
- 9.3 OAuth 2.0 & OpenID Connect
- 9.4 API Keys & Service-to-Service Auth
- 9.5 Rate Limiting
- 9.6 Encryption & TLS

9.7 Secrets Management

Module 10 — Performance Engineering

10.1 Bottleneck Analysis & Profiling

10.2 Slow Queries

10.3 Connection Pool Exhaustion

10.4 N+1 Queries

10.5 Compression

10.6 Pagination

10.7 Batch Processing & Streaming

Module 11 — Distributed Systems Toolkit

11.1 Consistent Hashing

11.2 Bloom Filters

11.3 Merkle Trees (Overview)

11.4 Vector Clocks (Overview)

11.5 Leader Election

11.6 Gossip Protocol (Overview)

11.7 Distributed Locks

11.8 Clock Synchronization

11.9 Eventual Consistency (Revisited, Mechanically)

Module 12 — Observability

12.1 The Three Pillars: Logs, Metrics, Traces

12.2 Monitoring & Alerting

12.3 Prometheus & Grafana (and the metrics pipeline)

12.4 OpenTelemetry (High Level)

12.5 How Engineers Actually Debug Production Issues

Module 13 — Classic System Design Problems (Part 1)

13.1 URL Shortener / TinyURL

13.2 Instagram Feed / Facebook News Feed / Twitter(X) Timeline

13.3 WhatsApp / Chat Application

13.4 YouTube / Netflix (Video Platforms)

13.5 Google Drive / Dropbox (File Sync & Storage)

13.6 Uber / Ola (Ride-Hailing)

13.7 Airbnb / Hotel Booking / Flight Booking

13.8 Payment Gateway / Payment System

13.9 E-commerce Platform & Inventory Management

Module 13 — Classic System Design Problems (Part 2)

13.10 Notification Service

13.11 Search Autocomplete (Typeahead)

13.12 Web Crawler

13.13 Distributed Cache (design Redis/Memcached itself)

13.14 Rate Limiter (as a system)

13.15 API Gateway (as a system)

1 outage cause $\Rightarrow$ validation + canary + auto-rollback on error-rate spike);

13.16 Logging System (Centralized Log Platform)

13.17 Analytics Platform (Events $\rightarrow$ Insights)

Module 14 — Interview Strategy

14.1 The 45–60 Minute Structure (with time budget)

14.2 Requirement Gathering (minutes 0–5)

14.3 Capacity Estimation (minutes 5–8)

14.4 API Design & Data Model (minutes 8–12)

14.5 High-Level Design (minutes 12–20)

14.6 Deep Dives (minutes 20–35) — where offers are decided

14.7 Bottlenecks, Scaling, Failure Sweep (minutes 35–40)

14.8 Monitoring, Security, Final Review (minutes 40–45)

14.9 How Interviewers Evaluate You

14.10 Common Failure Patterns (and the fix)

14.11 Preparation Plan & Mock Protocol

System Design Interview Mastery

A Production-Grade Guide for Senior, Staff, and Backend Engineers

This guide is written for engineers with 5+ years of backend experience preparing for Senior Software Engineer, Staff Engineer, and Backend Engineer system design interviews at companies like Google, Meta, Amazon, Netflix, Uber, Stripe, Microsoft, Airbnb, LinkedIn, and Cloudflare.

It deliberately skips beginner material and academic theory. Every topic here is included because it (a) appears repeatedly in real interviews and (b) is used daily in real production systems.

How This Guide Is Organized

Module	Focus
1	Foundations: latency, scaling, CAP/PACELC, SLA/SLO/SLI, load patterns
2	Networking: HTTP/1.1–3, TCP/UDP/QUIC, DNS, CDN, LB, gRPC, WebSocket
3	Caching: Redis, Memcached, patterns, stampede, hot keys
4	Database design: replication, sharding, indexing, CQRS
5	Messaging: Kafka, RabbitMQ, SQS, delivery semantics, DLQ, EDA
6	Storage: object/block/file/blob, distributed file systems
7	Microservices: discovery, circuit breakers, sagas, outbox, idempotency
8	Reliability: HA, failover, DR, consensus, Raft, split brain
9	Security: authn/authz, JWT, OAuth 2.0, OIDC, rate limiting, TLS
10	Performance: bottlenecks, profiling, N+1, pagination, streaming
11	Distributed systems: consistent hashing, Bloom filters, locks, clocks
12	Observability: logs, metrics, traces, Prometheus, Grafana, OTel
13	27 classic system design problems, solved end to end
14	Interview strategy: how to run a 45–60 minute interview and how you are graded

The Topic Template

Every core topic follows the same 15-part structure so you can drill it into a repeatable mental model:

1. **Why interviewers ask this** — the signal they are probing for
2. **Core concept** — the one-paragraph mental model
3. **Internal working** — what actually happens under the hood
4. **Visual architecture** — an ASCII diagram
5. **Real production example** — how a large company uses it
6. **Advantages** — when it wins
7. **Trade-offs** — what you pay for it
8. **Common mistakes** — what weak candidates say
9. **Scaling considerations** — behavior at 10x and 100x
10. **Failure scenarios** — how it breaks in production
11. **Monitoring & debugging** — what to measure and how to investigate
12. **Interview questions** — direct questions you will be asked
13. **Follow-up questions** — the second-order probes that separate senior from mid-level

14. **Best practices** — production rules of thumb
15. **Hands-on design exercise** — apply it yourself

Where topics are tightly coupled (for example SLA/SLO/SLI), they are taught together so the comparisons — which are what interviewers actually grade — stay in one place.

How Interviewers Grade You (Preview)

Interviewers at senior+ level are not checking whether you know what a load balancer is. They are grading:

- **Requirement discipline** — do you scope before you design?
- **Quantitative reasoning** — can you estimate QPS, storage, and bandwidth and let the numbers drive the design?
- **Trade-off fluency** — do you present two or three options and argue for one, or do you dump a single memorized architecture?
- **Failure thinking** — do you volunteer what breaks, or wait to be asked?
- **Depth on demand** — when the interviewer says "zoom into X", can you go three levels deep?
- **Communication** — can a colleague implement your design from your whiteboard?

Keep this rubric in mind through every module. Module 14 expands it into a full playbook.

Module 1 — Foundations

The vocabulary of every system design interview. You will use every concept in this module in every design you ever draw. Interviewers rarely ask these as standalone questions at senior level — instead they listen for whether you use them *correctly and quantitatively* while designing.

1.1 Latency vs Throughput

Why Interviewers Ask This

It is the fastest way to detect whether a candidate thinks quantitatively. Senior candidates are expected to know that the two are different axes, that improving one can hurt the other, and that tail latency — not average — is what pages you at 3am.

Core Concept

- **Latency:** time for a single operation to complete (ms). What one user feels.
- **Throughput:** operations completed per unit time (QPS/RPS). What the fleet handles.

They are related through concurrency by **Little's Law**:

$$\text{concurrency} = \text{throughput} \times \text{latency}$$
$$(L = \lambda \times W)$$

A service doing 1,000 QPS at 200 ms average latency has ~200 requests in flight at any moment. That single formula sizes your thread pools, connection pools, and load balancer limits.

Internal Working

Latency decomposes into a pipeline; know the rough magnitudes cold:

Operation	Typical latency
L1 cache reference	~1 ns
Main memory reference	~100 ns
Read 1 MB sequentially from memory	~10 μ s
SSD random read	~100 μ s
Read 1 MB from SSD	~1 ms
Round trip within same datacenter	~0.5 ms
Read 1 MB over 1 Gbps network	~10 ms
Disk seek (HDD)	~10 ms
Round trip US East ↔ US West	~60–70 ms
Round trip US ↔ Europe	~80–100 ms
Round trip US ↔ Asia	~150–250 ms

Throughput is limited by the narrowest stage of the pipeline (the bottleneck). Adding capacity anywhere else changes nothing.

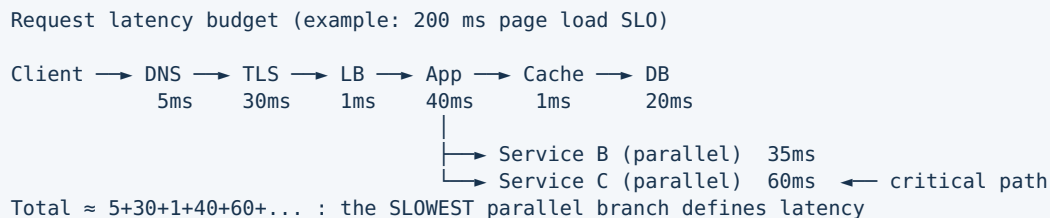
Latency is reported as a distribution, never an average:

- **p50** (median): the typical experience

- **p95 / p99**: the tail — slow requests caused by GC pauses, cold caches, lock contention, retries
- **p99.9**: matters at scale — at 1B requests/day, p99.9 is still 1M slow requests/day

Tail latency compounds with **fan-out**: if one page requires 100 parallel backend calls and each has p99 = 1s, then ~63% of pages hit at least one slow call ($1 - 0.99^{100}$). This is why Google and Amazon obsess over tails ("The Tail at Scale", Dean & Barroso).

Visual Architecture



Real Production Example

Amazon famously measured that every +100 ms of latency cost ~1% in sales. Google found +500 ms in search page load dropped traffic ~20%. Netflix keeps its API gateway p99 budget in tens of milliseconds and uses request hedging (send a backup request if the first is slow) to clip the tail.

Advantages / Trade-offs of Optimizing Each

- Optimizing **latency**: better UX, but often costs money (more caching, more replicas closer to users, hedged requests double load).
- Optimizing **throughput**: better cost efficiency via batching/queueing/pipelining — but batching *adds* latency (you wait to fill the batch). Kafka is the canonical example: huge throughput, deliberately relaxed per-message latency.

Common Mistakes

- Quoting averages. "Average latency is 50 ms" hides a 2-second p99.
- Assuming more servers reduce latency. Horizontal scaling raises throughput; a single request is not faster.
- Ignoring queueing: as utilization $\rightarrow 100\%$, latency $\rightarrow \infty$ (queueing theory hockey stick). Run hot at 70–80%, not 95%.
- Forgetting fan-out amplification of tail latency.

Scaling Considerations

- Throughput scales horizontally (more machines) if the workload is partitionable.
- Latency scales geographically (CDN, edge, regional replicas) — you cannot beat the speed of light, so move data closer.
- Past ~80% utilization, every extra percent of load disproportionately inflates tail latency.

Failure Scenarios

- **Retry storms**: a latency spike causes clients to retry, multiplying load, raising latency further — a metastable failure. Mitigate with exponential backoff + jitter and retry budgets.
- **Slow dependency, not down dependency**: a service that answers in 5s is far more dangerous than one that fails fast, because it holds threads/connections hostage. Timeouts are a latency tool.

Monitoring & Debugging

- Dashboards: p50/p95/p99/p99.9 latency histograms per endpoint, QPS, utilization, queue depth.

- Debug a latency regression with distributed tracing: find which span grew. Correlate with deploys, GC logs, cache hit ratio drops, DB slow-query logs.

Interview Questions

1. Your p50 is 20 ms but p99 is 2 s. What are the likely causes and how do you investigate?
2. How does Little's Law help you size a connection pool?
3. Your service must go from 1k QPS to 100k QPS — what changes? Must latency change?

Follow-up Questions

- "Would you hedge requests? What does that do to backend load?" (roughly doubles tail-triggered load; cap hedges at ~5% of traffic)
- "Why does batching improve throughput but hurt latency?"

Best Practices

- Define a per-endpoint latency budget and allocate it across dependencies.
- Alert on p99, capacity-plan on throughput, and keep utilization headroom (~30%).
- Measure at the client or LB, not just inside the service (you'll miss queueing and network time otherwise).

Hands-on Design Exercise

Your API must serve 50k QPS with p99 < 100 ms. Each request does one cache lookup (1 ms, 95% hit) and, on miss, one DB query (20 ms). Estimate: (a) DB QPS, (b) in-flight requests, (c) the app instance count if one instance handles 500 concurrent requests. (*Answers: 2.5k DB QPS; $\approx 50,000 \times \sim 0.002s \approx 100\text{--}150$ in flight overall depending on mix; work through it — then double everything for headroom.*)

1.2 Horizontal vs Vertical Scaling

Why Interviewers Ask This

Every design eventually gets the question "what happens at 10x load?" The interviewer wants to see that you know scaling out is an *architectural property you must design for* (statelessness, partitionability), not a knob you turn later.

Core Concept

- **Vertical scaling (scale up):** bigger machine — more CPU, RAM, faster disks.
- **Horizontal scaling (scale out):** more machines behind a load balancer or partitioning scheme.

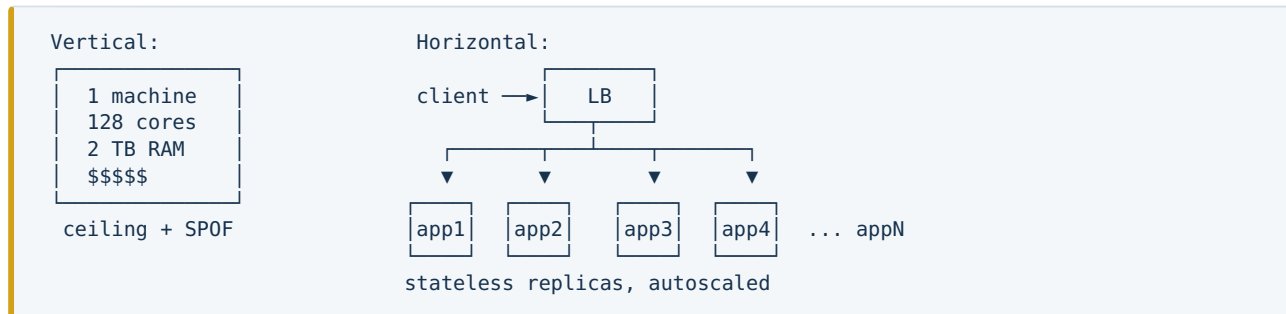
Vertical is simple but has a hard ceiling and a single point of failure. Horizontal is theoretically unbounded but forces you to solve distribution problems: state, coordination, partial failure.

Internal Working

Horizontal scaling of **stateless** services is trivial: LB + N replicas + health checks + autoscaling. Horizontal scaling of **stateful** systems (databases) is the hard part and requires replication (copies for reads/HA) and sharding (partitioning for writes/capacity) — Module 4.

Amdahl's law governs the ceiling: if fraction s of the work is serialized (a lock, a single leader, a hot row), maximum speedup is $1/s$ no matter how many machines you add.

Visual Architecture



Real Production Example

Stack Overflow famously ran for years on a handful of very large SQL Server boxes (vertical) — simplicity was a feature. Google, Meta, and Amazon scale horizontally on commodity hardware because at their size no single machine is big enough and failure is constant. Stripe scales its API tier horizontally but scales its primary databases with a mix of vertical headroom plus sharding.

Advantages

- **Vertical:** no code changes, no distributed-systems complexity, strong single-node consistency, great for relational databases early on.
- **Horizontal:** near-unlimited capacity, fault tolerance (N-1 survives), cost-efficient commodity hardware, enables rolling deploys.

Trade-offs

- Vertical: exponential cost curve, downtime to resize, hard ceiling, SPOF.
- Horizontal: needs statelessness or partitioning, introduces network hops, partial failures, data consistency questions, operational complexity (orchestration, service discovery).

Common Mistakes

- Jumping to "shard everything" for a system doing 200 QPS. Senior answer: one Postgres primary + replicas + cache covers a surprisingly large scale; shard when write volume or dataset size forces you.
- Claiming horizontal scaling is "infinite" while the design contains a single leader, a global lock, or one hot partition.
- Forgetting that scaling the app tier just moves the bottleneck to the database.

Scaling Considerations

- Scale reads first (replicas, cache), then writes (sharding), because reads usually dominate and are easier.
- Autoscaling policies need warm-up time and should scale on leading indicators (CPU, queue depth, concurrency), with limits to prevent runaway cost.

Failure Scenarios

- Vertical: hardware failure = full outage; resize = downtime window.
- Horizontal: cascading failure when one replica dies and its load overwhelms the rest (always provision N+1/ N+2); thundering herd on cold start of many new instances (cold caches).

Monitoring & Debugging

- Fleet-wide: instance count, per-instance CPU/memory, LB request distribution (detect skew), autoscaler activity.
- Debug skew: one hot instance usually means sticky sessions, a bad hash, or a hot key.

Interview Questions

1. When would you choose vertical scaling despite the SPOF?
2. Your write-heavy DB is at capacity — walk through your scaling options in order.
3. What prevents a service from scaling horizontally?

Follow-up Questions

- "You doubled instances but throughput didn't move. Why?" (bottleneck is downstream: DB, lock, external API)
- "How does Amdahl's law show up in real architectures?" (single leader, sequence generators, global rate limiter)

Best Practices

- Design services stateless from day one; externalize state to Redis/DB.
- Scale up until it's expensive or risky, then scale out — in that order for databases.
- Always run N+2 for critical tiers across ≥ 3 availability zones.

Hands-on Design Exercise

An image-resizing service is CPU-bound: 1 instance = 50 resizes/s. Traffic is 2,000 resizes/s peak, 200 off-peak. Design the fleet: instance count, autoscaling signal, and what you'd do if a batch client suddenly submits 50,000 images. (*Expect: ~40+ headroom at peak, scale on queue depth not CPU alone, absorb the batch with a queue rather than autoscaling to absurdity.*)

1.3 The Core "-ilities": Availability, Reliability, Scalability, Durability

Why Interviewers Ask This

These words appear in the first five minutes of every interview ("what are your non-functional requirements?"). Interviewers check that you attach *numbers* to them and understand they conflict — you cannot maximize all four for free.

Core Concept

Property	Question it answers	Measured by
Availability	Can I reach the system right now?	uptime %, error rate
Reliability	Does it do the <i>correct</i> thing consistently?	MTBF, defect rate, correctness
Scalability	Does it keep working as load grows?	throughput vs resources curve
Durability	Once acknowledged, is my data safe forever?	probability of data loss

Availability $\neq$ reliability: a service that is up but returns wrong data is available and unreliable. Durability $\neq$ availability: S3 can be unreachable for an hour (availability incident) without losing a single byte (durability intact).

Internal Working

Availability math (know these cold):

Nines	Downtime/year	Downtime/day
99%	3.65 days	14.4 min
99.9%	8.77 hours	1.44 min
99.99%	52.6 min	8.6 s

Nines	Downtime/year	Downtime/day
99.999%	5.26 min	0.86 s

Composition rules:

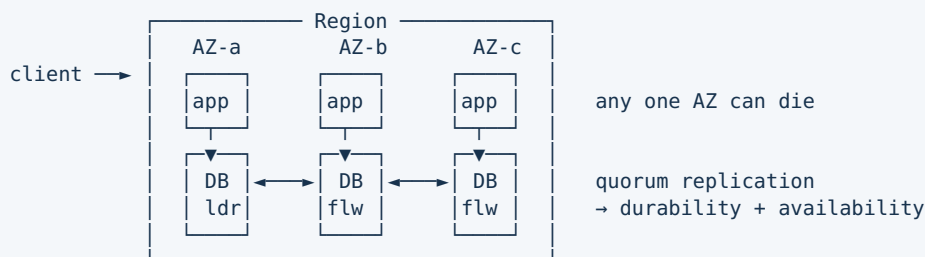
Serial (A depends on B): $\text{Availability} = A \times B$
 two 99.9% services in series $\rightarrow 99.8\%$
 Parallel (either works): $\text{Availability} = 1 - (1-A)(1-B)$
 two 99% replicas in parallel $\rightarrow 99.99\%$

Serial chains destroy availability; redundancy multiplies it. That is the entire mathematical justification for replication and for keeping dependency chains short.

Durability comes from replication (3 copies, ideally across AZs), write-ahead logs with fsync, erasure coding (S3 style: 11 nines), backups, and — critically — protection against *correlated* failures (same rack, same firmware bug, same `rm -rf` script). Replication is not backup: replication faithfully copies your accidental DELETE to every replica.

Visual Architecture

Availability through redundancy (3 AZs):



Real Production Example

Amazon S3 advertises 99.999999999% (11 nines) durability via erasure coding across devices and AZs, but "only" 99.99% availability — an honest admission that the two are different engineering problems. Google Spanner offers 99.999% availability for multi-region deployments by paying for synchronous cross-region replication.

Advantages / Trade-offs

- Each extra nine of availability roughly **10x-es the cost and complexity**: more redundancy, multi-region, automated failover, and it constrains deploy velocity (less room for bad deploys — which cause most outages).
- Higher durability (synchronous replication, fsync per write) directly costs write latency.
- Know your real requirement: an internal analytics tool does not need 99.99%.

Common Mistakes

- Promising "five nines" casually. 5 minutes of downtime *per year* means fully automated failover, multi-region, and near-zero-risk deploys. Most businesses need 99.9–99.99%.
- Ignoring dependency math: your service cannot be more available than the serial chain of its hard dependencies.
- Conflating replication with backup.
- Treating scalability as binary rather than "scales linearly up to X, then bottleneck Y".

Scaling Considerations

At larger scale, failures stop being rare events and become a constant rate (with 100k disks, disks die *daily*). Architecture must treat failure as routine: automated detection, automated replacement, graceful degradation.

Failure Scenarios

- **Correlated failure** defeats redundancy: whole-AZ outage, shared config push, a bad deploy hitting all replicas simultaneously. Mitigate with AZ isolation, staged rollouts, cell-based architecture.
- **Gray failure**: node is "up" per health checks but slow/wrong — often worse than clean death.

Monitoring & Debugging

- Availability: success rate SLI at the load balancer (user-facing), synthetic probes from outside your network.
- Durability: checksum scrubbing (S3 continuously verifies data integrity), backup restore drills — an untested backup is not a backup.

Interview Questions

1. Your service depends on 5 services, each 99.9%. What's your ceiling? ($\approx 99.5\%$)
2. How do you design for 99.99% when your cloud provider's single region offers 99.9%?
3. Difference between durability and availability for a storage system?

Follow-up Questions

- "The business demands five nines. What do you push back with?" (cost, deploy freeze implications, whether the dependency chain even allows it)
- "How would you *verify* your durability claim?" (restore drills, scrubbing, chaos testing)

Best Practices

- State explicit numeric targets in the first 5 minutes of any design.
- Minimize hard (serial) dependencies; convert them to soft dependencies with graceful degradation (serve stale cache if DB is down).
- Deploy gradually (canary $\rightarrow$ 1% $\rightarrow$ 10% $\rightarrow$ 100%) — deploys cause most incidents.

Hands-on Design Exercise

You run a checkout service (must be 99.99%) that calls: payments provider (99.95%), inventory service (99.9%), email service (99.5%). Rework the architecture so email and inventory failures don't count against checkout availability. (*Expect: async email via queue; inventory check with cached fallback or optimistic reserve + reconcile.*)

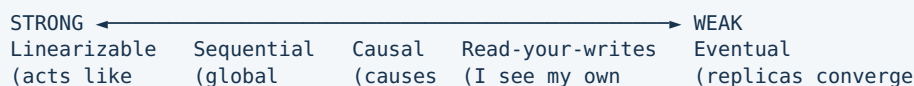
1.4 Consistency (and the Consistency Spectrum)

Why Interviewers Ask This

Consistency choices drive database selection, replication mode, and UX. Senior candidates must place a use case on the consistency spectrum and justify it, not just chant "eventual consistency" at every problem.

Core Concept

Consistency = do all observers see the same data at the same time? It is a spectrum:



one copy)	order)	before updates) effects)	...eventually)
-----------	--------	-----------------------------	----------------

- **Linearizable / strong:** every read sees the latest acknowledged write. Needed for: account balances, inventory decrement, unique username claims, lock services.
- **Causal:** if B was caused by A, everyone sees A before B (comment appears after the post it replies to).
- **Read-your-writes:** a user always sees their own updates (profile edits) — often implemented by pinning that user's reads to the leader or their own session replica.
- **Monotonic reads:** a user never sees data go *backwards* in time between requests.
- **Eventual:** replicas converge if writes stop. Fine for: like counts, follower counts, feeds, DNS.

Internal Working

Strong consistency in a replicated system requires coordination on the write path: either a single leader that serializes writes plus reads served by the leader (or quorum reads), or a consensus protocol (Raft/Paxos) per write. Quorums: with N replicas, write to W , read from R ; if $R + W > N$, read and write sets overlap so reads see the latest write (e.g., $N=3$, $W=2$, $R=2$). Eventual consistency drops the coordination: accept writes anywhere/asynchronously, replicate in background, resolve conflicts (last-writer-wins, vector clocks, CRDTs).

Real Production Example

- Meta: your own profile edit is read-your-writes (session pinned to region of write via a "write-through" cookie); friend's like counts are eventual.
- Bank ledgers, Stripe payments: strongly consistent, serialized per account/idempotency key.
- Amazon DynamoDB: eventually consistent reads by default, strongly consistent reads on request at double the read cost — a literal price tag on consistency.

Advantages / Trade-offs

- Strong: simple to reason about, no anomalies — but higher latency (coordination round trips), lower availability under partition, doesn't span regions cheaply.
- Eventual: fast, available, partition-tolerant — but the application must tolerate anomalies (stale reads, conflicting writes) and sometimes resolve conflicts.

Common Mistakes

- Applying one consistency level to the whole system. Real systems mix levels per data type — say this explicitly in interviews.
- Believing eventual consistency means "wrong": it means *temporarily stale*, with convergence guarantees.
- Ignoring the user-facing anomaly: "user posts a comment, refreshes, comment disappears" = missing read-your-writes.

Interview Questions

1. Which operations in an e-commerce site need strong consistency? (*inventory decrement at checkout, payment; not product views or reviews*)
2. How do you implement read-your-writes over async replicas?
3. Explain $R+W>N$ quorum consistency.

Best Practices

- Classify every entity in your design: strong / causal / eventual — and say why.
- Push strong consistency to the smallest possible scope (a single row, a single partition) so the rest of the system can be fast and available.

Hands-on Design Exercise

For a Twitter-like app, assign a consistency level to: tweet publish, timeline read, follower count, DM delivery, username registration. Justify each in one line.

1.5 CAP Theorem and PACELC

Why Interviewers Ask This

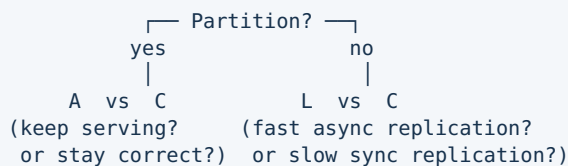
It's the classic filter question — but at senior level the interviewer wants the *correct, nuanced* version, and PACELC, because CAP alone is widely misquoted.

Core Concept

CAP: when a network **P**artition happens, a distributed system must choose between **C**onsistency (every read sees the latest write, or errors) and **A**vailability (every request gets a non-error response, possibly stale). Partition tolerance is not optional — networks *will* partition — so the real choice is only what to do *during* a partition:

- **CP**: refuse/fail some requests to stay consistent (ZooKeeper, etcd, HBase, Spanner).
- **AP**: keep answering, possibly stale, reconcile later (Dynamo-style: Cassandra, Riak, DynamoDB defaults, DNS).

PACELC completes the picture: *if Partition, trade Availability vs Consistency; Else (normal operation), trade Latency vs Consistency*.



- Cassandra: **PA/EL** — available under partition, low latency normally (async).
- Spanner, CockroachDB: **PC/EC** — consistent under partition, pays latency normally (sync/consensus).
- MongoDB (default): **PC/EC**-leaning (primary steps down during partition).
- DynamoDB: **PA/EL** default with per-read strong option.

Internal Working

During a partition, a CP system with a leader on the minority side must stop accepting writes (it can't reach quorum); the majority side elects a new leader and continues — so "unavailability" is usually partial and lasts seconds. An AP system lets both sides accept writes, creating divergent versions that must be merged after healing (vector clocks, LWW timestamps, CRDTs, or application-level merge like Amazon's shopping cart union).

Real Production Example

Amazon's original Dynamo paper chose AP for the shopping cart: it is better to accept an add-to-cart on a stale replica and merge carts later than to show an error at the moment a customer wants to buy. Google Spanner chooses CP plus TrueTime to get externally consistent transactions — and eats the write latency of cross-region Paxos because ad billing data must be correct.

Common Mistakes

- "Choose 2 of 3." Wrong — you can't choose CA in a distributed system; partitions happen. CAP only constrains behavior *during* partitions.

- Ignoring that CAP says nothing about latency in normal operation — that's why PACELC exists and why interviewers love hearing it unprompted.
- Treating a whole database as CP or AP when many (DynamoDB, Cassandra, Cosmos DB) are *tunable per request*.

Interview Questions

1. Why can't you sacrifice partition tolerance?
2. Classify Cassandra, ZooKeeper, and Spanner under PACELC and justify.
3. Your two datacenters lose connectivity for 5 minutes. Walk through what your design does.

Best Practices

- In interviews, phrase it as: "During partitions this component chooses X because business requirement Y; in normal operation I replicate asynchronously/synchronously because of the latency budget."

Hands-on Design Exercise

Design the partition behavior for a ticket-booking system spanning 2 regions: what happens to seat purchases in each region when the inter-region link dies? (*Expect: partition seat inventory by region/event home region — CP within the owning region, degrade the remote region to read-only or queued intents.*)

1.6 SLA, SLO, SLI

Why Interviewers Ask This

They reveal operational maturity — whether you've actually run a service. Staff-level answers connect these to error budgets and engineering decisions (deploy velocity vs reliability).

Core Concept

SLI	→ what you MEASURE	"successful requests / total requests"
SLO	→ what you TARGET	"99.9% success over rolling 30 days"
SLA	→ what you PROMISE	"99.5% or you get service credits" (contract, penalty)

SLA < SLO always: promise externally less than you target internally, so the buffer absorbs bad months.

Error budget = 1 – SLO. At 99.9% monthly, you may "spend" 43.8 minutes of unavailability. Budget healthy → ship fast, take risks. Budget exhausted → freeze features, work on reliability. This converts reliability from a religious argument into a resource-allocation decision — the core idea of Google SRE.

Internal Working

Good SLIs are user-centric and measured as a ratio of good events / valid events at the point closest to the user (load balancer, client-side):

- Availability SLI: non-5xx responses / total
- Latency SLI: requests faster than 300 ms / total (a threshold ratio — better than tracking "p99" directly as SLI because it composes over time windows)
- Freshness/durability SLIs for pipelines and storage

Windows: rolling 28–30 days is standard. Burn-rate alerts fire when the budget is consumed too fast (e.g., 14.4× burn over 1 hour = page; 6× over 6 hours = page; 1× over 3 days = ticket).

Real Production Example

Google SRE runs the error-budget policy: if a product team exhausts the budget, launches halt until reliability work restores it. Stripe publishes API SLAs and is known for 99.999%-level API uptime targets internally. AWS SLAs pay service credits (e.g., EC2 region-level < 99.99% monthly → credits) — note the SLA is weaker than what they actually deliver, by design.

Common Mistakes

- Conflating the three terms (instant mid-level signal).
- 100% SLO targets — impossible and wasteful; every extra nine costs ~10x.
- Measuring SLIs inside the server (misses network/LB failures the user sees).
- Alerting on causes (CPU) instead of symptoms (SLO burn).

Interview Questions

1. Define an SLI/SLO for a payment API and the alerting around it.
2. What is an error budget and how does it change team behavior?
3. Why should the SLA be weaker than the SLO?

Best Practices

- 2–4 SLOs per service, user-journey oriented; review quarterly.
- Multi-window burn-rate alerting (fast burn pages, slow burn tickets).

Hands-on Design Exercise

Write the SLIs, SLOs, and a burn-rate alert policy for a file-upload service (uploads must complete, must be durable, and must be < 5 s for files < 10 MB).

1.7 Load Patterns & Read-heavy vs Write-heavy Systems

Why Interviewers Ask This

The read/write ratio is the single most design-determining number in an interview. Say it out loud in the first ten minutes and derive the architecture from it.

Core Concept

Load patterns:

- **Diurnal**: daily peaks (social apps: evening peak 2–5× trough). Plan capacity for peak, autoscale the difference.
- **Spiky / event-driven**: flash sales, ticket drops, breaking news — 10–100× baseline in seconds. Autoscaling is too slow; you need queues, pre-provisioning, load shedding, waiting rooms.
- **Seasonal**: Black Friday, tax season.
- **Thundering herd**: synchronized clients (cron at :00, cache expiry, push notification sent to 10M users who all open the app). Add jitter everywhere.

Read-heavy (100:1 to 1000:1 — Twitter timelines, Netflix browse, Google search): optimize with caching layers, read replicas, CDNs, denormalization, precomputation (materialized views, fan-out-on-write feeds).

Write-heavy (metrics ingestion, IoT, logging, chat): optimize with partitioning/ sharding, LSM-tree storage engines (Cassandra, RocksDB) that turn random writes into sequential I/O, batching, and buffering through Kafka; keep indexes minimal (every index is write amplification).

Visual Architecture



Real Production Example

Twitter: ~300k QPS timeline reads vs ~6k tweet writes → fan-out-on-write (precompute home timelines into Redis at tweet time), with fan-out-on-read fallback for celebrities. Conversely, Datadog/metrics pipelines are write-dominated: millions of points/sec into Kafka, then into columnar/LSM stores, with reads being rare aggregations.

Common Mistakes

- Designing for average instead of peak (and for peak instead of *spike*).
- Adding read replicas to fix a write bottleneck (replicas replay every write too — they don't add write capacity).
- Caching a write-heavy dataset with low re-read rates (cache hit ratio will be terrible).

Interview Questions

1. How does a 1000:1 read ratio change your database architecture?
2. Design intake for a ticket sale where 1M users click "buy" in the first minute for 50k seats.
3. Why are LSM trees favored for write-heavy workloads? (*sequential I/O, batched memtable flushes; cost: read amplification + compaction*)

Best Practices

- State read:write ratio and peak:average ratio during capacity estimation, always.
- For spikes: queue + shed + waiting room; for diurnal: autoscale; for herds: jitter.

Hands-on Design Exercise

You ingest 500k sensor readings/sec (write-heavy) but ops dashboards query only per-minute aggregates (read-light). Sketch the pipeline and storage choice, and state where you'd batch. (*Expect: Kafka → stream aggregator → columnar/TSDB; raw data to object storage.*)

1.8 Stateless vs Stateful Services

Why Interviewers Ask This

Statelessness is the property that makes horizontal scaling, autoscaling, rolling deploys, and failover trivial. Interviewers check that you know *where you moved the state to* — because the state never disappears, it just relocates.

Core Concept

- **Stateless service:** any instance can serve any request; nothing request-relevant lives only in instance memory. State is externalized to databases, Redis, object storage, or carried in the request (JWT).

- **Stateful service:** correctness depends on local state — databases, WebSocket connection servers, game servers, stream processors with local RocksDB state (Kafka Streams/Flink).

Internal Working

Making a service stateless in practice:

- Sessions → Redis or signed tokens (JWT) instead of in-memory session maps.
- File uploads → stream to S3, not local disk.
- In-memory caches are OK only as *disposable* performance layers.

When state is unavoidable, you manage it explicitly: **sticky routing** (consistent hashing of user → connection server), **replication + failover** (databases), **partitioned ownership** (each instance owns a shard of state, with rebalancing protocols — this is how Kafka consumer groups and Flink operate).

Visual Architecture



Real Production Example

Netflix's API/edge tiers are stateless and autoscale by thousands of instances daily. WhatsApp's chat servers are stateful (millions of long-lived TCP connections per box in its famous Erlang setup); a crashed chat server drops connections, clients reconnect through the LB, and durable message state lives in the message store — the connection state is sacrificial, the message state is not.

Advantages / Trade-offs

- Stateless: trivial scaling/deploys/failover; but per-request latency to fetch state externally, and the external state store becomes the thing you must scale.
- Stateful: data locality (fast), enables long-lived connections and stream processing; but rebalancing, draining before deploys, and failover protocols are on you.

Common Mistakes

- "Stateless" services with hidden state: local caches that matter for correctness, sticky sessions in the LB nobody remembers, cron state on one box.
- Forgetting that WebSockets make the connection tier inherently stateful — you need a routing layer (who holds user X's socket?) and a pub/sub backplane (Redis pub/sub, Kafka) to deliver messages across servers.
- Moving all state to Redis and then treating Redis as infinitely scalable/free.

Interview Questions

1. How do you deploy a new version of a WebSocket server fleet without dropping a million connections cleanly? (*connection draining, client reconnect with backoff+jitter, resume via session state in Redis*)
2. Where does session state go when you make the app tier stateless, and what's the trade-off of JWT vs server-side sessions? (*JWT: no lookup, but revocation is hard; Redis session: revocable, but adds a dependency*)
3. How do Kafka Streams / Flink handle stateful failover? (*changelog topics / checkpoints restore local state on another node*)

Best Practices

- Default stateless; document and justify every piece of state that stays local.
- For stateful tiers: plan the rebalance story (consistent hashing), the drain story (deploys), and the recovery story (where is the source of truth?) before anything else.

Hands-on Design Exercise

Design the connection tier for a chat app with 10M concurrent WebSocket users: how many servers (assume 100k conns/server), how a message from user A reaches user B on another server, and what happens when one server dies. (Expect: ~100 servers + headroom, registry mapping user → server in Redis, pub/sub or direct server-to-server routing, reconnect + resume from message store on failure.)

Module 1 Cheat Sheet

LATENCY vs THROUGHPUT	L = time per op (report p50/p95/p99); T = ops/sec Little's Law: concurrency = T × L. Tail compounds w/ fan-out.
SCALING	Up: simple, ceiling, SPOF. Out: needs stateless/partitionable. Amdahl: serial fraction caps speedup. Scale reads → then writes.
AVAILABILITY	Serial: multiply (chain kills you). Parallel: $1 - \prod(1 - a)$. 99.9% = 8.8h/yr 99.99% = 53min/yr 99.999% = 5min/yr (each 9 ≈ 10x cost)
DURABILITY	Replication ≠ backup. WAL+fsync, 3x replicas, erasure coding, restore drills.
CONSISTENCY SPECTRUM	linearizable > sequential > causal > RYW > monotonic > eventual Quorum: R+W>N ⇒ strong reads. Mix levels per entity.
CAP	Partition ⇒ choose C (refuse) or A (serve stale). No "CA".
PACELC	Else: Latency vs Consistency. Cassandra PA/EL; Spanner PC/EC.
SLI/SLO/SLA	measure / target / promise (SLA < SLO). Error budget = 1-SLO. Burn-rate alerts on symptoms, measured at the edge.
LOAD PATTERNS	diurnal-autoscale; spike-queue+shed+prewarm; herd-jitter.
READ-HEAVY	cache, CDN, replicas, precompute (fan-out-on-write).
WRITE-HEAVY	partition, LSM engines, batch, buffer via Kafka, few indexes.
STATELESS	externalize state (Redis/DB/S3); enables scale & deploys.
STATEFUL	sticky routing, ownership+rebalance, drain, changelog recovery.

Top Interview Questions (Module 1)

1. p50 20 ms / p99 2 s — diagnose.
2. Size a connection pool with Little's Law.
3. Five 99.9% serial dependencies — availability ceiling?
4. CP vs AP for a booking system, per component.
5. PACELC classification of Cassandra vs Spanner.
6. Define SLO + error budget policy for a payments API.
7. Architecture for 1000:1 read ratio.
8. Flash-sale intake design.
9. Deploying stateful WebSocket fleets.
10. Why replication is not backup.

Common Mistakes Recap

Averages instead of percentiles • "infinite" horizontal scaling with a hidden serial bottleneck • five-nines promises without costing them • one consistency level for the whole system • "CA systems" • SLI measured inside the server • designing for average load • hidden state in "stateless" services.

Mock Interview Exercise

"Design a global product-catalog read API: 200k QPS reads, 50 writes/sec, p99 < 80 ms worldwide, catalog must never lose updates." Spend 20 minutes. You should produce: read:write ratio observation → CDN + regional Redis + read replicas; async cross-region replication (PA/EL, eventual reads are fine for catalog); strong consistency

only on the write path to the leader; SLOs + burn alerts; and the failure story for a region loss. Grade yourself against the rubric in the introduction.

Module 2 — Networking

Interview-relevant networking only: the protocols and middleboxes that appear on every system design whiteboard, with enough internals to survive a deep dive.

2.1 TCP, UDP, and QUIC

Why Interviewers Ask This

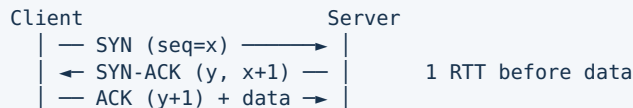
Transport choice determines latency behavior, connection cost, and what your load balancers and proxies must do. It comes up directly ("why QUIC?") and indirectly ("why is your p99 bad on mobile?").

Core Concept

- **TCP**: connection-oriented, reliable, ordered byte stream. Handshake, retransmission, flow control, congestion control. The default for everything.
- **UDP**: connectionless datagrams. No delivery/order guarantees, minimal overhead. Used where the application supplies its own semantics: DNS, video/game real-time traffic, QUIC.
- **QUIC**: a reliable, multiplexed, encrypted transport built *on* UDP in user space. Fixes TCP's head-of-line blocking and slow handshakes; the transport under HTTP/3.

Internal Working

TCP connection setup:



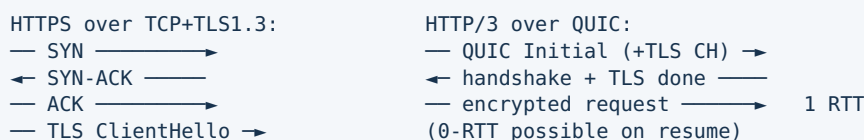
Then TLS 1.3 adds 1 more RTT (TLS 1.2 added 2). So a fresh HTTPS-over-TCP request costs 2–3 RTTs before the first byte. On a 100 ms RTT mobile link, that's 200–300 ms of pure setup — this is why connection reuse and QUIC matter.

Reliability: sequence numbers + cumulative/selective ACKs + retransmission timers. **Flow control**: receiver advertises a window. **Congestion control**: sender probes network capacity — slow start (exponential growth of cwnd), congestion avoidance (linear), loss → back off. Modern algorithms: CUBIC (loss-based, default), BBR (model-based, Google — keeps queues short, better on lossy/long-fat links).

TCP head-of-line (HOL) blocking: one lost packet stalls delivery of *all* subsequent bytes in the stream, even if they belong to logically independent requests. This is the flaw HTTP/2 multiplexing inherits and QUIC fixes.

QUIC: streams are independent (loss in stream A doesn't stall stream B), TLS 1.3 is fused into the handshake (1 RTT, or **0-RTT** for resumed sessions), and connections are identified by a connection ID, not the 4-tuple — so a phone hopping from Wi-Fi to LTE keeps its connection alive (**connection migration**).

Visual Architecture





Real Production Example

Google serves most Chrome/YouTube traffic over QUIC; Meta and Cloudflare report double-digit p95 latency wins on mobile from HTTP/3, mostly from handshake savings and loss isolation. Uber uses QUIC in its mobile apps for exactly the lossy-network/connection-migration reasons.

Advantages / Trade-offs

- TCP: universal, kernel-optimized, middlebox-friendly. But HOL blocking, slow setup, connection breaks on IP change.
- UDP: minimal latency, multicast-capable. But you own reliability/congestion if you need them.
- QUIC: fast setup, no transport HOL blocking, migration, always encrypted. But higher CPU (user-space, per-packet crypto), some networks throttle/block UDP (need TCP fallback), harder to inspect/debug on the wire.

Common Mistakes

- "UDP is unreliable so it's bad" — real-time media *prefers* dropping a late packet over retransmitting it.
- Thinking QUIC is just "HTTP over UDP" without naming the actual wins (handshake RTTs, stream independence, migration).
- Ignoring 0-RTT replay risk: 0-RTT data can be replayed by an attacker, so it must carry only idempotent requests.

Scaling / Failure / Monitoring

- Long-lived TCP at scale: tune keepalives, beware NAT idle timeouts (~5 min) killing "idle" connections; LBs must handle millions of concurrent connections (epoll, SO_REUSEPORT).
- Monitor: retransmission rate, RTT distributions, connection setup failures, cwnd behavior for bulk transfer paths. `ss -ti`, packet captures, eBPF tooling.
- Failure: SYN floods (mitigate with SYN cookies), ephemeral port exhaustion on high-connection-rate proxies (fix with keepalive/pooling), bufferbloat inflating latency.

Interview Questions

1. Walk through what happens on the wire for a first-ever HTTPS request. How many RTTs, and how do you cut them?
2. Why does packet loss hurt HTTP/2 more than HTTP/3?
3. When would you build on UDP directly?

Follow-ups

- "Why can QUIC survive a client IP change but TCP can't?" (connection ID vs 4-tuple)
- "CUBIC vs BBR — when does it matter?" (long-fat/lossy links; video delivery)

Best Practices

- Reuse connections aggressively (keepalive, pooling); handshakes are the tax.
- Enable HTTP/3 with TCP fallback at the edge; keep 0-RTT for idempotent GETs only.

Hands-on Exercise

Your mobile p95 API latency is 900 ms while Wi-Fi p95 is 250 ms. RTT on mobile is 120 ms. Itemize where the time likely goes and what protocol changes recover it.

2.2 HTTP/1.1 → HTTP/2 → HTTP/3, and HTTPS

Why Interviewers Ask This

Every external API in your design speaks HTTP. Version differences explain real performance behavior (multiplexing, HOL blocking) and real operational choices (gRPC needs HTTP/2; browsers cap HTTP/1.1 connections).

Core Concept & Internal Working

HTTP/1.1 (text): one request at a time per connection (pipelining exists but is dead due to HOL + broken proxies). Browsers open ~6 parallel connections per host as a workaround; the era of domain sharding, spriting, and bundling was all HTTP/1.1 mitigation.

HTTP/2 (binary, 2015): a single TCP connection carries many concurrent **streams** (multiplexing). Frames (HEADERS, DATA) are interleaved. **HPACK** header compression (huge win — cookies/headers repeat on every request). Stream prioritization. Server push (deprecated in practice). Remaining flaw: all streams share one TCP stream, so one lost TCP packet stalls every stream (transport-level HOL).

HTTP/3 (2022): same HTTP semantics mapped onto QUIC streams — loss on one stream doesn't stall others; QPACK replaces HPACK (designed for out-of-order delivery); handshake savings from QUIC.

HTTPS = HTTP over TLS. TLS 1.3 handshake: client sends supported ciphers + key share; server replies with its cert + key share; both derive session keys (1 RTT). Certificates chain to a trusted CA; SNI tells the server which cert to present when many domains share an IP. Session resumption / 0-RTT skip round trips on reconnection. Everything is encrypted: confidentiality + integrity + server authentication.

Visual Architecture

HTTP/1.1:	conn1: [req1—resp1][req2—resp2] conn2: [req3—resp3]	serialized per connection (browser opens ~6)
HTTP/2:	one TCP conn: [h1][d3][h2][d1][d2][d3] x lost TCP packet → ALL streams stall	interleaved frames, streams 1..N
HTTP/3:	one QUIC conn: stream1 [x] stream2 [x] ✓ lost packet stalls only its own stream	independent recovery per stream

Real Production Example

Cloudflare/Fastly/Akamai edges negotiate h3 with browsers and often speak HTTP/2 or 1.1 to origins. gRPC (Google, Netflix, Uber internal RPC) mandates HTTP/2 for multiplexed streaming. Meta measured meaningful engagement wins moving mobile traffic to HTTP/3.

Common Mistakes

- "HTTP/2 solves head-of-line blocking" — it solves *application-layer* HOL, and moves it to the transport layer. Naming this distinction is a strong senior signal.
- Recommending domain sharding on HTTP/2 (it's an anti-pattern there — kills one-connection benefits).
- Forgetting that a single HTTP/2 connection between two proxies can be a throughput bottleneck (per-connection flow control windows) — internal L7 proxies often open multiple.

Monitoring & Failure

- Track protocol mix, handshake failures, TLS version/cipher distribution, cert expiry (a classic self-inflicted outage — automate renewal, alert 30/14/7 days out).
- HTTP/2 stream resets and GOAWAYS signal server overload or LB connection recycling.

Interview Questions

1. Why did HTTP/2 need to be binary and framed?
2. Your site is slow for users with 2% packet loss — compare behavior on h1/h2/h3.
3. What problems does TLS 1.3 solve over 1.2? (1 RTT, forward secrecy by default, dead ciphers removed)

Best Practices

- Terminate TLS at the edge/LB; use modern certs with automated rotation (ACME).
- h3 at the edge, h2 for internal gRPC, keep connections warm.

Hands-on Exercise

A dashboard page fires 40 XHRs to one API host. Explain concretely what changes in connection count, header bytes, and stall behavior as you move h1.1 → h2 → h3.

2.3 DNS

Why Interviewers Ask This

DNS is the first hop of *every* request, a global eventually-consistent database everyone already runs, and the crudest-but-most-universal traffic steering and failover tool.

Core Concept & Internal Working

Hierarchical, cached name resolution:

```
Browser cache → OS cache → Recursive resolver (ISP/8.8.8.8/1.1.1.1)
    | (on miss)
    ▼
Root servers (.) → TLD servers (.com) → Authoritative NS (example.com)
                                   |
                                   A/AAAA 93.184.216.34 (TTL 300)
```

Record types you must know: **A/AAAA** (IP), **CNAME** (alias), **NS**, **MX**, **TXT** (verification, SPF), **SRV**, **ALIAS/ANAME** (apex CNAME workaround). **TTL** controls cache duration = your failover speed vs query volume trade-off.

DNS-based traffic management (Route 53, NS1): weighted routing (canary), latency-based routing (nearest region), geo routing (data residency), health-checked failover records. **Anycast** (same IP announced from many locations via BGP) is how root servers and CDNs make DNS itself fast and DDoS-resistant.

Real Production Example

The 2016 Dyn DDoS took down Twitter/GitHub/Spotify — not their servers, their DNS provider. Lesson repeated in interviews: DNS is a dependency with its own availability, use multiple providers for critical domains. Netflix and AWS use Route 53 health checks + low TTLs for region failover.

Common Mistakes

- Believing TTL=60 guarantees 60-second failover — resolvers and OSes disrespect TTLs; some clients (JVMs with default settings!) cache forever. Plan failover assuming a long tail of stale clients.
- Using DNS as a load balancer and expecting even distribution — resolver caching skews it badly.
- Forgetting negative caching (NXDOMAIN cached) — a typo'd record hurts for a while.

Monitoring, Failure, Scaling

- Monitor resolution latency and failure rate from multiple vantage points; alert on serving NXDOMAIN/SERVFAIL spikes.
- Failures: expired domains, DNSSEC misconfiguration, provider outage, cache poisoning (mitigated by DNSSEC, 0x20 randomization).

Interview Questions

1. Trace a cold `api.example.com` resolution end to end.
2. Design DNS-based multi-region failover — what limits how fast it works?
3. Why anycast for DNS?

Best Practices

- TTL 30–300 s for records you may need to move; longer for stable ones.
- Dual DNS providers for tier-0 domains; automate record changes; never hand-edit.

Hands-on Exercise

Design DNS for a service in 3 regions with latency routing and automatic failout of an unhealthy region, and state the worst-case user impact window during a region failure.

2.4 CDN

Why Interviewers Ask This

Any read-heavy or global design should put a CDN in front. Interviewers probe: what do you cache, how do you invalidate, and what happens on the miss path.

Core Concept & Internal Working

A CDN is a globally distributed cache + smart network edge. Users are routed to the nearest **PoP** (via anycast or DNS). Edge cache hit → served in ~10–30 ms. Miss → fetch from a **shield/parent** tier (regional cache) → origin.

```
user → edge PoP (300+ worldwide) —miss→ shield PoP —miss→ origin
      | cache-control, TTL,           | collapses many edge
      ▼ stale-while-revalidate        ▼ misses into 1 origin fetch
      HIT ~90–99% for static           origin sees ~1% of traffic
```

Key mechanics: cache keys (URL + selected headers — keep it minimal or hit rate dies), `Cache-Control/s-maxage`, invalidation (purge APIs are seconds-fast but global purges are expensive; prefer **versioned URLs**: `app.v42.js` — immutable, cache forever), **request collapsing** (one origin fetch for N concurrent misses — stampede protection), **stale-while-revalidate** and **serve-stale-on-error** (origin down ≠ site down). Modern CDNs also run edge compute (Cloudflare Workers, Lambda@Edge) and terminate TLS/h3 near the user. Dynamic (uncacheable) traffic still benefits: TLS terminated at the edge + warm, optimized edge → origin connections cut 1–2 RTTs.

Real Production Example

Netflix built its own CDN (**Open Connect**): appliances installed inside ISPs, pre-loaded with the catalog during off-peak hours — video never crosses the public internet backbone at peak. This is the canonical "we outgrew commercial CDNs" story. Meanwhile, image-heavy products (Instagram, Airbnb) serve virtually all media via CDN with versioned URLs and long TTLs.

Common Mistakes

- "CDN is only for static files" — API GETs with short TTLs and edge logic are routinely cached.
- Purge-based invalidation as the primary strategy instead of versioned/immutable URLs.
- Caching personalized responses (cookie in the cache key explosion, or worse, leaking user A's data to user B — a real class of incident).

Monitoring, Failure, Scaling

- Watch cache hit ratio (per content class), origin offload %, origin error rate, PoP-level latency.
- Failure: cache poisoning via unkeyed headers; a global CDN config push taking sites down (Fastly 2021, Cloudflare incidents — even CDNs need canaried config).

Interview Questions

1. How do you invalidate CDN content the instant a product price changes? (short TTL or purge for price API; versioned assets elsewhere; or don't cache price — cache the shell)
2. What's request collapsing and why does it matter for a viral video?
3. Design the miss path so origin survives a global cache flush.

Best Practices

- Immutable, versioned asset URLs + `Cache-Control: max-age=31536000, immutable`.
- Shield tier on; serve-stale-on-error on; keep cache keys minimal and explicit.

Hands-on Exercise

A news site gets 2M RPS at breaking-news peak; article pages change occasionally, the homepage every 30 s. Design TTLs, invalidation, and the origin protection story.

2.5 Load Balancer, Reverse Proxy, API Gateway (and NAT)

Why Interviewers Ask This

These three boxes appear in every diagram and candidates routinely blur them. Being precise about L4 vs L7, and about what belongs in a gateway vs a service, is an easy senior signal.

Core Concept

- **Load balancer**: distributes traffic across replicas. **L4** (transport: forwards TCP/UDP by IP:port, blazing fast, no content awareness — AWS NLB, IPVS, Maglev) vs **L7** (application: parses HTTP, routes by path/host/header, retries, TLS termination — Envoy, NGINX, ALB, HAProxy).
- **Reverse proxy**: a server-side intermediary that terminates client connections and forwards to backends — TLS termination, caching, compression, buffering slow clients. An L7 LB is a reverse proxy with balancing.
- **API gateway**: a reverse proxy specialized for API management — authentication/JWT validation, rate limiting, quotas, routing to microservices, request transformation, canary splitting, analytics (Kong, Apigee, AWS API Gateway, Zuul, Envoy-based gateways).
- **NAT**: rewrites private IPs to public at the network boundary. Interview relevance: NAT gateways are how private subnets reach the internet (egress), they keep connection state (idle timeouts kill long-lived connections), and they're why peer-to-peer/WebRTC needs STUN/TURN hole-punching.

Internal Working

Balancing algorithms: round robin, weighted RR, **least connections** (best general default for uneven request costs), **consistent hashing** (cache affinity, sticky WebSockets), **power of two choices** (pick 2 random, take less loaded — near-optimal with $O(1)$ state, used in Envoy/HAProxy). Health checks: active (probe /healthz) + passive (eject on error rate — outlier detection). High availability of the LB itself: pairs with VRRP/keepalived, or L4 via ECMP/anycast + consistent hashing (Google Maglev, Meta Katran — connection-stable despite router hashing).



Real Production Example

Netflix Zuul (now Envoy-based gateways elsewhere) fronts all API traffic: auth, routing, canary, region failover, and — crucially — load shedding by priority during incidents. Google's Maglev L4 balancer uses consistent hashing so any Maglev can handle any packet, making the LB tier itself horizontally scalable and hitless under LB failure.

Common Mistakes

- Putting business logic in the gateway (it becomes a shared monolith with a platform team bottleneck) — keep it to cross-cutting concerns.
- Ignoring the LB as SPOF ("draw one box" — say how the LB tier itself is redundant).
- Retrying non-idempotent requests at the proxy (duplicate payments!) — retries only on idempotent methods or with idempotency keys.
- Sticky sessions as a crutch for stateful apps instead of externalizing state.

Monitoring, Failure, Scaling

- Golden signals at the LB: RPS, error rate by upstream, latency percentiles, active connections, healthy-host count. The LB is the best vantage point in the system.
- Failure: health-check flapping (add hysteresis), slow drains during deploys, connection pile-ups when upstreams slow down (cap queue + shed early).

Interview Questions

1. L4 vs L7 — when is each the right choice? (L4: raw TCP/UDP, extreme scale, WebSocket passthrough; L7: routing, retries, TLS, canary)
2. How do you make the load balancer tier itself highly available?
3. What belongs in an API gateway vs in each service?

Best Practices

- Two tiers: L4 at the edge for scale/DDoS, L7 behind it for smarts.
- Least-connections or P2C over round robin; outlier ejection on; drain before deploy.
- Idempotency-aware retries with budgets (e.g., ≤ 1 retry, only 5xx/connect-fail, exponential backoff + jitter).

Hands-on Exercise

Design the traffic path for a 500k-RPS public API across 3 regions: DNS policy, L4 edge, L7 gateway features (auth, rate limit, canary), and what fails over at each layer.

2.6 WebSocket

Why Interviewers Ask This

Any real-time design (chat, presence, live scores, collaborative editing, trading) needs a server-push story, and WebSocket is the default answer — with a stateful connection tier whose scaling story interviewers love to probe.

Core Concept & Internal Working

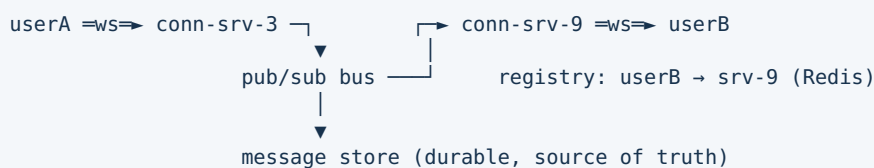
WebSocket = a persistent, full-duplex, message-framed channel over a single TCP connection, established by an HTTP/1.1 **Upgrade** handshake:

```
GET /chat HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: <nonce>    →    HTTP/1.1 101 Switching Protocols
                                Sec-WebSocket-Accept: <hash>
                                ...then raw framed messages both ways, forever
```

After 101, it's no longer HTTP — proxies/LBs must support upgrade passthrough and long-lived idle connections (tune idle timeouts; NAT/LB defaults of 60 s will sever "quiet" sockets — send ping/pong heartbeats ~every 30 s).

Alternatives to know and compare: **SSE** (server → client only, plain HTTP, auto-reconnect built in — great for feeds/notifications, simpler than WS), **long polling** (fallback, high overhead), **gRPC streaming** (service-to-service), **WebTransport/HTTP3** (emerging).

Scaling model (the part interviews grade): the connection tier is stateful. - Capacity: a tuned server holds 100k–1M+ mostly-idle connections (memory-bound: ~ tens of KB per conn). - Routing: a **registry** (Redis) maps `userId` → server, or you use consistent hashing at the LB; cross-server delivery goes through a **pub/sub backplane** (Redis pub/sub, Kafka, or a mesh of server-to-server streams). - Deploys/failover: drain connections gracefully; clients reconnect with backoff + jitter (else you thundering-herd yourself) and resume via cursor/sequence number against the durable message store.



Real Production Example

Slack's real-time messaging runs on a WebSocket edge fleet with regional presence and a pub/sub fabric; Discord holds millions of concurrent connections per cluster on an Elixir/Rust gateway tier, with client resume protocols (session + sequence numbers) so a gateway restart doesn't lose messages — messages are durable in the store, sockets are sacrificial.

Common Mistakes

- Treating the socket as the source of truth: delivery must be backed by a durable store + resume cursor, or restarts lose messages.
- Forgetting heartbeats and LB idle-timeout tuning (mystery disconnects every 60 s).
- No reconnect jitter → a deploy causes a reconnect storm that DDoSes your own auth service.

- Choosing WS when SSE suffices (one-directional feed) — extra ops complexity for nothing.

Monitoring & Failure

- Metrics: concurrent connections per server, connect/disconnect rates, heartbeat failures, message delivery latency, backlog per client.
- Failure: one server death drops 100k+ connections at once — the reconnect surge is the real event to engineer for (jitter, token bucket at LB, warm capacity).

Interview Questions

1. WebSocket vs SSE vs long polling — pick per use case.
2. How does a message reach a user connected to a different server?
3. Deploy the connection fleet with zero message loss — walk me through it.

Best Practices

- Heartbeats + jittered reconnect + resume cursors; durable store as truth.
- Isolate the connection tier from business logic (dumb pipes, smart services).

Hands-on Exercise

Size and design the WebSocket tier for 5M concurrent users, average 1 msg/s per 1,000 users, with < 1 s delivery p99 and zero loss across deploys.

2.7 gRPC

Why Interviewers Ask This

It's the default for internal service-to-service RPC at scale, and comparing REST vs gRPC — with actual reasons — is a standard question.

Core Concept & Internal Working

gRPC = RPC framework over **HTTP/2** using **Protocol Buffers** as the IDL and wire format.

- **Contract-first:** `.proto` files define services/messages; codegen produces typed clients/servers in every language — the contract is the documentation.
- **Protobuf encoding:** binary tag-length-value; field numbers (not names) on the wire → compact (often 3–10× smaller than JSON) and fast to parse; unknown fields are ignored → forward/backward compatibility rules (never reuse/renumber fields, only add).
- **Four call types:** unary, server-streaming, client-streaming, bidirectional streaming (multiplexed as HTTP/2 streams on one connection).
- Built-ins interviewers expect you to name: **deadlines propagated across the call chain** (each hop knows remaining budget — the killer feature vs ad-hoc REST timeouts), cancellation propagation, per-call metadata (auth), client-side load balancing (pick-first/round-robin/lookaside), interceptors (auth, retries, telemetry), and status codes designed for retry semantics (UNAVAILABLE retryable, INVALID_ARGUMENT not).

```
.proto —codegen→ client stub (Go)          server (Java)
order = svc.GetOrder(id, deadline=80ms)  ==HTTP/2==> handler
      ← binary protobuf frames, one TCP conn, N concurrent streams —
```

Limitations to volunteer: browsers can't speak native gRPC (need gRPC-Web or a REST/JSON transcoding gateway); binary payloads aren't grep-able (need reflection/ tooling); L7 infrastructure must be HTTP/2-aware, and

long-lived HTTP/2 connections need **client-side or lookaside LB** (a naive L4 LB pins all requests of a client to one backend — the classic "gRPC doesn't balance" incident).

Real Production Example

Google internally (Stubby → gRPC) runs ~10¹⁰ RPCs/sec on this model. Netflix, Uber, Square, and Dropbox use gRPC for inter-service traffic; Uber pairs it with a service mesh so Envoy sidecars handle the HTTP/2-aware balancing. Public APIs usually stay REST/JSON (browser + ecosystem reach) — "gRPC inside, REST outside" is the standard answer.

Common Mistakes

- "gRPC is faster because binary" — the deeper wins are multiplexing, deadline propagation, streaming, and typed contracts.
- Breaking proto compatibility (renumbering fields, changing types).
- Load balancing gRPC with a connection-level L4 LB and wondering why one pod is hot.
- No deadlines: a missing deadline turns one slow service into fleet-wide thread exhaustion.

Monitoring & Failure

- Per-method RPS, latency, and status-code metrics come nearly free via interceptors; deadline-exceeded and UNAVAILABLE rates are your early-warning signals.
- Debug with grpcurl, server reflection, and channel-state logs.

Interview Questions

1. REST vs gRPC — decision criteria? (audience: browser/public vs internal; streaming needs; contract rigor; ecosystem)
2. How do deadlines propagate and why does that matter? (each hop subtracts elapsed time; prevents useless downstream work + resource exhaustion)
3. How do you evolve a proto without breaking old clients?

Best Practices

- Deadlines on every call; retries only on idempotent methods with backoff and retry budgets; interceptors for auth/tracing/metrics; reserve removed field numbers.

Hands-on Exercise

Define the `.proto` for an inventory service (get, batch-get, watch-stream), choose deadlines per method, and describe the LB setup in Kubernetes (headless service + client-side LB, or Envoy sidecar).

Module 2 Cheat Sheet

TCP	3-way handshake (1 RTT) + TLS1.3 (1 RTT). Reliable ordered stream. HOL blocking. CUBIC/BBR congestion control. Keepalive vs NAT timeouts.
UDP	No guarantees, no handshake. DNS, media, games, QUIC substrate.
QUIC	TLS fused (1/0-RTT), independent streams (no transport HOL), connection ID → migration. UDP-based, user space. 0-RTT = idempotent only.
HTTP/1.1	1 req at a time/conn → 6 conns, bundling era.
HTTP/2	Binary frames, multiplexed streams, HPACK. TCP HOL remains.
HTTP/3	h2 semantics over QUIC. Loss isolation per stream. QPACK.
HTTPS/TLS	1.3 = 1 RTT, forward secrecy. Certs→CA chain, SNI, automate rotation.
DNS	Hierarchy + caches. TTL = failover speed. Route: weighted/latency/geo/failover. Anycast. Clients disrespect TTLs. Dual providers for tier-0.
CDN	Edge→shield→origin. Versioned immutable URLs > purges. Request collapsing, stale-while-revalidate, serve-stale-on-error. Minimal cache keys.

LB	L4 (fast, opaque) vs L7 (routing/retries/TLS). Least-conn/P2C/consistent hash. Health checks + outlier ejection. LB tier itself: ECMP/anycast/VRRP.
REV PROXY	TLS term, caching, buffering. Gateway = proxy + auth/rate limit/quotas/routing. Keep business logic OUT.
NAT	Private→public rewrite; idle timeouts kill quiet conns; P2P needs STUN/TURN.
WEBSOCKET	HTTP Upgrade→101→full duplex. Stateful tier: registry + pub/sub backplane, heartbeats, jittered reconnect, resume cursor, durable store = truth. SSE for one-way feeds.
gRPC	proto contract + HTTP/2 + streaming + propagated deadlines. Client-side/lookaside LB. gRPC inside, REST outside. Never renumber fields.

Top Interview Questions (Module 2)

1. First-ever HTTPS request: every RTT on the wire. 2. h1 vs h2 vs h3 under packet loss. 3. Design multi-region DNS failover and its limits. 4. CDN invalidation strategy for a price change. 5. L4 vs L7 and making the LB tier HA. 6. Gateway responsibilities vs service responsibilities. 7. Scale WebSockets to 5M concurrent.
2. REST vs gRPC decision. 9. Why gRPC misbehaves behind L4 LBs. 10. What breaks long-lived connections in production (NAT/LB idle timeouts) and the fix.

Common Mistakes Recap

Blurring LB/proxy/gateway • "h2 fixed HOL" • purge-first CDN strategy • caching personalized content • no heartbeats/jitter on WebSockets • business logic in the gateway • retrying non-idempotent requests • gRPC behind connection-level LB • treating DNS TTLs as guarantees.

Mock Interview Exercise

"Design the global edge for a real-time sports app: 20M concurrent score subscribers, REST API for everything else, worldwide, p99 API < 150 ms." Expected shape: anycast/DNS → CDN (cache API GETs w/ short TTL + collapse) → L4 → L7 gateway (auth, rate limit) → services; scores via WebSocket/SSE fan-out tier fed by pub/sub; per-layer failure story; protocol choices justified (h3 edge, gRPC internal).

Module 3 — Caching

Caching is the highest-leverage performance tool in system design and the source of some of the nastiest production incidents (stampedes, hot keys, stale data). Every interview design should state: what is cached, where, with which pattern, which TTL, and how it's invalidated.

3.1 Why Caching Works (and the Cache Hierarchy)

Why Interviewers Ask This

They want the *reasoning*, not the buzzword: caching works because of skewed access patterns — and it fails when those assumptions don't hold.

Core Concept

Caching exploits three facts:

1. **Temporal locality** — recently accessed data is likely accessed again.
2. **Popularity skew (Zipf)** — a tiny fraction of keys gets most traffic. Roughly, the top ~10% of items can serve ~90% of requests. This is why a cache 1/100th the size of your dataset can absorb most reads.
3. **Cost asymmetry** — RAM read (~100 ns local, ~0.5 ms networked) vs DB query (1–50 ms) vs recomputation (arbitrary).

The math that matters: **effective latency** = $\text{hit_ratio} \times \text{cache_latency} + (1 - \text{hit_ratio}) \times \text{miss_latency}$, and **origin load** = $(1 - \text{hit_ratio}) \times \text{traffic}$. Going from 90% → 99% hit ratio cuts origin load **10×** — hit-ratio improvements are logarithmic in effort but multiplicative in effect.

Where caches live (a request may hit all of these):

```
browser cache → CDN edge → API gateway cache → app in-process cache (L1)
→ distributed cache: Redis/Memcached (L2) → DB buffer pool → disk
```

- **Browser cache:** Cache-Control, ETag/If-None-Match (304 revalidation), immutable versioned assets. Free, per-user, zero server cost.
- **CDN cache:** Module 2.4 — shared, geographic.
- **In-process (L1):** nanosecond access, per-instance, small (Caffeine/Guava); risk: N instances = N inconsistent copies; use short TTLs (seconds) or pub/sub invalidation.
- **Distributed (L2):** Redis/Memcached — shared truth for the fleet, millisecond access, the main character of this module.

Common Mistakes

- Caching data with uniform access (no skew) or write-mostly data — hit ratio will be terrible; know *why* your data caches well.
- Ignoring that a cache is a *derived* copy: every cache adds a consistency problem you must explicitly manage.

Interview Questions

1. Your cache hit ratio is 60% — is adding RAM the fix? (Investigate first: TTL too short? key cardinality too high? churn? uniform access?)
 2. Show the load math: 100k QPS at 95% vs 99% hit ratio → 5k vs 1k QPS to the DB.
-

3.2 Redis vs Memcached

Why Interviewers Ask This

The default cache-technology decision; they check you know both and can justify a choice beyond "Redis is popular".

Core Concept & Internal Working

Memcached: pure, multithreaded, in-memory key → bytes LRU cache. Slab allocator (fixed size-class chunks → no fragmentation, but memory waste per item). Scales vertically across cores and horizontally by *client-side* consistent hashing — servers don't know about each other. No persistence, no replication, no data structures. Dead simple = its virtue.

Redis: single-threaded event loop (commands execute atomically — no locks needed; I/O threads since 6.0 help with network overhead). Rich data structures: strings, hashes, lists, **sets**, **sorted sets (leaderboards, rate limiters)**, streams, bitmaps, HyperLogLog (approximate distinct counts in 12 KB), geo indexes. Optional persistence: **RDB** snapshots (fast restart, may lose last minutes) and **AOF** append-only log (fsync policies; slower, safer). **Replication** (async, leader → replica) + **Sentinel** (failover) or **Redis Cluster** (16,384 hash slots sharded across nodes, each slot range with replicas; client redirects via MOVED/ASK). Also: pub/sub, Lua scripting (atomic multi-step ops), transactions, TTL per key, eviction policies (`allkeys-lru` , `allkeys-lfu` , `volatile-ttl` ...).

```
Redis Cluster: keys —CRC16 mod 16384→ slot → node
node A (slots 0–5460)   + replica A'
node B (slots 5461–10922)+ replica B'   client caches slot map;
node C (slots 10923–16383)+ replica C'   MOVED on resharding
```

Real Production Example

Meta runs one of the largest Memcached fleets ever (the classic "Scaling Memcache at Facebook" paper: leases to prevent stale-set races and thundering herds, regional pools, cold-cluster warmup). Twitter/X historically ran both (Twemcache + Redis for timelines — sorted sets are literally the timeline structure). GitHub, Stripe, Airbnb: Redis for cache + rate limiting + queues + locks.

Advantages / Trade-offs

	Memcached	Redis
Model	bytes only, LRU	data structures, scripting
Threading	multithreaded (big single nodes)	single-threaded core (shard for CPU)
Persistence/replication	none	RDB/AOF, replicas, Cluster
Simplicity	extremely simple	more features, more ops surface
Use when	pure look-aside page/object cache at huge scale	anything else: counters, leaderboards, sessions, locks, queues, rate limits

Common Mistakes

- Using Redis as a durable primary store without understanding AOF/RDB loss windows and failover semantics (async replication can lose acknowledged writes on failover).
- Multi-key operations across Redis Cluster slots (not supported without hash tags `{user123}:...`).
- Storing huge values (MBs) — blocks the single thread; keep values small, use pagination/compression.

Monitoring & Failure

- Watch: hit ratio, evictions, memory fragmentation ratio, connected clients, slow log, replication lag, keyspace per node (skew).
- Failure: eviction storms when memory is full (suddenly everything misses), full sync avalanches after failover, `KEYS *` in production blocking the loop (use `SCAN`).

Interview Questions

1. Redis vs Memcached for session storage? (Redis: replication + persistence survive restarts)
2. How does Redis Cluster shard and what breaks? (hash slots; multi-key ops, big values, resharding moves)
3. Is Redis single-threaded a problem? (atomicity win; shard/Cluster for CPU; I/O threads for network)

Best Practices

- Treat cache as ephemeral even with persistence; the DB is the source of truth.
- Small values, TTL on every key, `allkeys-lfu` for skewed workloads, hash tags for related keys, never `KEYS` in prod.

3.3 Caching Patterns: Cache-Aside, Write-Through, Write-Back, Write-Around

Why Interviewers Ask This

The pattern determines your consistency model, failure behavior, and write latency. "Which pattern and why" is a guaranteed follow-up to any cache in your diagram.

Core Concept & Internal Working

Cache-aside (lazy loading) — the default; the *application* manages the cache:

```
READ:  get(key) → HIT: return
        → MISS: read DB → set(key, ttl) → return
WRITE: write DB → DELETE cache key (invalidate, don't update!)
```

Why delete instead of set on write: two concurrent writes can set the cache in the wrong order (stale value wins); delete forces the next read to fetch fresh. There is still a small race (read misses → reads old DB value → write commits + invalidates → stale set lands after the delete). Mitigations: TTL as backstop, or Meta-style **leases** (cache issues a token on miss; a set is only accepted with a valid, uninvalidated token).

Write-through — the cache sits in the write path; every write updates cache + DB synchronously. Cache is always fresh for written keys; write latency = cache + DB; you cache things that may never be read (combine with TTL).

Write-back (write-behind) — write to cache, ACK immediately, flush to DB asynchronously (batched/coalesced). Fastest writes, absorbs bursts, coalesces hot-key updates (great for counters: view counts, likes). Cost: **acknowledged data can be lost** if the cache dies pre-flush → need replicated cache or a durable log in front; complexity of flush ordering and retries. This is also literally how CPU caches and OS page caches work.

Write-around — write to DB only (optionally invalidate); cache fills on read. For write-heavy or write-once-rarely-read data (logs, bulk imports) so writes don't churn the cache.

	write path	read path
cache-aside	DB, then DEL cache	app fills cache on miss
write-through	cache+DB sync (both fresh)	always hit for written keys
write-back	cache ACK → async DB	hit; DB is behind (loss window!)
write-around	DB only	fills on first read (one miss)

Real Production Example

Meta: cache-aside + deletes + leases at extreme scale. DynamoDB Accelerator (DAX): write-through for a managed experience. Gaming leaderboards and metrics pipelines: write-back through Redis with periodic flush. Analytics event ingestion: write-around (events go to the pipeline; nobody rereads them via cache).

Common Mistakes

- Updating (SET) the cache on write in cache-aside → the classic stale-forever race. Say "invalidate on write" in interviews.
- Proposing write-back without addressing the loss window and flush-failure handling.
- One pattern for everything: real systems mix (aside for entities, write-back for counters, around for bulk writes).

Interview Questions

1. Walk through the cache-aside race and how leases fix it.
2. When is write-back worth the risk? (high-frequency counters, burst absorption; loss tolerable or log-backed)
3. Which pattern for a product catalog? For view counters? For log ingestion? (aside / write-back / around)

Best Practices

- Default: cache-aside + delete-on-write + TTL backstop.
- Every key gets a TTL — TTLs are your self-healing mechanism against every missed invalidation bug.

3.4 Cache Invalidation

Why Interviewers Ask This

"There are only two hard things in computer science: cache invalidation and naming things." They want your strategy for keeping derived copies honest, across *all* cache layers.

Core Concept & Internal Working

Strategies, weakest to strongest:

1. **TTL-only**: bounded staleness, zero coordination. Choose TTL = max tolerable staleness. Add $\pm 10\text{--}20\%$ **jitter** so a popular deploy/warmup doesn't expire everything simultaneously.
2. **Explicit invalidation on write**: delete keys when the source changes. Hard parts: knowing *all* keys derived from a row (entity key, list pages, aggregates, search results...), multi-layer fan-out (in-process L1s on N instances + Redis + CDN), and failure between DB commit and cache delete.
3. **Event-driven invalidation**: publish DB changes via **CDC** (Debezium reading the binlog → Kafka) and have consumers invalidate/refresh caches. Decouples producers from every cache; guarantees invalidation eventually happens even if the app crashed post-commit. This is how Meta wires MySQL → memcache invalidation across regions.
4. **Versioned keys**: don't invalidate — change the key (`user:123:v42`, or key = hash of content). Old entries age out via TTL/eviction. Perfect for CDN assets and computed results.

Delivery to in-process caches: pub/sub broadcast (Redis pub/sub, Kafka topic) of invalidation events to all instances.

Common Mistakes

- Forgetting derived/aggregate keys (invalidate `user:123` but not `team:9:members` page cache).
- No TTL backstop ("we invalidate explicitly, TTL unnecessary") — one lost delete = stale forever.

- Invalidating *before* the DB commit (a concurrent read re-fills stale data), or not handling delete failures (retry queue).

Interview Questions

1. Price changes must appear within 5 s across CDN + Redis + in-process caches. Design the invalidation.
2. What is CDC-based invalidation and why does Meta use it?
3. Versioned keys vs purge — when each?

Best Practices

- TTL everywhere + explicit/event-driven invalidation on top; jitter all TTLs; prefer versioned keys wherever the key can encode the version.

3.5 Cache Stampede (Thundering Herd)

Why Interviewers Ask This

It's the classic cache-related outage: the cache *protects* the DB until the moment it doesn't — and the follow-up "what happens when the cache node restarts?" is nearly guaranteed.

Core Concept

A popular key expires (or a cache node dies) → thousands of concurrent requests miss simultaneously → all of them hit the DB with the same expensive query → DB saturates → latency rises → more timeouts and retries → cascade. One key can take down the database.

Internal Working — the four defenses

1. **Request coalescing / single-flight**: only one request per key recomputes; others wait on the in-flight result. In-process: Go `singleflight`, Caffeine loading cache. Cross-fleet: a short-TTL Redis lock (`SET lock:key token NX PX 3000`) — winner recomputes, losers wait/poll or serve stale.
2. **Stale-while-revalidate**: keep serving the expired value while one background worker refreshes. Users see slightly-stale data; the DB sees one query.
3. **Probabilistic early expiration (XFetch)**: each reader refreshes *before* expiry with probability increasing as TTL approaches (`refresh if now -  $\Delta \cdot \beta \cdot \ln(\text{rand}()) \geq \text{expiry}$`), so refreshes desynchronize naturally — no herd forms.
4. **TTL jitter**: prevents synchronized mass expiry across many keys (deploy warms 1M keys with TTL 3600 → all expire together an hour later).

For the cold-cache case (node restart, new cluster): **cache warming** (replay top-N keys before taking traffic) and **gradual traffic ramp**; Meta's cold-cluster warmup fills a new cluster from a warm one rather than from the DB.

without protection:	with single-flight + stale-serve:
10,000 misses → DB ☹️	10,000 requests → 9,999 get stale value
	↳ 1 recomputes → DB (1 query)

Real Production Example

This pattern family is in Meta's memcache paper (leases solve stampede *and* the stale-set race with one mechanism). CDNs implement it as request collapsing. Internet-famous incidents: sites collapsing at cache-flush moments during deploys — the fix is always some combination of the four defenses above.

Interview Questions

1. The cache cluster restarts empty at peak. What happens next and what did you build to survive it?
2. Implement single-flight across 200 app instances. (Redis `SET NX PX` lock + serve-stale for losers)
3. Why does TTL jitter matter after a mass warmup?

Best Practices

- Single-flight + serve-stale on every expensive key; jitter every TTL; cap concurrent DB load with a semaphore regardless (belt and suspenders: the DB should be *unable* to receive a stampede).

3.6 Hot Keys

Why Interviewers Ask This

Sharding distributes *keys*, not *traffic*. A celebrity posts and one Redis shard melts while nine idle. Interviewers use it to test whether you understand skew.

Core Concept & Internal Working

A hot key is a single key receiving a disproportionate share of ops (Zipf tail: celebrity profile, flash-sale product, global config, viral tweet). Since one key lives on one shard (one single-threaded Redis node $\approx$ 100k–1M ops/s ceiling), no amount of horizontal scaling helps — the unit of distribution is the key.

Defenses:

1. **In-process L1 cache** for the hottest keys — even a 1–5 s local TTL absorbs virtually all reads for a viral key (each app instance asks Redis once per second instead of 10k times). The single best fix; costs bounded staleness.
2. **Key replication**: write `key#0..key#N` copies on different shards; readers pick a random replica. $N\times$ read capacity; writes must update all copies (reads-mostly keys only).
3. **Hot-key detection**: sample traffic (Redis `--hotkeys`, client-side sampling, proxy stats) and *automatically* promote hot keys to L1/replicated handling — hotness is dynamic and unpredictable.
4. For hot **counters** (likes on a viral post): shard the counter (`count:{post}:{rand%16}`, read = sum) or batch increments in-process and flush.

```
celebrity key "user:justin" 500k reads/s
└─ shard 3 (one node) ✗
fix: app-local L1 (ttl 2s) → ~N_instances reads/s to Redis ✓
    + replicas key#0..#9 → 10 shards share the remainder ✓
```

Real Production Example

Every social network. Twitter's timeline architecture treats celebrities as a special case end-to-end (Module 13). DynamoDB historically punished hot partitions (throughput was per-partition; adaptive capacity was built to auto-mitigate). Alibaba/Redis operators publish hot-key detection proxies for exactly the flash-sale scenario.

Interview Questions

1. One product page gets 100 $\times$ traffic during a flash sale — where does your cache break and what's the fix?
2. Why doesn't adding Redis shards help a hot key?
3. Design a hot-counter for likes at 1M increments/min.

Best Practices

- Assume skew always; L1 with short TTL for read-hot, sharded counters for write-hot, automated detection because you can't predict virality.

3.7 Distributed Cache Architecture

Why Interviewers Ask This

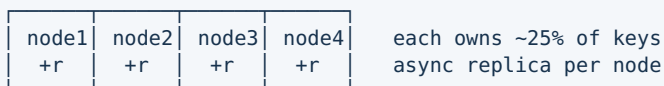
"Design a distributed cache" is itself a full interview question (Module 13.23), and every real design contains one; you must know how the pieces compose.

Core Concept & Internal Working

A distributed cache = many nodes each owning a partition of the keyspace:

- **Partitioning**: consistent hashing (client-side, or via proxy like Twemproxy/Envoy, or server-side like Redis Cluster hash slots). Consistent hashing (Module 11.1) means adding/removing a node remaps only $\sim 1/N$ of keys — no global reshuffle, no fleet-wide cold cache.
- **Replication**: each partition has replicas for read scaling + fast failover; async replication = possible brief staleness/loss on failover (fine for a cache).
- **Eviction**: LRU (recency), **LFU** (frequency — better under scans and skew; Redis `allkeys-lfu`, W-TinyLFU in Caffeine), TTL-based. Memory is the budget; eviction policy decides who pays.
- **Client behavior is part of the design**: timeouts (a cache call slower than ~ 50 ms should fail fast — falling through to the DB is better than queueing), circuit breaking, and *degradation policy*: cache down $\Rightarrow$ serve from DB with a concurrency cap + load shedding, or serve stale/default data. The DB must survive cache death — that's a capacity-planning requirement, not a hope.

```
apps (cache-aside, single-flight, L1)
|   consistent hashing / cluster slot map
▼
```



node2 dies $\rightarrow$ its slots fail over to replica; only $\sim 25\%$ of keys affected;
single-flight + DB concurrency cap absorb the refill.

Interview Questions

1. A cache node dies — trace exactly what happens to hit ratio, DB load, and latency, and what limits the damage.
2. LRU vs LFU — when does LRU fail? (scans/batch jobs evict the hot working set; LFU/W-TinyLFU resists)
3. Where do you put the partitioning logic: client, proxy, or server? (client = fewest hops but N client configs; proxy = central control + extra hop; server (Redis Cluster) = self-managing + client library support needed)

Best Practices

- Consistent hashing + replicas + LFU for skewed workloads; aggressive client timeouts; explicit "cache is down" degradation mode, load-tested.

Module 3 Cheat Sheet

WHY IT WORKS	Zipf skew: ~10% of keys ≈ 90% of traffic. $eff_lat = h \cdot L_hit + (1-h) \cdot L_miss$. 90%→99% hit ratio = 10× less origin load.
LAYERS	browser → CDN → gateway → in-process L1 → Redis/Memcached L2 → DB buffer.
REDIS	single-threaded atomic core, data structures, RDB/AOF, Sentinel/Cluster (16384 slots). Counters, zsets, locks, rate limits, sessions.
MEMCACHED	multithreaded LRU bytes, client-side sharding, dead simple. Pure lookaside.
PATTERNS	aside: read-miss-fill, WRITE ⇒ DB then DELETE key (+leases for race). write-through: sync both (fresh, slower writes). write-back: ACK at cache, async flush (fast, LOSS WINDOW; counters). write-around: DB only, fill on read (bulk/write-heavy).
INVALIDATION	TTL(+jitter) always as backstop → explicit deletes → CDC/event-driven (Debezium→Kafka) → versioned keys (best: never invalidate).
STAMPEDE	single-flight (lock/lease), stale-while-revalidate, probabilistic early refresh, TTL jitter, warmup + ramp for cold clusters, DB concurrency cap.
HOT KEYS	one key = one shard = ceiling. L1 short-TTL cache, key replication #0..N, sharded counters, automated hot-key detection.
DISTRIBUTED	consistent hashing (≈1/N remap), replicas, LFU>LRU under scans, fail-fast timeouts, planned degradation: DB must survive cache death.

Top Interview Questions (Module 3)

1. Design the caching for a product page (layers, pattern, TTLs, invalidation).
2. Cache-aside race + leases. 3. Full cache-cluster restart at peak — survival plan.
3. Celebrity hot key mitigation. 5. Redis vs Memcached, with a use case for each.
4. Write-back for counters: benefits + loss-window handling. 7. CDC-driven invalidation pipeline. 8. LRU vs LFU under a table scan. 9. Multi-layer invalidation within 5 s. 10. Why every key needs a TTL even with perfect invalidation.

Common Mistakes Recap

SET-on-write in cache-aside • no TTL backstop • no stampede protection • ignoring skew/hot keys • huge Redis values • Redis-as-durable-primary without understanding failover loss • uniform TTLs (mass expiry) • no degradation plan for cache death.

Mock Interview Exercise

"Your e-commerce site: 150k QPS product reads, 200 writes/s (price/stock), flash sales spike one product 500×, business tolerates 2 s price staleness." Produce: L1 (1–2 s TTL) + Redis cluster (aside, delete-on-write via CDC, TTL 5 min jittered) + CDN for rendered fragments; single-flight everywhere; sharded stock counters; hot-key auto-promotion; DB capacity floor for cache-loss mode. Then defend the 2 s staleness budget against the CDC path latency.

Module 4 — Database Design

The database is where most interview deep dives end up, because it's where the hard trade-offs live: consistency vs latency, reads vs writes, normalization vs performance. Master this module and you can defend any storage box you draw.

4.1 SQL vs NoSQL

Why Interviewers Ask This

"Which database and why" is asked in *every* interview. The senior answer is driven by access patterns, consistency needs, and scale numbers — not fashion.

Core Concept

- **Relational (PostgreSQL, MySQL, Aurora)**: tables + schema-on-write, ACID transactions, joins, secondary indexes, mature query planners. Model the data once, query it many ways.
- **Document (MongoDB, DynamoDB in doc-ish usage)**: JSON-like aggregates, schema-flexible, data that's read together is stored together. Query patterns should be known upfront.
- **Wide-column (Cassandra, HBase, Bigtable)**: partition key → sorted rows; built to shard from day one; LSM write-optimized; queries only along designed key paths.
- **Key-value (Redis, DynamoDB)**: get/put by key at predictable single-digit-ms latency at any scale.
- **Graph (Neo4j, Neptune)**: relationship traversals (fraud rings, social graphs) that would be N-way self-joins in SQL.
- **Search (Elasticsearch/OpenSearch)**: inverted indexes for text/faceted queries — a *secondary* system fed from the primary.
- **NewSQL (Spanner, CockroachDB, TiDB)**: SQL + ACID *and* horizontal scale via consensus replication; pay with write latency and cost.

The real decision axes:

- | | |
|------------------------------------|---|
| 1. Access patterns known & simple? | → NoSQL fits; ad-hoc/relational queries → SQL |
| 2. Transactions across entities? | → SQL/NewSQL (or redesign to single-partition) |
| 3. Scale: > single-node writes? | → built-to-shard NoSQL or sharded SQL or NewSQL |
| 4. Schema evolution speed | → document stores tolerate variance |
| 5. Team/ops maturity | → Postgres is the best default until proven otherwise |

Internal Working (the storage-engine level interviewers love)

- **B-tree engines** (Postgres, MySQL/InnoDB): read-optimized, in-place pages, great point/range reads, writes cause random I/O + WAL.
- **LSM-tree engines** (Cassandra, RocksDB, LevelDB): writes append to a memtable + WAL, flushed to sorted SSTables, background **compaction** merges them. Sequential-write-optimized (huge ingest throughput); reads may touch several SSTables (mitigated by Bloom filters); compaction consumes I/O/CPU and causes latency variance.

This B-tree vs LSM distinction *is* the "why is Cassandra write-fast" answer.

Real Production Example

Stripe: sharded MongoDB + strong internal tooling, with strict ledger-style invariants at the application layer. Uber: moved core trip storage from Postgres to a sharded MySQL-based system ("Schemaless", now on top of MySQL)

for predictable scaling. Netflix: Cassandra for viewing history (write-heavy, always-on, multi-region). Amazon: DynamoDB serves Prime Day at tens of millions of requests/sec. Meanwhile, most companies you interview at run their core on Postgres/MySQL — say that out loud; it's credible.

Advantages / Trade-offs / Common Mistakes

- SQL: flexibility, integrity (constraints, FKs), transactions; but single-writer ceilings and sharding is manual + painful.
- NoSQL: elastic scale, availability, predictable latency; but weak/eventual consistency by default, no joins (you denormalize and maintain copies), query patterns frozen into the schema.
- Mistakes: choosing NoSQL "for scale" at 100 QPS; claiming "NoSQL has no schema" (it has an implicit schema in the app — schema-on-read); forgetting Elasticsearch/analytics stores are *derived* systems needing a sync pipeline (CDC), not primaries.

Interview Questions

1. Pick a database for: a banking ledger, a shopping cart, viewing-history ingestion, a fraud graph, full-text product search — justify each.
2. Why are LSM stores write-fast and what do reads cost?
3. When do you move off a single Postgres, and to what — replicas, shards, or NewSQL?

Best Practices

- Default Postgres; add specialized stores per access pattern with CDC pipelines feeding them; never let a derived store become an unsynced second source of truth.

4.2 Replication: Leader-Follower, Multi-Leader, Leaderless (+ Read Replicas)

Why Interviewers Ask This

Replication is how you get read scale, availability, and durability — and every mode has a signature failure (lag, conflicts, stale quorums) they will probe.

Core Concept & Internal Working

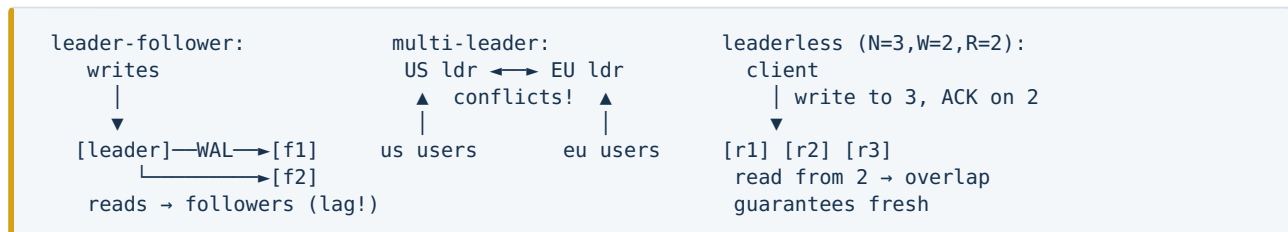
Leader-follower (primary-replica) — the default. All writes go to one leader; it ships its WAL/binlog to followers, which replay it.

- **Sync replication**: leader waits for follower ACK → no loss on failover, higher write latency; usually "semi-sync": wait for *one* follower.
- **Async replication**: leader ACKs immediately → fast, but on leader crash the most recent writes may not have reached any follower → **lost on failover**.
- **Read replicas** scale reads: point read traffic at followers. The cost is **replication lag** (ms to minutes under load) → users may not see their own writes. Fixes: read-your-writes routing (own profile → leader, or sticky-to-leader for N seconds after a write), monotonic reads (session pinned to one replica), or accept staleness per endpoint.
- **Failover**: detect leader death (timeout — careful: slow ≠ dead), promote the most-up-to-date follower, repoint clients. Dangers: split brain (old leader comes back and takes writes — fence it: STONITH/epoch numbers) and lag-loss (async writes vanish).

Multi-leader — a leader per region/datacenter, each accepting writes, replicating to the others asynchronously. Wins: local write latency in every region, region- outage tolerance. The price: **write conflicts** (same row updated in two regions concurrently) requiring resolution: last-writer-wins (data loss, but simple), per-field merge, CRDTs,

or application callbacks. Sane usage: partition ownership so each record has a "home" region (user's data written only in their region) — multi-leader topology, single-leader semantics per record.

Leaderless (Dynamo-style: Cassandra, Riak, DynamoDB internals) — any replica accepts writes; the client/coordinator writes to N replicas and requires W ACKs; reads query R replicas. $R + W > N \Rightarrow \text{overlap} \Rightarrow \text{latest write visible}$ (e.g., N=3, W=2, R=2). Tunable per query (Cassandra `QUORUM`, `ONE`, `LOCAL_QUORUM`). Anti-entropy: **read repair** (fix stale replicas during reads), **hinted handoff** (neighbor holds writes for a down node), **Merkle-tree repair** (background sync). No failover needed — no leader to fail. Conflicts still need LWW timestamps or vector clocks.



Real Production Example

GitHub, Shopify, most of the industry: MySQL/Postgres semi-sync leader-follower with orchestrated failover (orchestrator/Patroni). Netflix: Cassandra `LOCAL_QUORUM` writes per region. CouchDB/mobile sync and some multi-datacenter MySQL setups are true multi-leader — famous mostly for their conflict bugs; calendar/collaborative apps use CRDTs instead.

Common Mistakes

- Reading from replicas without addressing lag → "my comment disappeared" bugs.
- Multi-leader without a conflict story ("we'll just replicate both ways") — that's a data-corruption generator.
- Believing quorum = strong consistency in all cases (sloppy quorums, concurrent writes, and clock-skewed LWW still create anomalies).
- No fencing on failover → split brain, two leaders, divergent data.

Monitoring / Failure

- Watch replication lag (seconds behind leader), failover time, conflict rate (multi-leader), hinted-handoff queue depth (leaderless).
- Drill failovers regularly; an untested failover is downtime waiting to happen.

Interview Questions

1. Async vs sync replication — what exactly is lost on failover and how does semi-sync balance it?
2. Design read-your-writes on top of read replicas.
3. Cassandra N=3: pick W/R for a shopping cart vs a payment record. (cart: W=1/R=1 fast+available; payment: QUORUM/QUORUM)
4. How do you prevent split brain during failover? (quorum-based leader election + fencing tokens/epochs)

Best Practices

- Semi-sync + automated, *fenced*, regularly-drilled failover; per-endpoint staleness policy for replica reads; per-record home-region if you need multi-region writes.

4.3 Partitioning & Sharding

Why Interviewers Ask This

Sharding is *the* answer to "your writes/dataset outgrew one machine", and its follow-ups (hot shards, resharding, cross-shard queries) are the deep dive in half of all interviews.

Core Concept

- **Partitioning**: splitting data within one logical database (Postgres declarative partitions by month — prunes queries, eases retention).
- **Sharding**: horizontal partitioning across *machines* — each shard is an independent database holding a subset of rows.

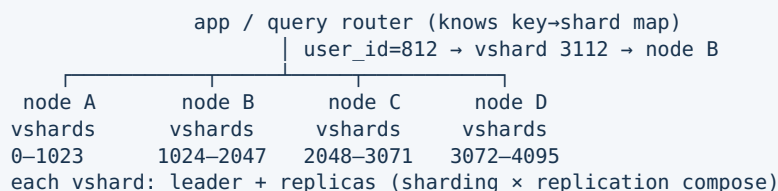
Choosing the shard (partition) key is the single most important schema decision at scale. Requirements: high cardinality, even load distribution, and — critically — *aligned with your dominant query pattern* so most queries hit ONE shard.

Strategies:

- **Hash sharding** ($\text{hash}(\text{user_id}) \bmod N \rightarrow$ better: consistent hashing / hash-range slots): uniform distribution; kills range scans; the default for entity-by-id access.
- **Range sharding** (users A–F, dates Jan–Mar; HBase/Bigtable, Spanner): efficient range queries; risk of hot ranges (monotonic keys like timestamps or auto-increment IDs write to ONE shard — the classic hot-shard anti-pattern; fix: prefix/salt the key, or use non-monotonic IDs).
- **Directory/lookup sharding**: a mapping service (tenant $\rightarrow$ shard) — maximum flexibility (move big tenants to dedicated shards), plus one more system to keep highly available (cached heavily).
- **Geo sharding**: by region for latency + data residency (Uber: city/geo cells).

The hard parts you must volunteer:

1. **Cross-shard queries** $\rightarrow$ scatter-gather (fan out to all shards, merge) — latency = slowest shard; avoid via key design or a derived/global index (async-maintained secondary index sharded by the other key).
2. **Cross-shard transactions** $\rightarrow$ avoid (choose keys so transactional units co-locate); else 2PC (blocking, fragile) or sagas (Module 7).
3. **Resharding**: with fixed $\text{mod } N$, changing N remaps nearly everything. Use **many virtual shards mapped to few physical nodes** (e.g., 4,096 vshards on 16 nodes) — scaling = moving whole vshards; or consistent hashing ($\sim 1/N$ movement); or range-split (Bigtable/CockroachDB auto-split hot ranges). Live migration: dual-write or CDC-backfill $\rightarrow$ verify $\rightarrow$ cut over reads $\rightarrow$ cut over writes.
4. **Hot shards**: celebrity/whale tenants — detect, split their range, isolate them on dedicated shards, or add a second-level key ($\text{celebrity_id} + \text{bucket}$).



Real Production Example

- **Vitess** (YouTube $\rightarrow$ open source): sharded MySQL with vshard-style resharding — powers Slack, Square, GitHub's move.

- **Instagram:** Postgres sharded by user via thousands of logical shards in fewer physical machines; their ID scheme (timestamp + shard id + sequence) bakes the shard into the ID.
- **Discord:** messages sharded by `(channel_id, bucket)` in Cassandra/ScyllaDB — key aligned with "read a channel's recent messages".
- **Slack:** workspace(team)-sharded MySQL/Vitess — a tenant's data co-located, while workspaces moved to their own shards.

Common Mistakes

- Sharding too early (operational tax forever) or by the wrong key (every query becomes scatter-gather — the fatal flaw; interviewers will ask "now fetch X by *other* field").
- `mod N` with no vshard/consistent-hash layer → resharding = full rehash outage.
- Monotonic shard keys (timestamps) → one hot shard absorbing all writes.
- Forgetting unique IDs across shards (need Snowflake-style IDs — timestamp + machine + sequence — or UUIDv7).

Monitoring / Failure

- Per-shard: size, QPS, latency, replication health; skew dashboards (max/median shard load). Failure of one shard = partial outage for its keys — degrade gracefully, don't 500 the whole product.

Interview Questions

1. Pick the shard key for: a multi-tenant SaaS, a chat app's messages, a payments ledger. Defend against the "query by other dimension" follow-up.
2. Double the cluster with zero downtime — walk the resharding play.
3. A shard is hot because one tenant is huge — options? (split, isolate, sub-partition)

Best Practices

- Shard late, but design IDs and schemas to be shardable early; virtual shards from day one; shard key = dominant access path; secondary access paths via derived indexes fed by CDC.

4.4 Indexing Strategy

Why Interviewers Ask This

"The query is slow" is the most common production incident, and indexing is the usual answer — with write-amplification trade-offs a senior engineer must quantify.

Core Concept & Internal Working

An index is a sorted (or hashed) redundant structure mapping column values → rows.

- **B+-tree** (the default everywhere): $O(\log n)$ point + range lookups; leaves linked for scans.
- **Composite indexes:** `(a, b, c)` supports filters on `a`, `a,b`, `a,b,c` (leftmost-prefix rule) and ordering by the indexed sequence. Design them from your WHERE + ORDER BY patterns: equality columns first, then range column, then sort columns.
- **Covering index:** include all columns the query needs → index-only scan, no heap fetch — the difference between 50 ms and 0.5 ms on hot endpoints.
- **Selectivity** matters: indexing a boolean rarely helps (unless partial); indexing `email` (unique) is ideal. **Partial indexes** (`WHERE status='active'`) keep hot indexes small. Expression indexes (`lower(email)`). GIN/inverted for JSONB/arrays/full-text; geospatial (R-tree/GIST); hash indexes for pure equality.

- **The cost:** every index adds write amplification (each INSERT/UPDATE touches every index), consumes RAM (indexes compete for buffer pool), and slows down the optimizer's choices. A table with 12 indexes is a write-latency incident in progress.
- Clustered vs secondary: InnoDB stores rows *in* the PK B-tree (secondary indexes point to PK → keep PKs small; UUIDv4 PKs shred insert locality — prefer UUIDv7/Snowflake); Postgres heap + all indexes secondary.

Read the plan: `EXPLAIN ANALYZE` — seq scan on a large table under a selective filter = missing index; index scan + huge filter = wrong composite order; sort node = missing ORDER BY coverage.

Real Production Example

Every mature engineering org runs slow-query monitoring (pt-query-digest, `pg_stat_statements`) and reviews indexes in schema migrations. GitLab and Shopify publish index-review checklists: every new query pattern ships with its index, every index must justify its write cost.

Common Mistakes

- Indexing every column "just in case" (write amplification, bloated buffer pool).
- Composite order wrong (`((created_at, user_id))` when queries filter by user and range by time → needs `((user_id, created_at))`).
- Functions on indexed columns in predicates (`WHERE lower(email)=...` without an expression index → seq scan).
- Ignoring index maintenance: bloat, unused indexes (drop them — measure with usage stats).

Interview Questions

1. Design indexes for: `WHERE tenant_id=? AND status='open' ORDER BY created_at DESC LIMIT 20 . ((tenant_id, status, created_at DESC) , possibly partial on status)`
2. Why did adding an index slow down the system? (write amplification / plan flipped / buffer-pool pressure)
3. What's a covering index and when is it worth it?

Best Practices

- Index for the query, not the table; leftmost-prefix discipline; partial + covering for hot paths; audit unused indexes quarterly; PKs sequential-ish.

4.5 Schema Design (Normalization vs Denormalization)

Why Interviewers Ask This

Schema decisions encode the read/write trade-off. Interviewers check you can start normalized and denormalize *deliberately*, with a plan to keep copies consistent.

Core Concept

- **Normalize (3NF)** for the transactional core: one fact in one place, updates are single-row, integrity by construction. Joins are the read cost.
- **Denormalize** when read patterns demand it: precomputed aggregates (`like_count` on posts — don't `COUNT()` *2M likes per render*), *embedded copies* (*order line items snapshot product name/price* — *historically correct**, not just fast), fan-out tables (precomputed timelines), and derived read models (CQRS, 4.7).
- Every denormalized copy needs a stated **sync mechanism** (same transaction, trigger, CDC pipeline, or periodic rebuild) and a **repair story** (recompute from source of truth).

- NoSQL modeling inverts the process: list the queries first, then design one table/collection per access pattern, duplicating data freely (DynamoDB single-table design: partition key + sort key overloading).
- Other essentials: soft deletes vs hard deletes (audit/GDPR tension), created/updated timestamps everywhere, avoid EAV unless truly dynamic attributes (use JSONB), plan schema migrations to be online (additive changes, backfill, dual-read, contract later — never `ALTER TABLE` a 2TB table blocking writes; use gh-ost/pt-osc for MySQL).

Real Production Example

Amazon retail order records snapshot price/title at purchase time (denormalized by correctness requirement). Reddit/Twitter store counters denormalized with periodic reconciliation jobs. DynamoDB's documented best practice is explicitly "model for access patterns, denormalize, one table".

Interview Questions

1. Where would you denormalize a marketplace schema and how do you keep each copy honest?
2. Why must order line items snapshot the product data?
3. How do you run a schema migration on a huge hot table with zero downtime?

Best Practices

- Normalized core, denormalized edges; every copy has an owner, a sync path, and a rebuild job; counters get reconciliation.

4.6 Connection Pooling

Why Interviewers Ask This

Connection-pool exhaustion is a top-3 real production outage, and pool sizing is a Little's-Law arithmetic check on the candidate.

Core Concept & Internal Working

DB connections are expensive: TCP + TLS + auth per connect, and each Postgres connection is a *process* (MBs of RAM, scheduler load). A pool keeps N warm connections and multiplexes app requests over them.

- App-side pools: HikariCP (JVM), pgx (Go), etc. Key knobs: max size, acquire timeout, max lifetime (recycle to survive failovers/DNS changes), idle timeout.
- **The fleet math problem:** 200 app instances × 20 pool connections = 4,000 connections vs Postgres comfort zone (~a few hundred active). Fix: an intermediary pooler — **PgBouncer** (transaction-mode pooling: a server connection is borrowed per transaction — thousands of client conns share tens of server conns) or RDS Proxy.
- **Sizing:** connections ≈ concurrency needed = QPS × avg query time (Little's Law). 1,000 QPS × 5 ms = 5 in-flight → a pool of ~10–20 suffices. Oversized pools *reduce* throughput (context switching, lock contention on the DB) — the famous HikariCP guidance: pool ≈ cores × 2 for CPU-bound DB work.
- **Exhaustion cascade:** one slow query → connections held longer → pool empties → requests queue at acquire → app threads block → upstream timeouts → retries pile on. Defenses: acquire timeout + fail fast, per-endpoint statement timeouts, circuit breaker around the DB, separate pools for critical vs batch traffic (bulkhead).

200 app pods × 20 conns = 4000 → PgBouncer (transaction mode) → 50 → Postgres
 thousands of idle client conns
 share tens of real ones

Interview Questions

1. Size the pool: 2k QPS, 10 ms avg query, 4 app instances. (≈ 20 in flight $\rightarrow \sim 5\text{--}10$ per instance + headroom)
2. Walk the exhaustion cascade and every mitigation layer.
3. Why can a *bigger* pool make the database slower?

Best Practices

- Small pools + fast acquire timeouts + statement timeouts; PgBouncer/proxy at fleet scale; bulkhead pools per workload class; recycle connections (max lifetime) to survive failovers.

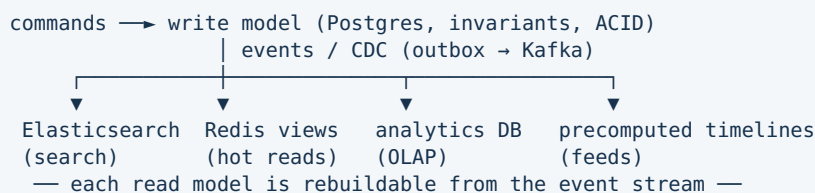
4.7 CQRS (High Level)

Why Interviewers Ask This

CQRS names the pattern behind most read-scaling tricks (replicas, caches, search indexes, materialized views are all "read models"). Interviewers want you to know when it's justified — and that it's not a default.

Core Concept

Command Query Responsibility Segregation: separate the write model (normalized, transactional, invariant-enforcing) from one or more read models (denormalized, query-shaped), connected by an async event/CDC pipeline.



Properties to state in interviews: read models are **eventually consistent** (the pipeline has lag — design the UX for it: optimistic UI, read-your-writes from the write side); each read model is independently scaled and *rebuildable* (replay events); often paired with (but does not require) **event sourcing** — storing the events as the source of truth rather than current state.

Use when: read:write ratios are extreme, read shapes diverge wildly from the write shape (search + feed + analytics from one dataset), or teams need independent scaling. Skip when: a CRUD app — the pipeline, lag handling, and rebuild machinery are a heavy tax.

Real Production Example

Any product with "DB + Elasticsearch + cache + feed" is running de facto CQRS: LinkedIn (profile writes $\rightarrow$ Kafka $\rightarrow$ search/graph/feed read models — Kafka was born for this), Uber (trip events $\rightarrow$ many read stores), e-commerce (order write model + order-history and search read models).

Interview Questions

1. The user edits a product but search shows the old title for 3 s — explain, and is it acceptable?
2. Your Elasticsearch cluster is corrupted — recovery plan? (rebuild from events/source table — this is why rebuildability is a requirement)
3. CQRS vs just read replicas — what's the actual difference? (replicas share the *same* model shape; CQRS reshapes data per query need)

Best Practices

- Outbox pattern (Module 7) to publish events atomically with writes; version events; make every read model rebuildable and monitored for lag.

Module 4 Cheat Sheet

CHOOSE DB	Access patterns → transactions → scale → team. Default Postgres. B-tree = read-optimized; LSM = write-optimized (memtable→SSTable→compaction, Bloom filters for reads).
REPLICATION	Leader-follower: async(fast, loss on failover)/semi-sync. Replicas scale reads; LAG → read-your-writes routing. Failover: fence old leader (split brain).
MULTI-LEADER	Write locally in each region; CONFLICTS (LWW/CRDT/merge). Sane mode: per-record home region.
LEADERLESS	Dynamo: W+R>N quorum, tunable; read repair, hinted handoff, Merkle repair. No failover; conflict resolution still needed.
SHARDING	Shard key = dominant query path, high cardinality, non-monotonic. Hash (uniform, no ranges) vs range (scans, hot ranges) vs directory. Virtual shards → cheap resharding. Cross-shard query = scatter-gather (avoid); cross-shard txn = saga/2PC (avoid harder). Snowflake IDs.
INDEXING	Composite: equality→range→sort (leftmost prefix). Covering = index-only. Partial for hot subsets. Every index = write amplification. EXPLAIN.
SCHEMA	Normalize core, denormalize deliberately w/ sync + rebuild story. Snapshot immutable facts (order items). Online migrations (gh-ost).
POOLING	pool ≈ QPS × latency (Little). Small pools win. PgBouncer at fleet scale. Acquire+statement timeouts. Bulkhead per workload.
CQRS	Write model + async-fed read models (search/cache/feed/OLAP). Eventually consistent, rebuildable, outbox-published. Not for CRUD.

Top Interview Questions (Module 4)

1. SQL vs NoSQL for five given workloads. 2. Async vs semi-sync replication and failover loss. 3. Read-your-writes over replicas. 4. Shard key for chat / SaaS / payments + resharding at 2x. 5. Hot shard from a whale tenant. 6. Index for a given query + why indexes hurt writes. 7. Zero-downtime migration of a 2 TB table.
2. Pool sizing math + exhaustion cascade. 9. Cassandra W/R tuning per use case.
3. CQRS vs read replicas; rebuild a corrupted read model.

Common Mistakes Recap

NoSQL-for-scale at toy scale • replica reads without a lag story • shard key misaligned with queries • mod-N sharding • monotonic keys → hot shard • index sprawl • UUIDv4 clustered PKs • giant pools • denormalized copies without sync/ repair • treating Elasticsearch as a primary.

Mock Interview Exercise

"Design storage for a marketplace: 10M sellers, 500M listings, 50k reads/s, 2k writes/s, listing search by text+facets, per-seller dashboards, strong consistency on inventory counts at checkout." Expected: Postgres core (sellers/listings/orders, seller_id-aligned schema) → shard by seller_id with vshards when write volume demands; inventory decrement single-row transactional; CDC → Elasticsearch (search read model) + Redis (hot listing cache) + warehouse; read replicas + RYW for seller dashboards; index plan for the top 5 queries; failover + resharding story.

Module 5 — Messaging & Event-Driven Architecture

Queues and logs are how large systems decouple, absorb bursts, and survive partial failure. Interviewers probe three things relentlessly: delivery semantics, ordering, and what happens when consumers fall behind or poison messages arrive.

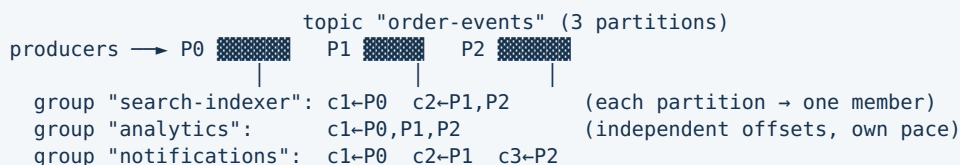
5.1 Core Model: Queues, Topics, Producers, Consumers, Consumer Groups

Why Interviewers Ask This

Vocabulary precision here determines whether the rest of your messaging discussion makes sense. "Queue vs topic" and "how do consumer groups scale" are standard warm-ups.

Core Concept

- **Producer:** writes messages. **Consumer:** reads and processes them.
- **Queue (point-to-point):** each message is processed by exactly ONE consumer among competing workers. Work distribution. (SQS, RabbitMQ queue.)
- **Topic (publish-subscribe):** each message is delivered to ALL subscriber groups. Event broadcast. (Kafka topic, SNS, RabbitMQ fanout exchange.)
- **Consumer group:** a set of consumer instances sharing one logical subscription; the broker partitions work among them. Each *group* gets every message once; *within* a group, each message goes to one member. Kafka: partitions are assigned to group members (max parallelism = partition count). This is how the same event stream feeds the search indexer group AND the analytics group AND the notification group independently.



Why queues at all (the interview answer): **decoupling** (producer doesn't know or wait for consumers), **burst absorption** (queue soaks the flash sale; consumers drain at their sustainable rate), **failure isolation** (email service down $\neq$ order placement down), **fan-out of one fact to many systems**, and **retryability** (redelivery instead of lost work).

Common Mistakes

- Using a queue where a synchronous call is required (the user needs the answer *now* — payment authorization is sync; the receipt email is async).
- Expecting more parallelism than partitions (Kafka: 3 partitions = max 3 active consumers in a group).

5.2 Kafka (Deep Dive)

Why Interviewers Ask This

Kafka is the backbone of event-driven architecture at nearly every large company; "how does Kafka work" and "why Kafka over a queue" are direct senior questions.

Core Concept

Kafka is not a queue — it is a **distributed, partitioned, replicated append-only log**. Messages are appended to partitions and *retained* (time/size based, or forever with compaction), and consumers track their own position (**offset**). Consumption doesn't delete data, so many groups read the same stream at their own pace, and you can **replay** history — the property that makes CDC pipelines, read model rebuilds, and reprocessing possible.

Internal Working

- **Topic** → **partitions**; each partition is an ordered log on disk. Ordering is guaranteed **within a partition only**. Producer chooses partition by key hash (`key=user_id` ⇒ all events of a user are ordered).
- **Replication**: each partition has a leader + followers (replication factor 3 typical). Producers/consumers talk to the leader. **ISR** (in-sync replicas): followers caught up. `acks=all` + `min.insync.replicas=2` ⇒ a write is ACKed only when a quorum has it — no loss on single-broker failure. `acks=1` ⇒ faster, loss window on leader crash.
- **Why it's fast**: sequential disk I/O (append-only), OS page cache, zero-copy sendfile, producer batching + compression, consumers pull in batches. Millions of msgs/sec per modest cluster.
- **Consumer groups & rebalancing**: group coordinator assigns partitions; member death triggers rebalance (brief pause — cooperative rebalancing reduces it). Offsets committed to an internal topic; commit *after* processing = at-least-once (duplicates possible on crash); commit *before* = at-most-once (loss possible).
- **Exactly-once (EOS)**: idempotent producer (sequence numbers dedupe broker-side retries) + transactions (atomic write to multiple partitions + offset commit) ⇒ exactly-once *within Kafka-to-Kafka pipelines* (Kafka Streams). The moment a side effect leaves Kafka (call an API, write a DB), you're back to at-least-once + idempotency (see 5.5).
- **Log compaction**: retain the latest value per key — a changelog that doubles as a table (CDC, state restore).
- KRaft (built-in Raft) replaced ZooKeeper for metadata/controller consensus.

```
topic "payments", partition 1 (leader on broker B, replicas A,C):
offset: 0   1   2   3   4   5   6   → append only
      [m0] [m1] [m2] [m3] [m4] [m5] [m6]
              ▲               ▲
      group X committed=3   group Y committed=6
      (replayable: reset offset to 0 anytime within retention)
```

Real Production Example

LinkedIn built Kafka (now trillions of messages/day) to unify activity data and feed every downstream system from one log. Uber: trip events through Kafka to pricing, fraud, analytics, driver matching. Netflix: all telemetry and CDC flows. Stripe: event backbone with strict idempotent consumers for money movement.

Advantages / Trade-offs

- Wins: throughput, durability, replay, fan-out to unlimited groups, ordering per key, stream processing ecosystem (Kafka Streams, Flink, Connect/Debezium).
- Costs: operational weight (self-managed), no per-message routing/priority/delay semantics (that's RabbitMQ territory), consumer-side complexity (rebalances, offset management), partition count = a semi-rigid scaling decision (repartitioning breaks key ordering during the transition).

Common Mistakes

- Claiming global ordering across a topic (per-partition only).
- Choosing a low-cardinality partition key (all events on 2 of 50 partitions) or keying by something hot (one celebrity ⇒ one hot partition).
- "Kafka guarantees exactly-once" without the Kafka-boundary caveat.

- Unbounded consumer lag ignored until retention deletes unread data (data loss by neglect).

Monitoring / Failure

- The metric: **consumer lag** (messages behind) per group/partition — alert on growth rate, not absolute value. Also: ISR shrinks, under-replicated partitions, rebalance frequency, produce/fetch latency.
- Failures: broker loss (leader election from ISR — clean if `acks=all`), rebalance storms from flapping consumers (tune session timeouts; static membership), poison messages stalling a partition (see DLQ 5.6).

Interview Questions

1. How does Kafka achieve both durability and throughput? (sequential log + replication + batching)
2. You need per-user ordering and 100k msg/s — design topic/partitions/keys.
3. What exactly does `acks=all`, `min.insync.replicas=2` buy you, and what does it cost?
4. Consumer lag is growing — enumerate causes and fixes. (slow processing → scale consumers up to partition count / optimize handler / batch; hot partition → rekey; rebalance churn)

5.3 RabbitMQ, ActiveMQ, Amazon SQS (and choosing among them)

Why Interviewers Ask This

"Kafka vs RabbitMQ vs SQS" is a canonical trade-off question; the senior answer maps message *semantics needed* to broker *model*.

Core Concept & Internal Working

RabbitMQ — a smart **broker-centric** message router (AMQP). Producers publish to **exchanges**; exchanges route to queues via **bindings**: direct (exact routing key), topic (wildcard patterns `order.*.eu`), fanout (broadcast), headers. Per-message ACK/NACK with redelivery, per-queue TTL, **priority queues**, **delayed delivery**, per-consumer prefetch (backpressure), dead-letter exchanges built in. Messages are *deleted on ACK* — it's a post office, not a log. Quorum queues (Raft) for replicated durability. Great at: task/job distribution, complex routing, RPC patterns, per-message control. Ceiling: throughput well below Kafka; no replay.

ActiveMQ (/Artemis) — the JMS-standard broker of the Java enterprise world; feature-similar to RabbitMQ (queues, topics, selectors, XA transactions). In interviews: mention it as the legacy/JMS-compatible choice; Artemis is the modern core.

Amazon SQS — fully managed queue-as-a-service. **Standard**: near-unlimited throughput, at-least-once, best-effort ordering (duplicates and reordering happen — by contract). **FIFO**: ordered + exactly-once-ish dedup within a 5-min window, throughput limited per message group. Mechanics: poll → message becomes *invisible* (visibility timeout) → consumer processes → deletes it; timeout expiry without delete = automatic redelivery (crash-safe by design). `ReceiveCount` + redrive policy → DLQ built in. Long polling to cut empty receives. Pairs with **SNS** (pub/sub fan-out: SNS topic → many SQS queues) and Lambda.

Decision sketch:

need replay / event log / many independent readers / streams	→ KAFKA
need routing / priorities / delays / per-job semantics	→ RABBITMQ
on AWS, want zero ops, simple work queue	→ SQS (+SNS fanout)
JMS/legacy Java estate	→ ActiveMQ Artemis

Real Production Example

Shopify and many fintechs: Kafka for events + RabbitMQ/SQS for job queues — *both*, because they solve different problems. Reddit historically ran RabbitMQ for async jobs. Amazon internally: SQS everywhere for decoupling (it predates Kafka).

Common Mistakes

- Treating the three as interchangeable "queues" — replay and fan-out (Kafka) vs routing and job semantics (Rabbit) vs zero-ops (SQS) are different products.
- Ignoring SQS standard's duplicates/reordering contract, then debugging "impossible" double-sends.
- Setting SQS visibility timeout shorter than processing time (guaranteed duplicate processing).

Interview Questions

1. Order events must feed search, analytics, and email — which broker(s) and why?
 2. SQS standard vs FIFO: what do you give up for ordering?
 3. Explain the visibility-timeout mechanism and its failure semantics.
-

5.4 Ordering

Why Interviewers Ask This

Ordering bugs cause real money loss ("shipped" processed before "paid"). They want to hear: global ordering doesn't scale — scope ordering to a key.

Core Concept & Internal Working

- **Global ordering** requires a single serialization point (1 partition / FIFO with one group) — throughput ceiling. Almost never actually required.
- **Per-key ordering** is what businesses need: all events *for one order / one user / one account* in order. Kafka: partition by key. SQS FIFO: `MessageGroupId`. Kinesis: partition key.
- What breaks ordering even with keyed partitions: producer **retries** without idempotence (msg2 succeeds, msg1's retry lands after — fix: `enable.idempotence=true`, `max.in.flight≤5` with idempotence, or `=1`), **repartitioning** (key → partition mapping changes), consumer-side **parallel processing** of one partition's batch (process per-key serially), and DLQ/retry paths (a retried message leaves its slot — see 5.6).
- If out-of-order is unavoidable, make consumers order-tolerant: **version numbers / sequence checks** on the entity (ignore stale updates), or buffer-and-reorder windows (stream processing watermark techniques).

Interview Questions

1. Payment events for the same order must be ordered; you need 50k msg/s across all orders. Design it. (key by `order_id`; scale via partitions — per-key serial, global parallel)
 2. Producer retry reorders messages — why, and which settings fix it?
 3. A consumer sees `order_shipped` before `order_paid`. List every place that could have reordered them.
-

5.5 Delivery Semantics: At-Most-Once, At-Least-Once, Exactly-Once

Why Interviewers Ask This

This is the single most-tested messaging concept, because it decides whether you double-charge customers. The expected senior take: *exactly-once delivery is impossible over a network; exactly-once processing = at-least-once + idempotency*.

Core Concept & Internal Working

The problem: producer sends, network drops the ACK. Did the broker get it? You must choose: don't retry (**at-most-once** — possible loss) or retry (**at-least-once** — possible duplicate). Same dilemma consumer-side: ACK

before processing (at-most-once: crash after ACK loses the message) vs ACK after processing (at-least-once: crash after processing, before ACK ⇒ redelivery ⇒ duplicate).

semantics	loss?	dupes?	cost	use
at-most-once	yes	no	cheapest	metrics, telemetry ticks
at-least-once	no	YES	idempotent consumers	the default for everything
exactly-once	no	no	heavy coordination, narrow scope	Kafka→Kafka streams; elsewhere: build it yourself

Building exactly-once *effect* (the answer interviewers want):

1. **Idempotency keys:** every message/command carries a unique ID; consumer records processed IDs (unique constraint / `INSERT ... ON CONFLICT DO NOTHING` in the *same transaction* as the business write) — duplicates become no-ops. This is how Stripe's API works (`Idempotency-Key` header: same key ⇒ same result returned, charge executed once).
2. **Naturally idempotent operations:** `SET status='shipped'` (safe to repeat) vs `balance += 100` (not — convert to an insert of an immutable ledger entry keyed by event ID).
3. **Transactional coupling:** store the offset/message-ID with the state change in one ACID transaction; on restart, resume from stored position.
4. Kafka EOS: covers Kafka-in → process → Kafka-out atomically (idempotent producer + transactions); the moment you touch an external system, revert to (1).

Real Production Example

Stripe idempotency keys (public API design). Every payment processor's consumer: `processed_events(event_id PK)` table checked in-transaction. Kafka Streams EOS for internal aggregation pipelines at LinkedIn/Confluent scale.

Common Mistakes

- "We use Kafka so we have exactly-once" (only within Kafka; your DB write + Kafka consume is not atomic without your own idempotency).
- Deduplicating in a cache (Redis set of seen IDs) *outside* the business transaction — crash between them and the guarantee evaporates. The dedup record and the effect must commit together.
- Choosing at-most-once by accident (auto-ACK/auto-commit before processing).

Interview Questions

1. Prove exactly-once delivery is impossible; then design exactly-once *payment processing* anyway.
2. Where exactly must the idempotency check live relative to the business write, and why?
3. Which semantics for: clickstream analytics, order emails, ledger postings?

5.6 Retries, Dead Letter Queues, Backpressure

Why Interviewers Ask This

Retry design separates people who've run consumers in production from people who've read about them. A bad retry strategy *causes* outages (retry storms) and stalls pipelines (poison pills).

Core Concept & Internal Working

Retry strategy:

- **Exponential backoff + jitter** (1s, 2s, 4s... ×random) — jitter prevents synchronized retry waves.

- **Retry budget/cap** (e.g., max 5 attempts) — infinite retries on a permanent failure is a stuck pipeline.
- **Classify errors**: transient (timeout, 503, deadlock) → retry; permanent (validation, 400, business rule) → straight to DLQ, retrying can't help.
- **In-broker patterns**: SQS redelivers automatically via visibility timeout + `maxReceiveCount` → DLQ redrive. Kafka has no built-in delay/retry: standard pattern is **tiered retry topics** (`orders-retry-5m`, `orders-retry-1h`) — failed message forwarded to a retry topic with delayed consumption, finally to `orders-dlq`. Note this *sacrifices per-key ordering* for the retried key — if ordering matters, either park the whole key (track keys-in-retry and divert their subsequent messages to the retry path too) or block the partition briefly instead.
- **Poison pill**: a message that fails deterministically (bad schema, corrupt payload). Without a DLQ + max attempts it stalls its partition forever (Kafka) or burns redeliveries (Rabbit/SQS).

Dead Letter Queue: the quarantine for messages that exhausted retries. A DLQ is only useful with: **alerting** on arrival (a silent DLQ is a data-loss bin), **tooling to inspect + redrive** after the bug is fixed, and retention long enough to survive a slow incident response. DLQ entries should carry failure metadata (exception, attempt count, original topic/offset).

Backpressure (consumers falling behind): the queue *is* the buffer — monitor **lag/queue depth and its derivative**. Responses, in order: scale consumers (up to partition count in Kafka — then add partitions), batch processing, optimize the handler (usually a slow downstream call — parallelize per-key-safe work), shed or degrade (skip enrichment), and rate-limit producers if the contract allows. RabbitMQ: prefetch limits + flow control; Kafka: pull model means consumers naturally take only what they can, lag absorbs the rest — until retention.

Real Production Example

Uber's and Airbnb's engineering blogs both document the tiered-retry-topic + DLQ Kafka pattern. AWS prescribes DLQs on every SQS queue and Lambda event source. Retry storms feature in half of all public postmortems — the fix is always jitter + budgets + circuit breaking.

Interview Questions

1. Design the retry/DLQ pipeline for order processing on Kafka, preserving per-order ordering.
2. What's a poison pill and what happens without a DLQ?
3. Consumer lag doubled after a downstream API slowed — walk your response playbook.

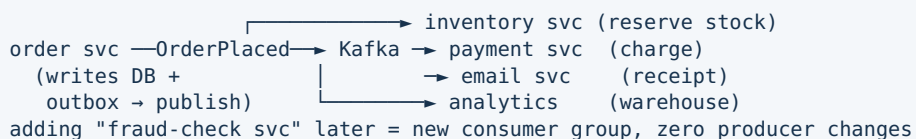
5.7 Event-Driven Architecture (EDA)

Why Interviewers Ask This

EDA is the architectural context for everything above, and its pitfalls (implicit coupling, debugging pain, eventual consistency UX) are staff-level discussion bait.

Core Concept

Services communicate by publishing immutable **facts** ("OrderPlaced") rather than sending commands ("SendEmail"). Producers don't know consumers; new consumers attach without touching producers.



Key design points:

- **Events vs commands:** events are facts, past tense, no expected reply; commands are directed requests. Mixing them muddles ownership.
- **Event schema is a public API:** version it (schema registry — Avro/Protobuf, compatibility rules), never break consumers. Thin events (IDs, fetch details) vs fat events (full payload — self-contained but stale-prone and big): choose deliberately.
- **Choreography** (services react to each other's events — loose coupling, but the workflow is invisible, spread across services) **vs orchestration** (a coordinator like Temporal/Step Functions drives the flow — visible, debuggable, one more component). Long multi-step business flows usually deserve orchestration (see Sagas, Module 7).
- **Consistency:** the system is eventually consistent between services — design UX for it (order shows "processing"), and use the **outbox pattern** (Module 7.7) so DB-write + event-publish can't diverge.
- The honest costs: harder debugging (need correlation IDs + tracing across async hops), duplicate handling everywhere (5.5), and *implicit* coupling through event schemas that's easy to underestimate.

Real Production Example

Uber (trip lifecycle as events through Kafka), LinkedIn (everything-as-a-stream — the origin of Kafka), Netflix (playback + operational events), Amazon (EventBridge/ SNS/SQS fabric between retail services).

Interview Questions

1. When do events beat synchronous calls, and what do you lose? (decoupling, resilience, fan-out vs immediacy, debuggability, consistency)
2. Choreography vs orchestration for a 6-step order fulfillment flow?
3. How do you evolve an event schema without breaking 12 consumer teams?

Module 5 Cheat Sheet

MODEL	queue = 1 consumer/message (work). topic = all groups get all (facts). consumer group: partitions split among members; parallelism $\leq$ partitions.
KAFKA	partitioned replicated LOG; retention + offsets $\Rightarrow$ replay + multi-reader. Ordering per partition (key!). acks=all+min.isr=2 = no-loss quorum. Fast: sequential I/O, page cache, zero-copy, batching. Lag = the metric. EOS only Kafka $\rightarrow$ Kafka; else idempotent consumers.
RABBITMQ	exchanges $\rightarrow$ bindings $\rightarrow$ queues (direct/topic/fanout). ACK/NACK, priority, delay, prefetch, DLX. Deleted on ACK — no replay. Jobs + routing.
SQS	managed; standard = at-least-once, unordered, ∞ throughput. FIFO = ordered per MessageGroupId + 5-min dedup. Visibility timeout (must exceed processing time!). +SNS for fanout. Redrive $\rightarrow$ DLQ built in.
ORDERING	global ordering doesn't scale; order per KEY (partition key / msg group). Breakers: producer retries (fix: idempotence), repartitioning, parallel handlers, retry paths. Consumers: version checks tolerate disorder.
SEMANTICS	exactly-once DELIVERY impossible. at-least-once + IDEMPOTENCY = exactly- once EFFECT. Dedup record must commit IN the business transaction. Ledger inserts (keyed) $>$ in-place increments.
RETRY/DLQ	backoff + jitter + max attempts; classify transient vs permanent. Kafka: tiered retry topics (mind ordering); SQS: visibility+redrive. DLQ needs alerting + inspect/redrive tooling. Poison pill stalls partitions.
BACKPRESSURE	lag & its slope; scale consumers $\leq$ partitions, batch, optimize handler, shed, rate-limit producers. Retention is the deadline.
EDA	events = immutable facts; schema = versioned public API (registry). choreography (invisible flow) vs orchestration (coordinator). Outbox for atomic write+publish. Correlation IDs for debugging.

Top Interview Questions (Module 5)

1. Kafka vs RabbitMQ vs SQS for three given workloads.
2. Per-user ordering at 100k msg/s.
3. Exactly-once payment processing end to end.
4. Retry + DLQ design that preserves ordering.
5. Consumer lag runbook.
6. acks/ISR trade-offs.
7. Visibility timeout semantics.
8. Outbox pattern (why dual-write is broken).
9. Choreography vs orchestration.
10. Schema evolution across many consumers.

Common Mistakes Recap

Global-ordering claims • low-cardinality/hot partition keys • "Kafka = exactly-once" • dedup outside the transaction • visibility timeout < processing time • infinite retries / no DLQ / silent DLQ • retry topics silently breaking per-key order • async where the user needs a sync answer • unversioned event schemas.

Mock Interview Exercise

"Design order processing for a flash-sale e-commerce site: 30k orders/min peak, each order → charge payment, reserve inventory, send confirmation; payment double-charging is unacceptable; email may lag minutes."

Expected: API writes order + outbox row in one transaction → Kafka `orders` keyed by `order_id` → payment consumer with idempotency table in-transaction → tiered retries + DLQ + alerting → inventory via reservation events with version checks → email as an independent lagging group → lag/DLQ/duplicate-rate monitoring; explain every duplicate and reordering path and why the design survives it.

Module 6 — Storage

Every design with files, media, backups, or big data needs a storage tier, and interviewers expect you to pick the right *class* of storage (object vs block vs file) with numbers: durability, latency, cost per GB, and access patterns.

6.1 Object Storage (S3, GCS, Azure Blob)

Why Interviewers Ask This

"Where do the images/videos/backups go?" — the answer is object storage in ~100% of modern designs, and the follow-ups (consistency, multipart upload, presigned URLs, tiering) test real experience.

Core Concept

A flat namespace of **immutable objects** (bytes + metadata) addressed by key over an **HTTP API** (GET/PUT/DELETE) — no directories (prefixes only simulate them), no in-place edits (replace whole objects; versioning keeps history), no POSIX. Effectively infinite capacity, pay per GB + per request.

Internal Working

- **Durability (11 nines)** via **erasure coding**: an object is split into k data + m parity shards spread across disks/AZs (e.g., 6+3 survives any 3 losses at 1.5× overhead vs 3× for replication). Background scrubbing verifies checksums and re-shards.
- Metadata service maps key → shard locations (itself a massive distributed DB — S3's metadata layer is one of the largest database systems on earth).
- **Consistency**: S3 is strongly consistent since 2020 (read-after-write for all ops) — stop citing the old eventual-consistency caveat.
- **Multipart upload**: big files uploaded as parallel parts (5 MB–5 GB each), then committed — resumable, parallel, the standard for anything > ~100 MB.
- **Presigned URLs**: time-limited signed URLs so clients upload/download *directly* to object storage, bypassing your servers — the pattern for every upload flow in interviews (your API issues the URL, the bytes never touch your fleet).
- **Storage classes / lifecycle**: hot (Standard) → infrequent (IA) → archive (Glacier: minutes–hours retrieval) with lifecycle rules auto-transitioning by age; cost drops ~10–20× hot → archive.
- **Event notifications**: object-created events → SQS/Lambda (triggers thumbnailing, virus scan, indexing pipelines).

```
client —(1) POST /upload-url→ API (auth, validate) → returns presigned URL
client —(2) PUT parts directly→ S3 (multipart)
S3 —(3) ObjectCreated event→ queue → workers (thumbnail, transcode, scan)
CDN ← serves the public reads (origin = bucket)
```

Real Production Example

Netflix stores masters + encodes in S3, distributes via Open Connect. Dropbox famously *left* S3 for its own exabyte-scale "Magic Pocket" (custom erasure-coded storage) when scale economics justified building — the canonical build-vs-buy story. Every photo app (Instagram, Airbnb listings) = object storage + CDN + presigned uploads.

Trade-offs / Common Mistakes

- Not a filesystem: no rename (copy+delete), no append, latency in tens of ms, LIST is expensive — don't design chatty small-file access patterns on it.
- Mistake: proxying file bytes through your API servers (bandwidth + memory burn) instead of presigned direct transfer.
- Mistake: millions of tiny objects (per-request overhead dominates — batch/pack them); hot-prefix throttling under extreme request rates (spread key prefixes).
- Storing DB-like mutable state in objects (race conditions; use a database).

Interview Questions

1. Design the upload path for 10 GB video files from mobile clients. (presigned multipart, resume, event-driven processing)
 2. How does erasure coding get 11 nines cheaper than 3× replication?
 3. Photos older than 1 year are viewed 100× less — cut the storage bill. (lifecycle tiering + CDN for the hot set)
-

6.2 Block Storage (EBS, Persistent Disk, SAN)

Core Concept & Internals

Raw **fixed-size blocks** presented as a virtual disk to ONE server (attach → format with a filesystem → mount). This is what your **databases** run on: lowest latency (sub-ms), high IOPS (provisioned: e.g., EBS io2 up to 256k IOPS), random read/write in place. Cloud block stores replicate blocks within an AZ and support **snapshots** (incremental, stored in object storage) for backup/cloning.

Trade-offs: expensive per GB vs object storage, capacity bounded, single-attach (mostly), AZ-bound (snapshot/restore to move) — and network-attached block storage adds a subtle failure mode: the disk can "hang" when the storage network degrades (EBS incidents), which looks like a frozen database.

Interview line: *databases and boot volumes → block; shared media/backups → object; shared POSIX across many hosts → file.*

Interview Questions

1. Why does Postgres run on block storage, not S3? (random in-place I/O, fsync latency, POSIX)
 2. What are snapshots and how do they enable fast DB clones for staging?
-

6.3 File Storage (NFS, EFS, Filestore)

Core Concept & Internals

A **shared POSIX filesystem over the network**: hierarchy, permissions, appends, partial writes, many concurrent clients (NFS/SMB protocols; managed: EFS — elastic, multi-AZ, pay-per-use). The fit: legacy apps expecting a filesystem, shared content (CMS/WordPress uploads), ML training data mounted across a GPU fleet, home directories.

Costs: network round trip per metadata op (small-file workloads crawl), locking semantics across clients are subtle, throughput below local NVMe. Interview posture: file storage is the *compatibility* choice; for new designs prefer object storage unless you specifically need POSIX-shared semantics.

6.4 "Blob Storage" and Handling Large Binary Data in Databases

"Blob storage" in practice = object storage (Azure literally names it Blob Storage). The recurring interview point: **don't store big binaries in the database**. The pattern is always:

```
DB row:    { id, owner, size, content_type, checksum, s3_key, status }
Object:    s3://bucket/ab/cd/abcd1234... (bytes)
Why: DB stays small/fast (backups, replication, cache), storage is cheap,
CDN can serve bytes, and the DB transactionally owns METADATA + lifecycle.
```

Keep the two consistent: write metadata row (status=pending) → upload → mark committed; a janitor reconciles orphans (objects without rows, rows without objects). Small exception: tiny blobs (< a few KB, e.g., avatars) sometimes inline fine — know the threshold argument, not a dogma.

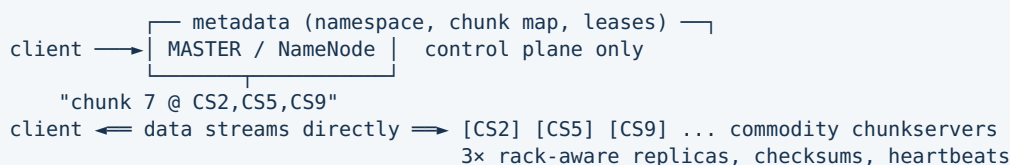
6.5 Distributed File Systems (GFS/HDFS lineage, Colossus, Ceph)

Why Interviewers Ask This

"How would you store petabytes with commodity machines" is the design behind GFS — and its ideas (chunking, metadata/data separation, replication pipelines) power everything from HDFS to your object store, so it doubles as an internals question.

Core Concept & Internal Working (GFS/HDFS model)

- Files split into large **chunks/blocks** (64–256 MB — large to amortize metadata and favor streaming reads).
- A **metadata service** (GFS master / HDFS NameNode) holds the namespace + chunk → server mapping *in memory*; **data servers** (chunkservers/DataNodes) store chunks on local disks, checksummed.
- Data path bypasses the master**: client asks master "where is chunk 7?" then streams directly from/to chunkservers — the master is a control plane, never a data bottleneck.
- Replication** (3×, rack-aware) or erasure coding; heartbeats detect dead servers; master re-replicates lost chunks automatically.
- Designed assumptions: huge sequential reads/appends, failure is constant, throughput >> latency. The NameNode SPOF/memory ceiling drove HDFS federation + HA standbys, and Google's successor **Colossus** distributed the metadata itself (metadata stored in Bigtable — which runs on Colossus; bootstrap via a small Paxos core).
- Ceph**: no central mapping at all — placement computed by the **CRUSH** hash algorithm (any client can compute where data lives), giving object/block/file interfaces on one substrate.



Real Production Example

GFS → MapReduce → the entire big-data era; HDFS under Hadoop/Spark at Yahoo, Meta, LinkedIn (though cloud object storage has largely replaced new HDFS deployments); Colossus under nearly all Google storage (Bigtable, Spanner, YouTube); Ceph under many private clouds.

Interview Questions

1. Why 64 MB+ chunks? (metadata volume, seek amortization, streaming)

- Why does the data path bypass the master, and what still limits the master? (namespace size in RAM, ops/sec — hence federation/Colossus)
- Replication vs erasure coding trade-off here? (3× fast recovery/reads, 1.5× EC cheaper but reconstruction costs CPU/IO)

6.6 Storage Trade-offs (the decision table)

	latency	throughput	shared?	mutability	cost/GB	use for
local NVMe engines	~100 μs	GB/s	no	full	low*	caches, temp, LSM
block (EBS) volumes	<1 ms	provisioned	1 host	full	\$\$\$	databases, boot
file (EFS) ML data	~ms	good	many hosts	full POSIX	\$\$\$\$	legacy/shared POSIX,
object (S3) lake	10–100 ms	huge ()	everyone	replace-only	\$	media, backups, data
archive backups	min-hours	batch	everyone	replace-only	¢	compliance, cold
*local NVMe is ephemeral on cloud instances – data dies with the host						

Cross-cutting decision drivers to narrate in interviews:

- Access pattern first:** random small writes → block; streaming large reads → object; shared POSIX → file.
- Durability model:** replication (fast, 3× cost) vs erasure coding (1.5×, CPU/rebuild cost) vs snapshot+backup (point-in-time, not HA).
- Cost architecture:** hot/warm/cold tiering with lifecycle automation is expected in any large-media design; egress fees often dominate — put a CDN in front.
- Consistency:** object stores give per-object atomicity only — no cross-object transactions; databases own coordination.

Mock scenario answers you should be able to produce instantly

- Video platform: object (masters + renditions) + CDN + archive tier for cold originals; metadata in DB.
- OLTP database: block storage with provisioned IOPS + snapshots + WAL archiving to object storage.
- ML training corpus: object as source of truth, file/parallel FS or local NVMe cache at the GPUs.
- Log/analytics lake: object storage + columnar files (Parquet) + lifecycle to archive.

Module 6 Cheat Sheet

OBJECT	HTTP objects, immutable, flat keys. 11-nines via erasure coding + scrubbing. S3 now strongly consistent. Multipart uploads; PRESIGNED URLs (never proxy bytes); lifecycle tiering hot-IA-archive; events → processing pipelines.
BLOCK	raw disk for ONE host; sub-ms, provisioned IOPS; databases live here. Snapshots (incremental, to object storage). AZ-bound; net-attached can hang.
FILE	shared POSIX over network (NFS/EFS). Compatibility + shared-mount niches. Metadata round trips kill small-file workloads.
BLOB+DB	bytes in object storage, metadata row in DB (key, checksum, status); reconcile orphans. Never blob columns for media.
DFS	GFS/HDFS: big chunks, in-RAM metadata master (control plane only), direct client-chunkserver data path, 3× rack-aware replicas, heartbeat re-replication. Colossus: distributed metadata. Ceph: CRUSH computed placement.
TRADE	latency: NVMe<block<file<object<archive. cost: reverse. random-write → block; streaming/shared/scale → object; POSIX-shared → file. CDN for egress.

Top Interview Questions (Module 6)

1. Upload pipeline for 10 GB videos (presigned multipart + events). 2. Erasure coding vs replication math. 3. Why DBs need block not object storage. 4. GFS architecture and why data bypasses the master. 5. Storage-bill reduction via lifecycle tiering. 6. Blob-in-DB vs object storage + metadata. 7. What breaks with millions of tiny objects. 8. Snapshot-based backup/clone strategy. 9. Dropbox's build-vs-buy S3 story. 10. Pick storage for four given workloads (table above).

Common Mistakes Recap

Proxying file bytes through app servers • blobs in the database • treating S3 as a filesystem (renames, appends, chatty small I/O) • ignoring lifecycle/tiering in cost questions • citing S3 eventual consistency (outdated) • no orphan reconciliation between DB metadata and objects • forgetting egress/CDN in media designs.

Mock Interview Exercise

"Design storage for a Google Photos clone: 500M users, 1B photo uploads/day (avg 3 MB), originals kept forever, thumbnails served hot, p99 view < 200 ms worldwide." Expected: presigned multipart uploads → object storage (originals, EC-coded, lifecycle: 30d hot → IA → archive) + thumbnail derivation pipeline via events → hot thumbnail tier + CDN; metadata (album/photo rows, checksums, s3 keys) in sharded DB; dedupe by content hash; capacity math: ~3 PB/day ingest → tiering is the whole cost story; failure: orphan janitor, checksum scrubbing, multi-region replication for DR.

Module 7 — Microservices & Resilience Patterns

This module is where staff-level interviews live: distributed transactions, idempotency, and the resilience patterns (circuit breaker, bulkhead, timeout budgets) that decide whether one slow dependency takes down your whole product.

7.1 Monolith vs Microservices

Why Interviewers Ask This

It's an architecture-judgment question. The wrong answer is dogma in either direction; the senior answer is that microservices are an *organizational* scaling tool that trades local complexity for distributed complexity.

Core Concept

- **Monolith:** one deployable, one process, in-process calls, one database, one transaction boundary. A *modular* monolith (enforced internal boundaries) is the modern respectable default.
- **Microservices:** independently deployable services, each owning its **own data** (no shared database — the defining rule), communicating over the network (gRPC/REST/events).

What microservices actually buy: independent deploys (team velocity at 50+ engineers — Conway's law is the real driver), independent scaling (resize only the hot service), fault isolation (if designed for it), tech heterogeneity. What they cost: every in-process call becomes a network call (latency, partial failure, retries), every transaction becomes a saga, every debug session becomes distributed tracing, and you now run a platform (discovery, mesh, CI/CD, observability).

```
Monolith: [ UI | orders | payments | inventory ] — [ one DB ] 1 deploy, ACID
Micro:    [orders svc] — DB1  [payments svc] — DB2  [inventory svc] — DB3
           └─ network calls + events; consistency = YOUR problem now
```

Real Production Example

Amazon and Netflix decomposed monoliths when team count, deploy contention, and scaling asymmetry forced it. Counter-examples interviewers respect: Shopify runs a famously successful *modular monolith* at enormous scale; Amazon Prime Video published a 2023 case where merging microservices back into a monolith cut costs ~90% for that workload. Segment's "goodbye microservices" post is another classic. Moral: decomposition is contextual, not virtuous.

Common Mistakes

- Distributed monolith: services that must deploy together and share a database — all of the costs, none of the benefits. The shared-DB smell is the #1 thing to call out.
- Microservices at a 6-person startup ("you must be this tall": platform maturity, observability, on-call capacity).
- Nano-services: a service per entity; boundaries should follow *business domains* (DDD bounded contexts) sized so one team owns each.

Interview Questions

1. What forces would make you split a monolith, and what must be true first?
2. What defines a good service boundary? (domain-driven, own data, low chattiness across the boundary, one team)

3. What is a distributed monolith and how do you detect one? (lockstep deploys, shared DB, chatty synchronous chains)

7.2 Service Discovery

Core Concept & Internal Working

In a dynamic fleet (autoscaling, deploys, failures), "where is service B right now" needs an answer that updates in seconds.

- **Registry-based:** instances register (and heartbeat) with a registry — Consul, Eureka, etcd/ZooKeeper, or the Kubernetes API. **Client-side discovery:** clients fetch the instance list and load-balance themselves (fewer hops, smarter LB — Netflix Eureka+Ribbon lineage). **Server-side:** clients hit a stable VIP/LB that resolves instances (simpler clients — Kubernetes Services, cloud LBs).
- **DNS-based:** SRV/headless-service records — simple, but caching makes it sluggish.
- **Service mesh** (Istio/Linkerd, Envoy sidecars): discovery + LB + mTLS + retries + telemetry pushed into a sidecar proxy via a control plane (xDS) — the app just calls `localhost`; the mesh does the rest.
- Health-integration is the point: discovery must *stop* routing to bad instances (heartbeats, readiness probes, outlier ejection). Registry availability matters: Eureka chose AP (serve stale instance lists during partitions — stale routing beats no routing); ZooKeeper-based systems chose CP.

```
pod starts -register/heartbeat→ registry (Consul/K8s API)
client -"who is payments?"→ registry → [10.0.3.7, 10.0.9.2, ...]
client — picks instance (P2C) → payments pod (or Envoy sidecar does all this)
```

Interview Questions

1. Client-side vs server-side discovery trade-offs?
2. Why did Eureka choose availability over consistency for the registry?
3. What does a service mesh move out of application code, and what does the sidecar cost? (per-hop latency ~ms, fleet resource overhead, control-plane blast radius)

7.3 API Gateway in a Microservice Architecture

(Networking view in Module 2.5; here, the architecture role.)

- One front door: routing to services, authn/z (validate JWT once, pass claims), rate limiting/quotas, canary/traffic splitting, protocol translation (REST outside, gRPC inside), response aggregation.
- **BFF pattern** (Backend-for-Frontend): a gateway variant per client type (mobile/web) that shapes and aggregates APIs per experience — avoids one generic gateway becoming everyone's compromise; this is how Netflix's edge evolved.
- Anti-pattern to name: business logic creeping into the gateway → a shared bottleneck owned by no product team ("the ESB reborn"). Keep it to cross-cutting concerns; keep it horizontally scaled and stateless.

Interview Questions

1. Gateway vs service mesh — which handles what? (gateway = north-south/edge; mesh = east-west/inter-service)
2. When do BFFs beat one gateway?

7.4 Resilience: Timeout, Retry, Circuit Breaker, Bulkhead

Why Interviewers Ask This

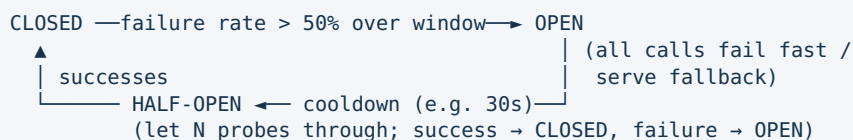
This quartet is *the* "senior engineer who has been paged" question set. The scenario is always: one dependency degrades — does your system degrade proportionally, or collapse entirely?

Core Concept & Internal Working

Timeouts — every network call has one, sized from the dependency's p99 ($\sim p99 \times 1.5$, not a folklore 30 s). No timeout = threads/connections accumulate behind a slow dependency until exhaustion (the actual kill mechanism in most cascades). **Deadline propagation**: the edge sets a total budget (e.g., 800 ms); each hop passes the remaining budget down (gRPC does this natively) so nobody works on a request the user already abandoned.

Retries — handle *transient* failures. Rules: only idempotent operations (or with idempotency keys), exponential backoff + **jitter**, small caps (2–3), and a **retry budget** (e.g., retries $\leq 10\%$ of requests) so retries can't multiply load during an incident. The amplification trap: 3 layers each retrying $3\times$ = up to $27\times$ traffic onto the struggling dependency — retry at *one* layer (usually the caller closest to the failure), not every layer.

Circuit breaker — stop calling a dependency that's failing; fail fast and give it room to recover.



Fail-fast matters twice: your latency stays flat (no waiting on a corpse), and the dependency isn't hammered while it recovers. Every breaker needs a **fallback**: cached/stale data, default value, degraded feature, or an honest error. Modern placement: in the mesh/sidecar (Envoy outlier detection) or libraries (Resilience4j; Hystrix is the ancestor).

Bulkhead — partition resources so one failing dependency can't consume everything: separate connection pools / thread pools / semaphores *per dependency* (payment pool $\neq$ recommendation pool), per-tenant quotas, even per-workload deployments (batch vs interactive). Named after ship compartments: one flooded compartment shouldn't sink the ship. The cascade it prevents: recommendations slows $\rightarrow$ all 200 worker threads end up parked waiting on it $\rightarrow$ checkout (which never needed recommendations) has no threads left $\rightarrow$ total outage from a cosmetic feature.

Real Production Example

Netflix built and open-sourced Hystrix after exactly these cascades; their standard story: recommendations down $\rightarrow$ serve popular titles (fallback), checkout unaffected (bulkhead). AWS SDKs ship retry budgets + adaptive retry. Google SRE codified deadline propagation and "retry amplification" postmortems.

Common Mistakes

- Retrying non-idempotent calls (double charge), retrying on every layer, no jitter (synchronized retry waves), timeout longer than the caller's own deadline.
- Circuit breaker without fallback (= faster errors, same outage UX).
- One global thread/connection pool for all dependencies.

Interview Questions

1. Payment provider p99 went from 200 ms $\rightarrow$ 20 s. Walk through what happens with and without each pattern.
2. How do you size timeouts and where do deadlines come from?
3. Explain retry amplification and how budgets bound it.

7.5 Idempotency

Why Interviewers Ask This

Retries + at-least-once delivery are everywhere (7.4, Module 5), so duplicate requests are a *certainty*. Idempotency is what makes them safe; payments interviews hinge on it.

Core Concept & Internal Working

An operation is idempotent if executing it N times has the same effect as once. GET/PUT/DELETE are naturally idempotent; "create order", "charge card", "increment balance" are not — you make them idempotent with **idempotency keys**:

```
POST /charges Idempotency-Key: 7f3e-...-a12b (client-generated UUID per logical op)

server, atomically (single DB transaction):
  INSERT INTO idempotency_keys(key, status='in_progress') ON CONFLICT →
    • conflict + completed → return the STORED response (no re-execution)
    • conflict + in_progress → 409/retry-later (a concurrent duplicate is running)
  ... execute business logic, write business rows ...
  UPDATE idempotency_keys SET status='completed', response=... -- same txn/commit
```

Design points interviewers probe:

- The key must cover the *logical operation* (client retries reuse it; a genuinely new attempt gets a new key) — key scoping is a client-contract question.
- Dedup record and business effect must commit **atomically** (same DB transaction, or a unique constraint on the business row itself, e.g., `UNIQUE(order_id)` on payments). A Redis-based "seen set" checked before a separate DB write has a crash window that breaks the guarantee.
- Store the original *response* so retries return identical results.
- Keys expire (Stripe: 24 h) — bounded storage, and clients shouldn't retry across days.
- Alternative formulations: natural idempotency by design (`SET status='paid'`; insert-only ledgers keyed by event ID; state machines where transitions are idempotent), and **fencing/versioning** (optimistic concurrency: `UPDATE ... WHERE version=7`) for stale-duplicate protection.

Real Production Example

Stripe's `Idempotency-Key` header is the industry-reference design (they return the stored response byte-for-byte). Every serious payment, ordering, and provisioning API does the equivalent internally.

Interview Questions

1. Client timeout on `POST /orders` → user clicks again. Walk both requests through your idempotent design.
2. Why is check-then-act in Redis + DB insufficient?
3. Make "transfer \$100 A→B" idempotent. (transfer row keyed by `transfer_id`, ledger entries derived from it, unique constraints enforce once)

7.6 Distributed Transactions: 2PC vs Saga

Why Interviewers Ask This

Once each service owns its DB, "order + payment + inventory must all succeed" has no ACID answer. This is the definitive staff-level microservices question.

Core Concept & Internal Working

Two-Phase Commit (2PC): coordinator asks all participants to *prepare* (lock + promise), then *commit*. Gives atomicity, but: **blocking** (participants hold locks while waiting; a crashed coordinator leaves everyone stuck "in doubt"), coordinator SPOF, latency of 2 round trips × slowest participant, and poor availability (any participant down = no transaction). Used inside databases and some brokers (XA, Kafka's internal transactions), avoided *between* microservices.

Saga: break the transaction into a sequence of *local* transactions, each with a **compensating action** to semantically undo it if a later step fails.

```
Order saga: create order → reserve inventory → charge payment → confirm
failure at charge:                ▲                x
compensate ← release inventory ← cancel order    (run compensations in reverse)
```

- **Orchestrated saga:** a coordinator (state machine — Temporal, AWS Step Functions, or a saga table + worker) explicitly commands each step and each compensation. Visible, debuggable, testable; the orchestrator must itself be durable (its state in a DB / workflow engine).
- **Choreographed saga:** each service reacts to the previous event (OrderCreated → InventoryReserved → PaymentCharged...). No central coordinator, but the workflow is invisible and cyclic-dependency-prone; fine for 2–3 steps, painful beyond.
- Sagas are **not ACID**: no isolation — intermediate states are visible (order exists while payment pending) → model them explicitly as statuses ("PENDING_PAYMENT") and design the UX for them; compensations can fail (retry + DLQ + human escalation queue); every step and compensation must be **idempotent** (messages redeliver).
- Some things can't be compensated (sent email, fired missile) — order steps so the hardest-to-undo runs **last** ("pivot" step), or use reservations (auth-then-capture in payments is exactly this: authorization is reversible, capture is the pivot).

Real Production Example

Uber trip lifecycle and Airbnb reservations run saga-style flows (Airbnb built and open-sourced ideas around this; Uber built Cadence → Temporal precisely to make orchestration durable). Payment auth/capture is the everyday saga everyone has used.

Common Mistakes

- Proposing 2PC across microservices without naming blocking/availability costs.
- Sagas without idempotent + retryable compensations, or without visible intermediate states.
- Forgetting the pivot-step ordering trick.

Interview Questions

1. Book flight + hotel + charge card atomically — design it and enumerate every failure point.
2. Why is 2PC "blocking" and what does that mean operationally?
3. Orchestration vs choreography for a 7-step fulfillment flow?

7.7 Outbox Pattern (and the Dual-Write Problem)

Why Interviewers Ask This

Every event-driven design contains the sentence "the service writes the DB *and* publishes an event". Interviewers pounce: those are two systems — one can fail. The outbox is the canonical fix and a reliable senior discriminator.

Core Concept & Internal Working

Dual-write bug: commit to DB; publish to Kafka — crash between them ⇒ state changed but no event (downstream never learns), or reversed order ⇒ event for a rolled-back change. There is no distributed transaction across DB + broker (worth having).

Outbox: write the event *into the same database transaction* as the state change, then a separate mechanism ships it to the broker.

```
BEGIN;
  UPDATE orders SET status='PLACED' WHERE id=42;
  INSERT INTO outbox(id, aggregate, type, payload) VALUES (uuid,'order-42','OrderPlaced',{...});
COMMIT;
    -- atomic: both or neither
```

├ relay option A: poller — SELECT unpublished → publish → mark published
└ relay option B: CDC — Debezium tails the WAL/binlog → Kafka (no polling, lower latency, no extra write load)

Delivery is AT-LEAST-ONCE (relay can crash post-publish pre-mark) ⇒ consumers must be idempotent (7.5). Ordering: publish per-aggregate in commit order, key by aggregate ID. Prune published rows.

The inverse sibling: **listen-to-yourself / CDC-first** (write only the event, derive state from it) and the transactional-log-tailing family. For inbound messages, the **inbox pattern** mirrors it (store message ID + effects atomically — i.e., 7.5's idempotency table).

Real Production Example

Debezium's outbox router is the de facto standard implementation; documented in production at scores of companies (e.g., banking and e-commerce engineering blogs). Any "DB + Kafka + no lost events" claim in your interview design should say "outbox via CDC" to be credible.

Interview Questions

1. Why not just publish to Kafka inside the request handler after commit? (crash window; also before commit is worse — event for nothing)
2. Poller vs CDC relay trade-offs? (latency, DB load, ops complexity, ordering control)
3. What delivery guarantee does the outbox give and what must consumers do? (at-least-once; idempotency)

Module 7 Cheat Sheet

MONO vs MICRO	Microservices = org-scaling tool; costs: network, sagas, tracing, platform. Own-your-data rule; distributed monolith = worst of both. Modular monolith is a respectable default (Shopify).
DISCOVERY	registry+heartbeats (Consul/K8s) client-side vs server-side; mesh sidecars (Envoy/xDS) add LB+mTLS+retries+telemetry. Registry is AP by design (stale beats none).
GATEWAY	edge concerns only (auth, rate limit, routing, BFF aggregation); north-south; mesh = east-west. No business logic.
TIMEOUT	every call; $\approx p99 \times 1.5$; propagate DEADLINES down the chain.
RETRY	idempotent-only, backoff+JITTER, cap 2–3, retry BUDGET, one layer only (amplification: $3 \times 3 \times 3 = 27 \times$).
CIRCUIT BREAKER	closed-open (fail fast + FALLBACK) → half-open probes → closed.
BULKHEAD	pools/quotas per dependency & tenant; cosmetic feature must not starve checkout.
IDEMPOTENCY	client key per logical op; dedup record + effect in SAME txn; store response; unique constraints; version/fencing for stale dupes.
2PC	atomic but blocking, coordinator SPOF, in-doubt locks — not between microservices.
SAGA	local txns + compensations; orchestrated (durable coordinator, visible) vs choreographed (≤ 3 steps); no isolation → explicit PENDING states; pivot (hardest-to-undo) last; all steps idempotent.

OUTBOX

event INSERT in same txn as state change; relay via poller or CDC (Debezium); at-least-once → idempotent consumers; fixes dual-write.

Top Interview Questions (Module 7)

1. Split this monolith: boundaries, order, prerequisites. 2. One dependency at 20 s p99 — full resilience walkthrough. 3. Retry amplification math. 4. Stripe-style idempotency implementation detail. 5. Flight+hotel+payment saga with failure at each step. 6. Why 2PC is avoided between services. 7. Dual-write bug + outbox/CDC.
2. Mesh vs gateway responsibilities. 9. Choreography vs orchestration at 7 steps.
3. Design "exactly-once-looking" order placement over at-least-once infrastructure.

Common Mistakes Recap

Shared database between services • retry-everywhere without budgets/jitter • timeouts unset or unpropagated • breaker without fallback • one pool for all dependencies • idempotency check outside the transaction • sagas without compensations/idempotency/visible states • dual writes • business logic in the gateway • microservices without platform maturity.

Mock Interview Exercise

"Design checkout for a marketplace: order service, inventory service, payment service (external PSP), notification service. Requirements: no double charges, no overselling, PSP p99 sometimes spikes to 10 s, notifications may lag." Expected: orchestrated saga (reserve inventory → auth payment → confirm → capture) with compensations (release, void); idempotency keys end-to-end (client → order → PSP); outbox+CDC for events; PSP wrapped in timeout (deadline-derived) + breaker + fallback (queue order in PENDING_PAYMENT, async complete); bulkheaded PSP pool; notification as lagging consumer group; enumerate the crash points and show each is safe.

Module 8 — Reliability & High Availability

This module turns Module 1's availability math into architecture: redundancy, failover, disaster recovery, health checking, and just enough consensus (Raft) to survive a deep dive.

8.1 High Availability & Redundancy

Why Interviewers Ask This

"Make it highly available" is in every prompt. They grade whether you apply redundancy *at every layer* and understand that redundancy without automated failover and without protection from correlated failure is decoration.

Core Concept

HA = eliminating single points of failure so component failures don't become outages. Redundancy models:

- **Active-active:** all replicas serve traffic (LB across them). Capacity is used, failover is instant (traffic just shifts), but state must be shared/synchronized.
- **Active-passive:** standby takes over on failure. Simpler state story; failover takes detection + promotion time; the standby is money doing nothing — and untested standbys rot ("the failover that never works when finally needed").
- **N+1 / N+2 provisioning:** capacity such that losing 1 (or 2) units leaves enough. N+2 lets you do maintenance (1 down) and still survive a failure (another down).

Redundancy must be applied per layer and *across failure domains*:

layer	redundancy	failure domain to cross
DNS	multi-provider	provider
LB	≥ 2 , anycast/VRRP	host, AZ
app	N+2 stateless replicas	host, AZ
cache	replicas per shard	host, AZ
DB	leader + replicas, quorum	host, AZ (region for DR)
region	multi-region (8.3)	region, provider(!)

The enemy is **correlated failure**: two "redundant" replicas on one rack, one AZ, one bad config push, one shared certificate, one deploy train. Real-world outages are dominated by correlated causes (config/deploy), which is why gradual rollouts and cell-based isolation matter as much as replica count.

Real Production Example

AWS builds AZs as physically separate failure domains and tells you to spread across ≥ 3 — a quorum needs a majority, and with 2 AZs a whole-AZ loss removes half your nodes. Google and AWS both run **cell-based architectures** (independent shards-of-everything called cells; a bad deploy or poison request takes out one cell, not the fleet) — AWS explicitly describes this for its control planes.

Interview Questions

1. Why 3 AZs and not 2? (majority quorums; 2 AZs = split-brain-or-downtime dilemma)
2. Active-active vs active-passive for a database tier?
3. Your two replicas share a rack switch — what's your actual availability?

8.2 Failover, Health Checks, Heartbeats

Why Interviewers Ask This

Failover is where HA succeeds or fails in practice, and it hides the hardest problem in distributed systems: *you cannot distinguish "dead" from "slow/ partitioned" over a network.*

Core Concept & Internal Working

Detection — heartbeats and health checks:

- **Liveness** ("process is alive — restart if not") vs **readiness** ("can serve traffic — route away if not") — Kubernetes encodes the distinction; conflating them causes restart storms of overloaded-but-healthy pods.
- **Shallow checks** (port open, `/healthz` returns 200) vs **deep checks** (can reach DB/dependencies). Deep checks catch more but propagate dependency failures: if the DB blips, every instance "fails" its deep check simultaneously and the LB removes the entire fleet. Rule: fail deep checks only for *local* problems; report dependency state separately. Add **hysteresis** (fail after 3 consecutive misses, restore after 2 passes) to stop flapping.
- Heartbeat tuning: interval $\times$ threshold = detection time. $1\text{ s} \times 3 =$ fast but false-positive-prone (GC pause = "death"); $10\text{ s} \times 3 =$ calm but 30 s of black hole. There is no correct constant — say the trade-off aloud (phi-accrual detectors adapt it statistically; Cassandra uses this).

Failover mechanics (DB leader example):

- | | |
|------------|---|
| 1. detect | leader misses N heartbeats (is it dead? or a partition? unknowable) |
| 2. elect | remaining nodes pick new leader — must be QUORUM-based (8.4) |
| 3. fence | old leader must be prevented from acting if it returns:
epoch/fencing tokens (storage rejects writes from old epoch),
or STONITH ("shoot the other node in the head") |
| 4. repoint | clients/proxies discover the new leader (service discovery, VIP) |
| 5. verify | data: async-replication tail may be LOST (Module 4.2) — reconcile |

Failback (returning to the original) is a second, deliberate operation — never automatic ping-pong.

Real Production Example

GitHub's 2018 43-second network partition triggered automated cross-DC MySQL failover; the old primary had writes the new one lacked → hours of reconciliation and degraded service. It's the canonical case study: failover automation + async replication + no clean fencing = split-brain data divergence. Patroni (Postgres) and orchestrator (MySQL) implement quorum + fencing to get this right.

Common Mistakes

- Deep health checks that fail fleet-wide on a dependency blip.
- Failover without fencing (split brain), or failover that's never been drilled.
- Aggressive detection timers that failover on a GC pause; DNS-based repointing with TTL assumptions (clients cache).

Interview Questions

1. Walk me through DB leader failover step by step — where can it go wrong?
 2. Liveness vs readiness; shallow vs deep — design the health checks for an API pod.
 3. How do fencing tokens prevent a zombie leader from corrupting state?
-

8.3 Disaster Recovery (DR)

Why Interviewers Ask This

"A whole region goes down" is a guaranteed follow-up in senior interviews. They want RTO/RPO vocabulary, the strategy ladder, and honesty about cost.

Core Concept

- **RTO** (Recovery Time Objective): how long until service is restored.
- **RPO** (Recovery Point Objective): how much data you may lose (time since last replication/backup).

The strategy ladder — each step ~10× the cost of the previous:

strategy	RTO	RPO	cost	how
backup & restore	hours–days	hours	\$	backups in object storage, cross-region copies, IaC to rebuild
pilot light	tens of min	minutes	\$\$	data replicated continuously; minimal core infra idling in DR region
warm standby	minutes	seconds–min	\$\$\$	scaled-down full stack running; scale up + shift traffic
multi-site active-active	~0	~0 (sync)	\$\$\$\$	active-active regions, traffic split, data replicated (hard part!)

Active-active's hard part is the data plane: sync cross-region replication costs 50–200 ms per write (physics), so most designs choose per-record home regions or async + conflict handling (Module 4.2) and accept a small RPO.

Non-negotiables interviewers listen for: **DR that isn't tested doesn't exist** (game days, chaos drills — Netflix Chaos Kong simulated region evacuation regularly); backups must be *restore-tested* and isolated from the primary blast radius (separate account/credentials — ransomware and fat-fingered deletes follow your replication); dependencies (DNS, auth, secrets, container registry!) must also survive the region loss or your recovery tooling is down too.

Real Production Example

The 2021 OVH datacenter fire destroyed customer data whose "backups" lived in the same building. AWS's us-east-1 incidents repeatedly took down *other* regions' control planes for customers whose tooling depended on us-east-1 — the "hidden global dependency" lesson. Netflix evacuates a region in minutes because they rehearse it.

Interview Questions

1. Define RTO/RPO for a payments system vs a photo archive, and pick strategies.
2. Your region died. Recovery step one? (traffic off via DNS/anycast — then data story)
3. What commonly breaks *during* DR that planning missed? (auth/secrets/DNS/registry dependencies, stale runbooks, capacity not reserved in DR region)

8.4 Consensus & Raft (Interview Level)

Why Interviewers Ask This

Consensus underlies leader election, distributed locks, and every "how does etcd/ ZooKeeper actually work" follow-up. Interview-level Raft — roles, election, log replication, quorum intuition — is expected at staff level.

Core Concept

Consensus = getting a cluster of unreliable nodes to agree on a sequence of values (a replicated log) such that agreement survives minority failure and never forks. FLP impossibility says you can't guarantee liveness in a fully

async network — practical systems (Paxos, Raft, ZAB) use timeouts/randomization and guarantee **safety always, liveness usually**.

The quorum arithmetic that answers half of all follow-ups: with **2f+1** nodes you tolerate **f** failures ($3 \rightarrow 1$, $5 \rightarrow 2$). Two majorities always intersect — that intersection is why committed data can't be lost or forked, and why **even node counts add nothing** (4 nodes still tolerate only 1).

Internal Working — Raft in one page

```
roles:    FOLLOWER —election timeout (randomized 150–300ms)→ CANDIDATE
          CANDIDATE —majority votes→ LEADER —heartbeats→ followers
          (higher term seen at any time → step down to follower)

election: candidate++term, votes for self, requests votes.
          voters grant one vote per term, and ONLY to candidates whose log is
          at least as up-to-date ← this rule preserves committed entries.
          Randomized timeouts make split votes rare (retry with new term).

log replication:
  client → LEADER: append entry to local log
  leader → AppendEntries RPC to all followers
  majority ACK → entry COMMITTED → apply to state machine → reply to client
  followers learn commit index via subsequent heartbeats
  conflicting follower logs are overwritten to match the leader (leader's log wins)

terms:    logical clock; every message carries it; stale-term messages rejected –
          this is the built-in FENCING that neutralizes zombie leaders.
```

Extras worth one sentence each: **log compaction/snapshots** (logs can't grow forever), **membership changes** go through the log (joint consensus/single-server changes), **leases/ReadIndex** let leaders serve linearizable reads without a full log round trip, and Multi-Paxos $\approx$ Raft in steady state (Raft's contribution is understandability: strong leader + explicit rules).

Where it runs: etcd (Kubernetes' brain), Consul, ZooKeeper (ZAB), CockroachDB/ TiDB/Spanner (consensus per data range), Kafka KRaft, cloud control planes. Consensus is expensive (quorum round trip per write) — use it for *coordination and metadata* (locks, leases, leader election, configs, schema), not bulk data.

Common Mistakes

- Deploying even-numbered clusters; putting a 3-node quorum in 2 AZs.
- Using etcd/ZK as a general database (it's a coordination kernel; small data, modest write rates).
- Claiming a Raft leader can serve stale-free reads with zero protocol (needs lease/ReadIndex — otherwise a deposed leader can serve stale reads).

Interview Questions

1. Why odd cluster sizes? Why does 5 tolerate 2? What breaks across 2 AZs?
2. Walk through a Raft election after leader death; how are split votes avoided; why can't a stale-log node win?
3. Why is committed data safe across leader changes? (majority intersection + up-to-date vote rule)

8.5 Split Brain

Why Interviewers Ask This

It's the catastrophic failure mode of every HA design — two nodes both believing they're the leader, both accepting writes, data diverging irreversibly. Interviewers inject it as "what if the network partitions?"

Core Concept & Internal Working

Cause: partition (or pause — GC, VM freeze) makes the standby think the leader died; both act as leader. Async-replicated pairs with naive failover are the classic victims.

Defenses (say all three):

1. **Quorum**: only the side with a majority may elect/serve as leader; the minority side steps down (this is *the* reason for 3+ nodes / 3 AZs). A 2-node pair fundamentally cannot self-arbitrate — it needs a third witness/tiebreaker.
2. **Fencing**: make the old leader *harmless* even if it doesn't know it's deposed — monotonically increasing **fencing tokens/epochs** checked by the storage layer (writes stamped with term 7 rejected once term 8 exists; Raft terms, ZooKeeper zxid, Kafka controller epoch), or STONITH (power off the old node), or storage-level reservations (SCSI-3).
3. **Leases**: leadership expires unless renewed; a paused/partitioned leader's lease lapses before a new one is granted (requires bounded clock drift — mention the GC-pause caveat from Kleppmann's Redlock critique: the pause can outlive the lease *and* the writes land after resume, hence tokens, not just leases).

AZ-a	AZ-b	partition splits 1 vs 2:
[node1]	[node2 node3]	left: 1/3 nodes → NO quorum → steps down (read-only/halt)
old	elects new	right: 2/3 → quorum → new leader, term++
leader	leader t=8	old leader's t=7 writes → REJECTED by storage (fenced) ✓

Real Production Example

GitHub 2018 (above) is the canonical split-brain-adjacent incident. Elasticsearch pre-7.x clusters with `minimum_master_nodes` misconfigured split-brained routinely — the fix (7.x) was building a proper quorum-based coordination layer. Every DB-failover tool's docs (Patroni, orchestrator) are essentially split-brain prevention manuals.

Interview Questions

1. Design leader failover for a 2-node primary/standby pair. (trap: add a witness/quorum — 2 nodes can't do it safely)
2. GC pauses the leader for 40 s; it wakes and keeps writing. What saves you? (fencing tokens at the storage layer — leases alone don't)
3. Where does split brain hide in application-level designs? (job schedulers, cron leaders, cache warmers — anything "only one of me should run")

Module 8 Cheat Sheet

REDUNDANCY	active-active (capacity used, instant shift) vs active-passive (rots untested). N+2. Cross failure domains: host<rack<AZ<region<provider. Correlated failure (config/deploy) beats replica count — canary, cells.
HEALTH	liveness (restart) vs readiness (route away). Deep checks: local failures only, else fleet-wide self-ejection. Hysteresis. Detection time = interval × threshold; slow ≠ dead.
FAILOVER	detect → quorum elect → FENCE (epoch/STONITH) → repoint → reconcile async tail. Drill it. Failback is manual.
DR	RTO (time) / RPO (data). backup→pilot light→warm→active-active (≈10× cost each). Test restores; isolate backups; DNS/auth/secrets/registry must survive too. Untested DR = no DR.
CONSENSUS	2f+1 tolerates f (3→1, 5→2); majorities intersect → no forks; odd sizes; 3 AZs. Safety always, liveness usually (FLP). For coordination & metadata, not bulk data.
RAFT	follower→candidate (randomized timeout)→leader (majority vote, only up-to-date logs electable). Commit = majority append. Terms = fencing.

SPLIT BRAIN Snapshots for compaction; lease/ReadIndex for linearizable reads.
quorum (minority steps down) + fencing tokens checked at storage +
leases (mind GC pauses). 2-node pairs need a witness.

Top Interview Questions (Module 8)

1. Why 3 AZs / odd quorums. 2. Full DB failover walkthrough with failure points.
2. Health-check design that survives a DB blip. 4. RTO/RPO ladder for payments vs photos. 5. Raft election + why committed entries survive. 6. Zombie leader after a GC pause. 7. What breaks during DR that planning missed. 8. Active-active data plane options across regions. 9. Cell-based architecture rationale. 10. Two-node HA pair — why it can't self-arbitrate.

Common Mistakes Recap

Redundancy in one failure domain • deep health checks ejecting the fleet • failover without fencing • untested standbys/backups/DR • even-sized quorums • etcd/ZK as a database • leases without tokens • automatic fallback ping-pong • forgetting recovery tooling's own dependencies.

Mock Interview Exercise

"Your single-region service (LB → 20 pods → Postgres primary + 2 async replicas + Redis) must reach 99.99% and survive region loss with $RPO \leq 1 \text{ min}$, $RTO \leq 15 \text{ min}$." Expected: 3-AZ spread; Patroni-style quorum failover with fencing (semi-sync to cut RPO within region); readiness vs liveness done right; warm standby region: WAL streaming (async, lag-monitored $\leq 1 \text{ min}$), scaled-down stack, DNS/anycast shift runbook, quarterly game day; backup vault in separate account; enumerate correlated- failure defenses (staged deploys, config canaries).

Module 9 — Security

Interviews don't expect a security specialist, but they expect the security *section* of every design: how users authenticate, how services authorize, where tokens live, how abuse is limited, and where secrets and encryption fit. Miss it and you lose easy points; nail it in 3–4 minutes and you look senior.

9.1 Authentication vs Authorization

Core Concept

- **Authentication (authn)**: who are you? (credentials → identity)
- **Authorization (authz)**: what may you do? (identity + policy → allow/deny)

Say them separately, always. Authn happens once at the edge (gateway/identity provider); authz happens on **every request, at the service that owns the resource** — the gateway can pre-check coarse scopes, but object-level checks ("is this *your* order?") belong to the owning service. The most common real-world vulnerability class is **broken object-level authorization** (IDOR — changing `/orders/123` to `/orders/124`), not exotic crypto.

Authz models to name: **RBAC** (roles → permissions; simple, coarse), **ABAC** (attributes/policies: "owner OR same-team AND business-hours"), **ReBAC** (relationship graphs — Google Zanzibar powers Drive/YouTube sharing; check `user` → `viewer` → `doc` edges at massive scale).

Password storage (one-liner expected): salted adaptive hashes — bcrypt/scrypt/ Argon2, never MD5/SHA-fast; plus MFA and breached-password checks.

Interview Questions

1. Where do authn and authz each happen in a microservice architecture?
 2. RBAC vs ABAC vs Zanzibar-style — pick for a Google-Docs-like sharing model.
-

9.2 Session Management vs JWT (Stateful vs Stateless Auth)

Why Interviewers Ask This

"Where does login state live" is asked in nearly every design with users, and the JWT-vs-session trade-off is a precision test: most candidates know JWT, few can articulate revocation.

Core Concept & Internal Working

Server-side sessions: login → random session ID in an HttpOnly, Secure, SameSite cookie → session data in Redis (`session:{id}` with TTL). Every request does a Redis lookup. **Instant revocation** (delete the key), easy "log out everywhere", data can change server-side. Cost: a shared session store every request touches (scale it, replicate it).

JWT (JSON Web Token): signed, self-contained claims.

```
header.payload.signature  (base64url each)
{ "alg": "RS256" } . { "sub": "u123", "roles": ["seller"],
                      "iat": 1719900000, "exp": 1719903600, "iss", "aud" } . SIG
```

Verification = check signature (public key) + exp/iss/aud — NO datastore call.
⇒ any service validates locally; perfect for microservices & API gateways.
△ Signed, NOT encrypted — anyone can read the payload (don't put secrets in it).

The revocation problem (the senior discriminator): a JWT is valid until `exp` no matter what — logout, password change, or ban can't recall it. Standard resolution: **short-lived access token (5–15 min) + long-lived refresh token stored server-side**. Revocation = kill the refresh token; exposure window = access-token TTL. Refresh tokens should be **rotated on every use** with reuse detection (a replayed old refresh token ⇒ theft ⇒ revoke the whole family). For higher-stakes needs, add a denylist of revoked JWT IDs (`jti`) in Redis — at which point you've partially rebuilt sessions, which is exactly the trade-off to articulate.

Key hygiene: RS256/EdDSA (asymmetric) so services verify with a public key (via JWKS endpoint) and only the identity provider signs; rotate keys with `kid` headers; never accept `alg:none`; validate audience.

Real Production Example

Big platforms hybridize: an edge session/gateway exchange (cookie → short internal JWT) so browsers get revocable sessions while internal services get stateless verification. Auth0/Okta/Cognito all implement access+refresh rotation exactly as above.

Common Mistakes

- JWTs in `localStorage` (XSS-stealable) — prefer `HttpOnly` cookies (with CSRF protection) for browsers.
- 24-hour access tokens "for convenience" — that's your breach window.
- "JWT is encrypted" (it isn't); skipping `aud` / `iss` checks (token from service A replayed at service B).

Interview Questions

1. Sessions vs JWT for: a bank web app, a public API, service-to-service calls.
2. User's token is stolen — walk through containment with your design.
3. Design "log out of all devices" for both models.

9.3 OAuth 2.0 & OpenID Connect

Why Interviewers Ask This

"Login with Google" and third-party API access are ubiquitous, and OAuth is the most commonly *mis*-explained protocol in interviews. Getting the roles and the authorization-code flow right is a strong signal.

Core Concept

OAuth 2.0 is delegated authorization: a user (resource owner) grants a third-party app (client) limited access (scopes) to their resources at an API (resource server), issued by an authorization server — *without sharing their password*. **OIDC is a thin identity layer on top** that adds an **ID token** (a JWT about *who the user is*) and a `userinfo` endpoint — OIDC is what "Login with Google" actually is. Access token = for calling APIs; ID token = for knowing the user. Confusing these is the classic mistake.

Internal Working — Authorization Code flow + PKCE (the one to know)

1. app → browser redirect → auth server `/authorize?client_id&scope&redirect_uri`
`&state=xyz&code_challenge=H(v)` (PKCE)
2. user authenticates & consents at the auth server (password never seen by app)
3. auth server → redirect back → app?`code=AUTH_CODE&state=xyz`
4. app backend → POST `/token {code, client_secret, code_verifier=v}`
5. ← `{ access_token, refresh_token, id_token(OIDC) }`
6. app → API with Authorization: Bearer `access_token`; API validates + checks scopes

Why the code indirection: tokens never transit the browser URL (history, logs, referrers); the code is single-use, short-lived, and only exchangeable by the authenticated client. **PKCE** (proof key) binds steps 1 and 4 so an

intercepted code is useless — mandatory for mobile/SPA (no client secret), recommended for all. `state` blocks CSRF; exact `redirect_uri` allowlisting blocks token redirection. Deprecated/avoid: implicit flow (tokens in URL fragment), password grant. Machine- to-machine: **client credentials** flow (no user). Scopes are coarse permissions (`repo:read`) — the resource server still does fine-grained authz.

Real Production Example

"Sign in with Google/GitHub/Apple" (OIDC), every third-party integration (Slack apps, GitHub Apps), and internal SSO (Okta) — one identity provider, authorization-code flow everywhere, tokens scoped per audience.

Interview Questions

1. Explain the authorization-code flow and *why* the code exists at all.
2. OAuth vs OIDC — which token proves identity, which grants API access?
3. Why is PKCE mandatory for mobile apps?

9.4 API Keys & Service-to-Service Auth

Core Concept

- **API keys:** long-lived opaque strings identifying a *calling application* (not a user). Right for server-to-server B2B APIs (Stripe: `sk_live_...`). Handling: show once at creation, store **hashed** (they're credentials), prefix for identification (`sk_live_` — enables secret scanning), scope + per-key rate limits, easy rotation with overlapping validity, and instant revocation. Weakness: bearer credential — anyone holding it is you; no user context, no expiry unless you add it.
- Stronger service-to-service options to name: **mTLS** (both sides present certs — the service mesh does this transparently; identity = certificate), **signed requests** (AWS SigV4: request signed with a secret, replay-resistant, key never on the wire), and short-lived platform identities (cloud IAM roles, SPIFFE/SVID) instead of static keys.
- Zero-trust framing: don't trust the network ("we're inside the VPC" is not authn); every hop authenticates and authorizes.

Interview Questions

1. Design the API-key system for a public developer API (issue, store, rotate, revoke, rate-limit).
2. API key vs OAuth client-credentials vs mTLS — when each?

9.5 Rate Limiting

Why Interviewers Ask This

It's both a security control (brute force, scraping, abuse) and a reliability control (protect capacity, fairness) — and "design a rate limiter" is a full interview question (Module 13.24). Expect algorithm + distributed-state discussion.

Core Concept & Internal Working

Algorithms, with the trade-offs interviewers want:

- **Fixed window:** counter per `key:window` (`user:123:12:05` → INCR, EXPIRE). Cheap; boundary burst flaw (100 at 12:04:59 + 100 at 12:05:01 = 200 in 2 s).
- **Sliding window log:** timestamps in a sorted set; exact but $O(\text{rate})$ memory.

- **Sliding window counter**: weighted blend of current+previous window — near-exact, cheap; the pragmatic default (Cloudflare uses this).
- **Token bucket**: capacity B , refill r/sec ; a request takes a token. Allows *bursts up to B* while enforcing average r — usually the right semantics for APIs (AWS/Stripe model). Two numbers to state: rate and burst.
- **Leaky bucket**: queue drained at constant rate — smooths output (traffic shaping), adds queueing delay.

Distributed enforcement: counters in **Redis** (atomic Lua: INCR+check+EXPIRE) — adds ~ 1 ms/request and makes Redis a critical dependency (decide **fail-open** (availability) vs **fail-closed** (protection) — say the choice). Common production compromise: **local token buckets per gateway node with async sync** — approximate global limit ($N \text{ nodes} \times \text{local slice}$), zero added latency; or Envoy/gateway-integrated global rate limit service.

Design details that score points: return **429 + Retry-After** and `X-RateLimit-Remaining` headers; layer limits (per-IP at edge for DDoS, per-user, per-API-key, per-endpoint cost-weighted, global per-service concurrency); shadow mode before enforcement; separate the *decision* (policy) from the *counting* (infra).

```
request → gateway → Lua @ Redis: tokens = min(B, tokens + r·Δt); tokens ≥ 1 ?
                        allow (tokens-1) → service          deny → 429 + Retry-After
```

Real Production Example

Stripe's published rate limiter suite: request-rate limiter (token bucket) + concurrency limiter + fleet-usage load shedders, all in Redis with fail-open. GitHub/Cloudflare document sliding-window counters at the edge.

Interview Questions

1. Token bucket vs sliding window — semantics and when each fits.
2. Make the limit *global* across 50 gateway nodes — options and their accuracy/latency trade-offs.
3. Redis is down — does your limiter fail open or closed, and why?

9.6 Encryption & TLS

Core Concept

Three states of data, three answers:

- **In transit — TLS everywhere** (external *and* internal: mTLS via mesh). TLS 1.3: asymmetric crypto (certificates, key exchange with forward secrecy) bootstraps a symmetric session key (AES-GCM/ChaCha20) — asymmetric authenticates, symmetric encrypts bulk (fast). Certificates chain to CAs; automate issuance/renewal (ACME/Let's Encrypt) — expired certs are a top self-inflicted outage.
- **At rest — envelope encryption**: each object/record encrypted with a **DEK** (data encryption key); DEKs encrypted by a **KEK** living in a **KMS/HSM**; ciphertext DEK stored alongside data. Rotation = re-encrypt small DEKs, not petabytes; access control + audit at the KMS; "crypto-shredding" (destroy key = destroy data) for deletion/GDPR.
- **In use / application-layer**: field-level encryption for the crown jewels (PANs, SSNs) so even DB admins/dumps can't read them; tokenization for card data (swap PAN for a token; only the vault maps back — shrinks PCI scope massively).

Non-negotiables: never roll your own crypto; hash passwords (Argon2/bcrypt), encrypt data — different tools; TLS termination point is a *decision* (edge for performance + re-encrypt internally, or end-to-end for zero-trust).

Interview Questions

1. Explain envelope encryption and why KMS doesn't encrypt your data directly. (throughput + blast radius + rotation economics)

2. Where does TLS terminate in your design and what runs in cleartext afterwards?
3. How does tokenization reduce PCI scope?

9.7 Secrets Management

Core Concept

Secrets (DB passwords, API keys, signing keys) must be: **out of code/images/env files** (the #1 breach vector is a leaked repo/laptop), centrally stored (Vault/AWS Secrets Manager/GCP Secret Manager), **access-controlled per workload identity** (the pod's platform identity — IAM role/K8s service account — fetches only its own secrets; solves "secret zero"), **audited, rotated** (best: short-lived *dynamic* credentials — Vault mints per-service DB users with TTLs, so leaks expire by themselves), and **revocable in an emergency** (documented break-glass + rotation runbook; assume any long-lived static secret is eventually leaked). Delivery: injected at deploy/runtime (sidecar/CSI/env-at-start), cached with TTL, never logged (log scrubbing — secrets in logs is a routine incident).

Interview Questions

1. How does a fresh pod get its DB password with no secret baked anywhere? (platform identity → KMS/Vault auth → scoped secret; "secret zero" via the platform)
2. A contractor's laptop leaked `.env` — walk the response and the design that makes it a non-event (short-lived dynamic creds).

Module 9 Cheat Sheet

AUTHN/AUTHZ	who vs what-may. Authn at edge/IdP once; authz per request at owning service (object-level! IDOR). RBAC→ABAC→ReBAC(Zanzibar).
SESSIONS	random ID in HttpOnly+Secure+SameSite cookie → Redis; instant revoke; store is a scaled dependency.
JWT	header.claims.signature; local verify (JWKS, RS256, kid, aud/iss, exp); readable not encrypted; REVOCATION: short access (5–15m) + rotated refresh (server-side, reuse detection); jti denylist if needed.
OAuth2/OIDC	delegated authorization; OIDC adds ID token (identity). Auth-code flow + PKCE + state + exact redirect URIs. access#id token. client-creds for M2M. No implicit/password grants.
API KEYS	app identity; hash at rest, prefix, scope, rotate, revoke. Stronger: mTLS (mesh), SigV4-style signing, short-lived platform identities. Zero trust: VPC ≠ authn.
RATE LIMIT	fixed window(burst flaw) < sliding counter(default) < token bucket (rate+burst, API standard) / leaky (shaping). Redis+Lua global vs local-approximate. 429+Retry-After. Layered: IP/user/key/endpoint. Declare fail-open vs fail-closed.
ENCRYPTION	transit: TLS1.3+mTLS, ACME automation. rest: envelope (DEK+KMS KEK), rotation, crypto-shred. fields: tokenize PANs (PCI scope). Argon2/bcrypt for passwords.
SECRETS	central vault, workload identity, dynamic short-lived creds, audit, rotation runbook, never in code/logs.

Top Interview Questions (Module 9)

1. Sessions vs JWT + revocation design. 2. Auth-code+PKCE flow and why the code exists. 3. Stolen-token containment. 4. Global rate limiter across 50 nodes.
2. Token bucket vs sliding window. 6. Envelope encryption + KMS rationale.
3. Service-to-service auth options (mTLS/signing/IAM). 8. Secret zero problem.
4. Zanzibar-style sharing authz. 10. Where TLS terminates and why.

Common Mistakes Recap

JWT with no revocation story • tokens in localStorage • "JWT is encrypted" • skipping aud/iss • implicit flow • plaintext/.env secrets • API keys stored unhashed • rate limiter with no fail-open/closed decision • authz only at the gateway (IDOR) • trusting the internal network • manual cert renewal.

Mock Interview Exercise

"Add the full security story to a marketplace API: browser + mobile apps, third-party developer API, internal microservices, card payments." Expected: OIDC login (auth-code+PKCE) → edge session cookie → short internal JWTs; refresh rotation + logout-everywhere; developer API keys (hashed, scoped, per-key token- bucket limits) or client-credentials OAuth; mesh mTLS internally; object-level authz in owning services; tokenized cards via PSP (PCI scope), envelope encryption at rest; Vault + workload identity + dynamic DB creds; layered rate limits + fail-open call; audit logging.

Module 10 — Performance Engineering

Performance questions in interviews come in two forms: "your system is slow — debug it" (a war-story test) and "how do you keep this design fast at scale" (a prevention test). This module arms you for both.

10.1 Bottleneck Analysis & Profiling

Why Interviewers Ask This

"Users report the app is slow" is a favorite senior scenario. They grade your *method*: do you measure before changing anything, and do you know the usual suspects in order of likelihood?

Core Concept — the method

1. **Define "slow" numerically**: which endpoint, which percentile, since when, for whom (all users? one region? one tenant?). A p99 regression with flat p50 points at tail causes (GC, lock contention, one bad shard); a uniform shift points at a code/deploy/dependency change.
2. **Follow the request path with data**: client → CDN → LB → service → dependencies. Distributed tracing (Module 12) shows which hop grew; without it, compare latency dashboards layer by layer.
3. **Utilization–Saturation–Errors (USE)** on the suspect host: CPU, memory, disk I/O, network — is a resource saturated (run queue, iowait, swap)? Queueing theory: latency explodes as any resource approaches 100%.
4. **Profile before optimizing**: CPU flame graphs (perf, async-profiler, pprof) show where time is actually spent — it is almost never where intuition says. Continuous profilers (Parca, Pyroscope, Datadog) let you diff "yesterday vs today". For latency (not CPU) problems, use wall-clock/off-CPU profiling — a thread waiting on a lock or socket is invisible to CPU profiles.
5. **The base-rate list** (most production slowness, in rough order): a slow **database query** (new unindexed access pattern, plan flip, table growth) → **N+1 patterns** → **connection/thread pool exhaustion** → cache hit-ratio drop → GC pauses → a slow downstream API → resource saturation from a noisy neighbor or organic growth.

Amdahl again: optimize the biggest span in the trace, not the most interesting one.

Interview Questions

1. p99 doubled at 9am; p50 flat; no deploys. Your first five actions?
 2. Why can CPU profiles mislead for latency problems? (off-CPU time invisible)
 3. What's the USE method?
-

10.2 Slow Queries

Core Concept & Playbook

The single most common production performance incident. The loop:

- **Find them**: `pg_stat_statements` / slow query log (log everything > 100 ms), APM traces, `EXPLAIN ANALYZE` the top offenders.
- **Read the plan**: seq scan under a selective WHERE = missing index; wrong composite order (Module 4.4); huge `Rows Removed by Filter`; sort/hash spilling to disk (`work_mem`); nested-loop joins on unindexed FKs; plan flips after data growth or stale statistics (`ANALYZE`).

- **Fix ladder:** add/repair the index → rewrite the query (avoid functions on indexed columns, `SELECT` only needed columns, EXISTS over IN for big sets, keyset pagination instead of OFFSET) → denormalize/precompute (materialized views, counter columns) → cache the result → move the workload (replica for analytics, columnar warehouse for aggregations — OLAP queries do not belong on the OLTP primary).
- **Contain them:** `statement_timeout` per role/endpoint so one runaway query can't hold locks and drain the pool; separate replica/warehouse for ad-hoc internal queries (a data analyst's cross join should never be able to take down checkout).
- Watch **lock waits** disguised as slow queries (`pg_locks`, blocked-by chains): a long transaction holding a row lock makes everything behind it "slow".

Interview Questions

1. A query got 100× slower overnight with no code change — hypotheses? (growth crossed a plan threshold, stats stale → plan flip, index bloat, lock contention, autovacuum behind)
2. Why is OFFSET 500000 pagination slow and what replaces it? (scans and discards; keyset/seek: `WHERE id < last_seen ORDER BY id LIMIT 20`)

10.3 Connection Pool Exhaustion

(Deep treatment in Module 4.6 — here, the incident pattern interviewers ask about.)

```
trigger: one slow query/dependency (2 s instead of 5 ms)
→ connections held 400× longer → pool (20) fully busy at 10 QPS instead of 4000
→ new requests queue at acquire → app threads block → upstream times out
→ RETRIES arrive (more load) → total stall. DB often looks IDLE (it's waiting, not working).
```

Diagnosis signature: latency spike + pool "active = max" + DB CPU *low* + acquire timeouts in logs. Fixes in order: kill/timeout the slow query (`statement_timeout`), fail fast at acquire (small timeout), bulkhead pools per workload, circuit-break the offending dependency, then restore. Prevention: Little's-Law sizing, per- endpoint timeouts shorter than the client's, PgBouncer at fleet scale, and load tests that include the slow-dependency scenario.

Interview Question

"The DB looks healthy but the app is down and every thread is 'waiting for connection'." — narrate the cascade above and the runbook.

10.4 N+1 Queries

Core Concept

The ORM classic: fetch N parents, then lazily query children once per parent — 1 + N round trips (100 posts = 101 queries; at 1 ms each that's the whole latency budget). Each query is fast; the *count* kills you — which is why it hides from slow-query logs and only shows in traces (a comb of tiny identical spans) or query counters per request.

Fixes:

- **Batch:** `WHERE post_id IN (...)` / ORM eager loading (`includes`, `select_related` / `prefetch_related`, JPA fetch joins).
- **JOIN** parent+child in one query where shapes allow.

- **Application-level batching — DataLoader pattern** (essential vocabulary for GraphQL designs): collect all child-IDs requested in one tick, issue one batched query, distribute results. GraphQL's resolver-per-field model makes N+1 the *default* failure mode; DataLoader is the standard answer.
- The same disease exists across services: a for-loop of RPC calls = distributed N+1 → batch APIs (`GET /users?ids=...`) — every internal API should expose batch endpoints.

Guardrails: per-request query counters in dev/CI (fail tests on query-count regressions), ORM lazy-loading warnings in staging.

Interview Questions

1. Feed page makes 400 queries — find it, fix it, prevent regression.
2. Why is GraphQL especially N+1-prone and what's the standard fix?

10.5 Compression

Core Concept

Trade CPU for bytes — almost always a win over WAN/mobile, often a wash on LAN.

- **HTTP responses:** gzip (universal) vs **Brotli** (~15–20% smaller for text, slower to compress — use high-level Brotli for *static* assets precompressed at build time, fast levels or gzip/dynamic for API JSON) vs **zstd** (best speed/ratio family, growing support). Compress text (HTML/JSON/JS); never recompress JPEG/MP4 (already compressed — wasted CPU).
- **Internal:** gRPC/protobuf is a form of schema compression (binary vs JSON ≈ 3–10× smaller); Kafka producer compression (lz4/zstd per batch) is standard and multiplies effective broker throughput; column stores (Parquet) get 10×+ from columnar encodings (RLE, dictionary, delta).
- **Trade-offs to name:** CPU cost at high QPS (measure — compression can become *the* CPU consumer at an edge tier), latency of buffering to compress, and small responses (< ~1 KB) not worth headers/CPU. BREACH/CRIME caveat: compressing secrets alongside attacker-controlled input in TLS responses leaks data — don't compress pages mixing both.
- Media is its own pipeline: images → WebP/AVIF renditions, video → H.264/265/AV1 adaptive bitrate ladders (Module 13 YouTube/Netflix) — "compression" there is transcoding, done once offline, not per request.

Interview Question

"Mobile users on slow networks: what do you compress, where, and what do you skip?"

10.6 Pagination

Core Concept

Never return unbounded lists — pagination is memory protection, latency protection, and DB protection at once.

- **Offset/limit** (`LIMIT 20 OFFSET 10000`): simple, jumpable to page N, but the DB *scans and discards* offset rows (page 500 costs 500× page 1) and pages **drift** when rows are inserted/deleted between requests (skipped/duplicated items).
- **Cursor/keyset (seek):** `WHERE (created_at, id) < (:last_created, :last_id) ORDER BY created_at DESC, id DESC LIMIT 20` — constant cost per page (pure index seek), stable under inserts; the cursor is an opaque token encoding the last-seen sort key. Cost: no "jump to page 37", requires a unique, indexed, stable sort key (tie-break with id).

- **Rules:** infinite scroll/feeds/APIs → cursor (Slack, Stripe, Twitter, GitHub APIs all use cursors); small admin tables where humans want page numbers → offset is fine; always enforce a **max page size**; return `next_cursor + has_more`; make cursors opaque (base64 the key material) so clients can't fabricate them.
- Deep-pagination at scale (page 10,000 of search results): nobody legitimate does it — cap it (Google stops at ~page 40; Elasticsearch caps `from+size` at 10k and offers `search_after` for the rest).

Interview Question

"Design pagination for a feed with new items arriving constantly — why does OFFSET break, exactly?"

10.7 Batch Processing & Streaming

Core Concept

Two execution models for work that isn't request/response:

- **Batch:** bounded dataset, scheduled, high throughput, latency = schedule + runtime (minutes–hours). MapReduce lineage → Spark. Right for: billing runs, ML training, backfills, reports, dedup/reconciliation. Design points: **idempotent + re-runnable** jobs (reruns after failure must be safe — write to a staging location and atomically swap), checkpointing for long jobs, and isolation from OLTP (run against replicas/warehouse/lake).
- **Streaming:** unbounded events processed continuously, latency in seconds. Flink / Kafka Streams / Spark Structured Streaming. Right for: fraud scoring, live dashboards/metrics, feed fan-out, CDC pipelines, sessionization. Design vocabulary interviewers probe: **event time vs processing time**, **windows** (tumbling/sliding/session), **watermarks** (how long to wait for late events), **stateful operators** with checkpointed state (RocksDB + changelogs — this is what makes exactly-once processing possible inside the pipeline).
- **The batching principle inside services** (micro-batching): amortize per-operation overhead everywhere — batched DB inserts (1000-row multi-insert ≈ 50× faster than 1000 singles), batched Kafka produces, batched cache MGETs, coalesced writes for hot counters. Throughput up, small latency cost (Module 1.1).
- **Lambda vs Kappa architecture** (one-liner): lambda = batch + speed layers reconciled (two codebases — operational pain); kappa = streaming-only with replayable log as source of truth (rebuild by replaying Kafka). Modern default leans kappa + occasional backfills.

```
events → Kafka → Flink (windows, state, watermarks) → serving store (live)
                  ↘ object storage (raw) → Spark batch (backfill/repair/training)
```

Interview Questions

1. Fraud detection needs a decision in < 2 s — batch or streaming, and what state does the pipeline keep? (streaming; per-card/session aggregates in checkpointed state)
2. Your nightly batch failed at 80% — what design choices make the rerun safe?
3. Event time vs processing time — why does the difference matter for "orders per minute" dashboards?

Module 10 Cheat Sheet

METHOD	Quantify (endpoint, percentile, cohort) → trace the path → USE per host → profile (flame graphs; off-CPU for latency) → fix biggest span.
SLOW QUERY	Base rates: slow query > N+1 > pool exhaustion > cache drop > GC > dep. <code>pg_stat_statements</code> → EXPLAIN ANALYZE → index/rewrite/precompute/cache/relocate. <code>statement_timeout</code> . OLAP off the OLTP box. Lock waits lie.
POOLS	slow dependency → held connections → acquire queue → stall while DB idles. Fail-fast acquire, statement timeouts, bulkheads, breakers.

N+1	1+N round trips; invisible in slow logs, visible in traces. Eager load / IN-batch / JOIN / DataLoader (GraphQL). Batch RPC endpoints. Query-count guardrails in CI.
COMPRESSION	Brotli static (precompressed), gzip/zstd dynamic, never re-compress media. gRPC/protobuf, Kafka lz4/zstd, Parquet columnar. CPU vs bytes; skip < 1 KB.
PAGINATION	OFFSET scans+drifts; keyset/cursor = constant cost, stable, no jumps. Opaque cursors, max page size, cap deep pages (search_after).
BATCH	bounded, scheduled, idempotent + re-runnable, staging+atomic swap, off the OLTP path.
STREAMING	unbounded, seconds latency; event-time windows + watermarks + checkpointed state (Flink/Kafka Streams). Kappa: replay the log.
MICRO-BATCH	amortize everything: multi-row inserts, MGET, producer batches.

Top Interview Questions (Module 10)

1. "App is slow" — full triage method.
2. Query 100× slower overnight.
3. Pool- exhaustion cascade with an idle DB.
4. N+1 in GraphQL + DataLoader.
5. OFFSET vs keyset internals.
6. What to compress for mobile.
7. Batch vs streaming for fraud/ billing/dashboards.
8. Idempotent rerunnable batch design.
9. Watermarks and late events.
10. Where batching adds latency and why it's usually worth it.

Common Mistakes Recap

Optimizing without profiling • average-latency thinking • OLAP on the OLTP primary • no statement timeouts • ORMs lazy-loading in loops • unbounded list endpoints • OFFSET on infinite scroll • compressing JPEGs, skipping JSON • non-idempotent batch jobs • ignoring event-time vs processing-time skew.

Mock Interview Exercise

"An e-commerce 'order history' page p99 went from 300 ms to 4 s over two months. Traffic doubled; the table grew 5×; there were many small deploys." Walk the triage: trace shows DB span dominant → EXPLAIN reveals OFFSET pagination + a plan flip on the grown table + an N+1 for item thumbnails. Fix: keyset pagination, composite index (`user_id, created_at DESC`), batched thumbnail fetch, statement timeout, and a CI query-count guard. Then state prevention: pagination standards, index review in migrations, continuous profiling.

Module 11 — Distributed Systems Toolkit

The named techniques interviewers expect at senior/staff level. Each is a small idea with outsized leverage; each appears by name in follow-up questions ("how would you avoid rehashing everything?" — they're fishing for consistent hashing).

11.1 Consistent Hashing

Why Interviewers Ask This

It's the standard answer to "you added a cache/DB node — now what?" and the core of Dynamo, Cassandra, and most distributed caches. Expect to explain the ring *and* virtual nodes.

Core Concept & Internal Working

Problem: $\text{hash}(\text{key}) \bmod N$ remaps nearly every key when N changes → a scaling event becomes a fleet-wide cold cache / data migration.

Consistent hashing: hash servers and keys onto a circular space ($0..2^{64}$); each key belongs to the first server clockwise from it. Adding/removing a server only moves the keys in its arc — **$\sim K/N$ keys move instead of $\sim \text{all}$.**

```
0 —————> 2^64 (wraps)
—[S3]—k1—[S1]—k2—k3—[S2]—k4—[S3...]—
k1→S1, k2,k3→S2, k4→S3
add S4 between S1 and S2: only k2,k3's arc splits; k1,k4 untouched ✓
```

Virtual nodes fix the two remaining problems (uneven arcs; a leaving node dumps its whole load on one neighbor): each physical server gets 100–1000 tokens on the ring, so load evens out statistically, departures spread across many neighbors, and heterogeneous hardware gets proportional token counts. Replication: store each key on the next R distinct physical successors.

Variants worth naming: **jump consistent hash** (no ring state, perfect balance, but only append/remove-at-end), **rendezvous (HRW) hashing** (highest-random-weight; simple, great for small N), Maglev hashing (LB lookup tables).

Real Production Example

DynamoDB/Cassandra/Riak partitioning; Memcached client libraries (ketama); Envoy/HAProxy ring-hash LB for session/cache affinity; Discord's ring for routing guilds to nodes.

Interview Questions

1. Why does mod- N break, quantitatively? (changing $N=10 \rightarrow 11$ remaps $\sim 10/11$ of keys)
 2. What exactly do virtual nodes buy? (balance, smooth rebalancing, weighting)
 3. How do you get replication out of the ring? (next R distinct successors)
-

11.2 Bloom Filters

Core Concept & Internal Working

A probabilistic set membership structure: bit array of m bits + k hash functions. Insert: set k bits. Query: if any bit is 0 → **definitely not present**; if all 1 → **probably present** (false-positive rate tunable: ~1% at ~10 bits/element; no false negatives; no deletes in the basic form — counting Bloom filters allow them at 4× space).

The design move it enables: **skip expensive lookups for things that don't exist**.

- LSM storage engines (RocksDB, Cassandra, HBase): a Bloom filter per SSTable — a read checks the filter before touching disk; misses skip whole files. This is *the* answer to "how do LSM reads stay fast".
- Cache-penetration defense: filter of "keys that exist" in front of DB, so attackers querying random missing keys don't hammer it.
- CDN "cache on second hit" (Akamai: filter of seen-once URLs keeps one-hit wonders out of cache), Chrome's old malicious-URL check, distributed joins, dedup in crawlers ("have I seen this URL?").

Sizing soundbite: 1B items at 1% FP $\approx$ 1.2 GB — a billion-key existence check in RAM. Cousins to name-drop: **Cuckoo filters** (deletes, better space at low FP), **HyperLogLog** (cardinality counting, 12 KB for billions $\pm$ 2% — "count distinct daily users"), **Count-Min Sketch** (approximate frequencies — heavy-hitter/hot-key detection).

Interview Questions

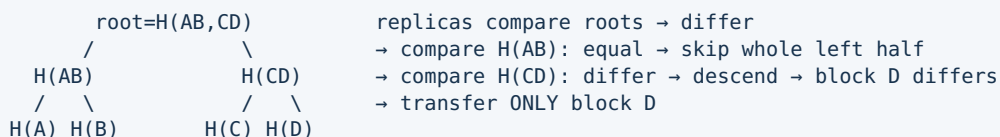
1. Where does a Bloom filter sit in an LSM read path and what does it save?
2. A false positive occurs — what happens in each of your use cases? (harmless extra lookup; must be tolerable by design)
3. Count unique visitors across 1B events in near-zero memory? (HyperLogLog)

11.3 Merkle Trees (Overview)

Core Concept

A tree of hashes: leaves = hashes of data blocks; parents = hash of children; the **root hash summarizes the entire dataset**. Compare two replicas: roots equal $\Rightarrow$ identical (one comparison); differ $\Rightarrow$ descend only into differing subtrees $\Rightarrow$ locate divergent blocks in **$O(\log n)$** comparisons instead of shipping/scanning everything.

Where it earns its name in interviews: **anti-entropy repair** in Dynamo-style stores (Cassandra `nodetool repair` builds Merkle trees per range and syncs only differing partitions), **Git** (commits point to hash trees — cheap diff/integrity/ dedup), **blockchains** (transaction inclusion proofs in $O(\log n)$), certificate transparency logs, and file-sync/dedup systems (content-addressed chunks).



Interview Question

"Two 10 TB replicas may have diverged after a partition — how do you find and fix the differences without comparing 10 TB?"

11.4 Vector Clocks (Overview)

Core Concept

Logical clocks that capture **causality** without synchronized time. Each node keeps a vector of counters (one slot per node); increment your slot on local events, merge (element-wise max) on receive.

- $VC(A) < VC(B)$ element-wise $\Rightarrow$ A **happened-before** B (B knew about A — safe to overwrite A).
- Neither $\leq$ the other $\Rightarrow$ **concurrent** $\Rightarrow$ a true conflict that timestamps would have silently destroyed (last-writer-wins picks an arbitrary winner under clock skew).

That's the interview point: vector clocks *detect* concurrent writes so the system can keep **siblings** and resolve them (Dynamo returned conflicting cart versions to the app, which merged by union; Riak exposed siblings). Costs: $O(\text{nodes})$ metadata per key, pruning complexity — which is why many systems (Cassandra) accept LWW instead, and modern designs often prefer **CRDTs** (data types that merge automatically: G-counters, OR-sets — collaborative editing, distributed counters). Name **Lamport clocks** as the scalar little sibling: total order, but can't distinguish concurrency.

Interview Question

"Two clients update the same key on opposite sides of a partition. How does the system even *know* it's a conflict, and what are the resolution options?" (vector clocks/siblings $\rightarrow$ app merge, CRDT, or LWW with data-loss acknowledged)

11.5 Leader Election

Core Concept & Internal Working

Many workloads need exactly-one-active: DB primaries, job schedulers, partition owners, cron singletons. Election = choosing it safely; the hard part is the old leader that doesn't know it lost (Module 8.5).

Implementations, in the order you should offer them:

1. **Lease-based on a coordination service** (the practical default): candidates race to create an ephemeral node / lease key in ZooKeeper/etcd/Consul (`create /leader` ephemeral; or etcd lease + campaign). Holder = leader; must keep renewing (session heartbeat); crash/partition $\Rightarrow$ lease expires $\Rightarrow$ others elect. Kubernetes controllers do exactly this (Lease objects). **Always pair with fencing tokens** (the monotonically increasing election/term number) checked by downstream systems, because a paused ex-leader can wake and act.
2. **Consensus-native** (Raft, Module 8.4): election is built into the replication protocol — for systems that *are* the replicated state (databases, Kafka controllers).
3. Classical algorithms (Bully, ring) — name-check only; production uses 1 or 2.

```
candidates —try acquire—> etcd lease "svc/leader" (TTL 10s, term=42)
winner: leads; renews every 3s; stamps all actions with term 42
GC-paused winner: lease lapses → new leader term 43 → storage rejects term-42 writes ✓
```

Interview Questions

1. Elect a singleton job scheduler across 5 instances — design it and kill it with a GC pause; what saves correctness? (fencing)
 2. Why not just use a Redis `SETNX` lock as the "election"? (fine informally, but see Redlock caveats — no fencing, replication async; use a real coordination service for correctness-critical roles)
-

11.6 Gossip Protocol (Overview)

Core Concept

Epidemic information spread: every interval, each node exchanges state with a few random peers; information reaches all N nodes in $O(\log N)$ rounds with no coordinator, no SPOF, and per-node load independent of cluster size.

What it carries in practice: **membership and failure detection** (who's in the cluster, who seems down — Cassandra, Consul/Serf via SWIM, DynamoDB internals), cluster metadata (schema versions, token ownership), and lightweight aggregates. SWIM's trick: instead of timeout-only suspicion, ping through *indirect* probes (ask k others to ping the suspect) → fewer false positives from your own bad link; "suspect → confirmed" states with incarnation numbers to refute false rumors.

Trade-offs: eventual (seconds of propagation delay — membership views briefly disagree), bandwidth tuning, and rumors need versioning to die. Contrast with consensus: gossip disseminates *facts loosely* (scales to thousands of nodes); consensus decides *values strictly* (small clusters). Big systems use both: gossip for membership, Raft for the metadata core.

Interview Question

"How does a 1,000-node Cassandra cluster know a node died without a central monitor, and why might two nodes briefly disagree about it?"

11.7 Distributed Locks

Why Interviewers Ask This

"Prevent double-processing / double-booking" appears in most designs, a naive Redis lock is subtly broken, and the Kleppmann–Redlock debate is assumed knowledge at staff level.

Core Concept & Internal Working

Single-Redis lock done *correctly-for-efficiency*:

```
acquire: SET lock:{res} {random_token} NX PX 30000    (atomic, owner-tagged, TTL)
release: Lua - GET == my token ? DEL : no-op          (never delete another's lock)
extend:  watchdog renews TTL while still working
```

Why TTL: crashed holders must not deadlock the system. Why token: your lock may have *expired* mid-work and been granted to someone else — blind DEL releases *their* lock.

The fundamental flaw (Kleppmann): a GC pause / network delay can suspend the holder past its TTL; it resumes and writes concurrently with the new holder. **The lock alone cannot guarantee mutual exclusion end-to-end** — the protected resource must check **fencing tokens** (monotonic lock generation number; storage rejects writes with a stale token). **Redlock** (quorum of 5 independent Redis nodes) raises availability of the *lock service* but still doesn't produce fencing tokens or fix the pause problem — know the critique.

Decision framework to recite:

- **Efficiency lock** (avoid duplicate work; a rare double-run is tolerable — cache refresh, cron): single Redis SET NX is perfect.
- **Correctness lock** (double-run = corruption/money): ZooKeeper/etcd (sessions + monotonic zxid/revision = built-in fencing) **plus token-checking at the resource** — or better, remove the lock: route all ops for a resource to **one partition owner** (Kafka-keyed serialization), or use **DB-native protection** (unique

constraints, `SELECT ... FOR UPDATE`, optimistic version checks) — the database is often the best lock manager you already have.

Interview Questions

1. Implement a Redis lock; now break it with a 40 s GC pause; now fix it. (token + fencing at storage)
2. Efficiency vs correctness locks — classify: cache warmer, seat booking, nightly report, payment capture.
3. When is "no lock" the right answer? (single-writer partition ownership; DB constraints)

11.8 Clock Synchronization

Core Concept

Every machine's clock drifts (ppm-level $\Rightarrow$ ms/day); NTP syncs to ~1–10 ms typical (worse under load/network jitter) and can **step time backwards**. Consequences you must design around: cross-machine timestamp ordering is unreliable (LWW conflict resolution silently loses newer writes under skew — the Cassandra caveat), timeouts/leases measured across machines are approximate, TTL/cert validation breaks with bad clocks, and monotonic operations (rate limiting windows, Snowflake ID generation) must use **monotonic clocks** locally (never wall-clock deltas) and handle backward steps (Snowflake generators stall or error if time regresses).

The famous fix: **Google TrueTime** (GPS + atomic clocks per datacenter) exposes a **bounded uncertainty interval** [earliest, latest]; Spanner assigns commit timestamps and **waits out the uncertainty** (~few ms) before acknowledging — buying *external consistency* (global serializability with real-time order) at the cost of that wait. AWS TimeSync/precision hardware clocks now offer microsecond bounds; CockroachDB approximates with NTP + max-offset assumptions (HLC — hybrid logical clocks combine physical time with Lamport counters: name-drop worthy).

Rule of thumb to state: **use physical clocks for humans and metrics; use logical clocks (Lamport/vector/sequence numbers) or single-writer ordering for correctness.**

Interview Questions

1. Why is last-writer-wins dangerous, exactly? (skew $\Rightarrow$ "later" write can carry an *earlier* timestamp $\Rightarrow$ silent loss)
2. What does TrueTime provide and what does Spanner do with it?
3. Your rate limiter misbehaved when NTP stepped the clock back 2 s — why, and the fix? (wall clock used for interval math $\rightarrow$ monotonic clock)

11.9 Eventual Consistency (Revisited, Mechanically)

Module 1.4 placed it on the spectrum; here's the machinery interviewers probe:

- **Convergence mechanisms:** read repair (fix stale replicas on read), hinted handoff (hold writes for down peers), anti-entropy with Merkle trees (background full sync), gossip dissemination. Together: "if writes stop, replicas converge."
- **Conflict handling** (concurrent writes will happen): LWW (simple, lossy), vector clocks + siblings (app merges), **CRDTs** (mathematically mergeable types: counters, sets, maps — Figma/collaborative apps, Riak, Redis CRDTs in Active-Active).
- **What you promise the user:** session guarantees (read-your-writes, monotonic reads — Module 4.2 routing tricks), bounded staleness ("search results $\leq$ 30 s stale"), and explicit UX for pending states.
- **What you must add operationally:** staleness/lag metrics per replica pipeline, reconciliation jobs (counters re-computed from source), and idempotent consumers everywhere (Module 5.5) because redelivery is part of the deal.

Interview Question

"Walk me through everything that happens after a partition heals in a Dynamo-style store." (hinted handoffs replay → read repair on touched keys → Merkle anti-entropy for the cold rest → conflicts surfaced via vector clocks → app/CRDT merge)

Module 11 Cheat Sheet

CONSISTENT HASH	ring $0..2^{64}$, key→next server clockwise; add/remove moves $\sim K/N$. Virtual nodes (100–1000/server): balance + smooth rebalance + weighting. Replicas = next R distinct successors.
BLOOM FILTER	k hashes → bits; "no" is certain, "yes" is probabilistic ($\sim 1\%$ @ 10 bits/elem). LSM SSTable skip, cache-penetration shield, dedup. Cousins: Cuckoo (deletes), HLL (count distinct), CMS (frequencies).
MERKLE TREE	hash tree; compare roots, descend diffs: $O(\log n)$ divergence location. Cassandra repair, Git, cert transparency.
VECTOR CLOCK	per-node counters; detects CONCURRENT vs ordered (LWW can't). Siblings → app merge; CRDTs merge automatically; Lamport = scalar.
LEADER ELECTION	etcd/ZK lease + ephemeral + renewal; or Raft-native. ALWAYS fencing tokens (terms) checked downstream. K8s Lease objects.
GOSSIP	random peer exchange, $O(\log N)$ spread, no SPOF; SWIM indirect probes + incarnation numbers. Membership/failure detection at 1000s of nodes. Loose facts vs consensus's strict values.
DIST. LOCKS	SET NX PX + owner token + Lua release + watchdog. GC pause breaks mutual exclusion ⇒ fencing at the resource. Redlock ≠ fix. Efficiency lock (Redis) vs correctness (ZK/etcd + fencing) vs NO lock (partition ownership, DB constraints) – prefer the last.
CLOCKS	NTP ~ms + backward steps; monotonic clocks for intervals; logical clocks/single-writer for ordering; TrueTime = bounded uncertainty + commit-wait (Spanner external consistency); HLC hybrid.
EVENTUAL	read repair + hinted handoff + Merkle anti-entropy + gossip; conflicts: LWW / siblings / CRDT; session guarantees for UX; reconciliation jobs + lag metrics operationally.

Top Interview Questions (Module 11)

1. mod-N vs ring, with numbers. 2. Virtual nodes' three benefits. 3. Bloom filter in the LSM read path. 4. Sync two diverged 10 TB replicas cheaply. 5. Detect (not just resolve) write conflicts. 6. Leader election + GC pause + fencing. 7. Why Redlock is contested. 8. SWIM failure detection. 9. LWW under clock skew.
2. TrueTime/commit-wait. 11. Post-partition healing sequence in Dynamo-style stores. 12. When to prefer partition ownership over any lock.

Common Mistakes Recap

mod-N sharding • rings without virtual nodes • Bloom filters where false positives aren't tolerable • trusting cross-machine timestamps • locks without TTL, owner tokens, or fencing • treating Redlock as correctness-grade • election without term checks downstream • forgetting reconciliation/lag metrics in eventually-consistent designs.

Mock Interview Exercise

"Design the coordination layer for a distributed cron service: 10k scheduled jobs, each must run exactly once per schedule across a 50-node worker fleet, workers crash freely." Expected: jobs partitioned across workers via consistent hashing (vnodes) → each partition has a lease-based owner (etcd) with fencing tokens → job-run records with unique (job_id, scheduled_time) constraint as the final idempotency backstop (DB as the true arbiter) → gossip or etcd watch for membership → missed-run detection via a sweeper. Then defend "exactly once" honestly: at-least-once triggering + idempotent execution.

Module 12 — Observability

"How do you know it's working?" is the closing question of most interviews, and a concrete answer — named metrics, alert policy, debug workflow — reliably earns the operational-maturity checkmark. This module also covers how engineers actually debug production.

12.1 The Three Pillars: Logs, Metrics, Traces

Why Interviewers Ask This

They want you to know what each signal is *for*, what each costs, and how they compose into a debugging workflow — not just recite the triad.

Core Concept

Signal	Answers	Shape	Cost profile
Metrics	"is something wrong? how much? trend?"	numeric time series, pre-aggregated	cheap to store/query; cardinality is the danger
Logs	"what exactly happened in this case?"	per-event records	expensive at volume; sampling/retention tiers
Traces	"where in the call graph did it happen?"	request tree with timed spans	sampled; the glue across services

Logging (production-grade): structured (JSON key-value, not printf prose) so it's queryable; **correlation/trace IDs on every line** (the single highest-value logging practice — without it, microservice logs are confetti); levels used honestly (ERROR pages someone); no secrets/PII (scrubbing); centralized (stdout → collector → Elasticsearch/Loki/cloud) with retention tiers (hot 7–30 d, archive to object storage); sample high-volume INFO, never sample ERROR. Anti-pattern to name: logs as metrics ("count ERROR lines to alert") — brittle and expensive; emit a counter instead.

Metrics: counters (requests_total), gauges (queue_depth), **histograms** (latency distributions — percentiles come from these). Two framing standards to cite: **RED** per service (Rate, Errors, Duration) and **USE** per resource (Utilization, Saturation, Errors); Google's four golden signals = latency, traffic, errors, saturation. **Cardinality discipline:** labels multiply series (endpoint × status × region fine; user_id in a label = series explosion that takes down the metrics system itself — the classic observability self-own).

Tracing: a request gets a **trace ID** at the edge; each hop creates **spans** (operation, start, duration, tags, parent) and **propagates context** in headers (W3C traceparent); the backend (Jaeger/Tempo/Zipkin/Datadog) assembles the tree:

```
trace 8f3a... POST /checkout                total 1.9s
├─ gateway (12ms)
├─ order-svc (1.85s)
│   └─ inventory-svc GET /stock (35ms)
│       └─ payment-svc POST /charge (1.72s) ← there's your problem
│           └─ psp-external (1.70s)         ← and its cause
│               └─ db INSERT order (18ms)
└─ notification enqueue (3ms)
```

Sampling is mandatory at scale: head-based (decide at start, e.g. 1%) vs **tail-based** (decide after completion — keep all errors and slow traces, sample the boring; more infra, far better signal).

Real Production Example

Google Dapper (2010) birthed modern tracing; Uber built Jaeger; Netflix Atlas handles ~billions of metrics series with aggressive cardinality control. Every serious postmortem you read follows the metrics → traces → logs funnel below.

Interview Questions

1. Which signal do you reach for first when p99 spikes, and in what order do you use the rest?
 2. Why is `user_id` fine in logs but catastrophic as a metric label?
 3. Head vs tail sampling — what do you lose with 1% head sampling? (most errors and rare slow paths never captured)
-

12.2 Monitoring & Alerting

Why Interviewers Ask This

Bad alerting causes both missed outages and burned-out on-call rotations; your alert philosophy reveals whether you've carried a pager.

Core Concept

- **Symptom-based paging, cause-based dashboards:** page on what users feel (SLO burn: error rate, latency, availability — Module 1.6); CPU/disk/replication-lag alerts are tickets or dashboard context, not 3am pages. Every page must be: actionable, urgent, and novel (else it's noise training people to ignore pages).
- **Burn-rate alerting** (the modern standard): page when the error budget is being consumed at e.g. 14× rate over 1 h (fast burn) or 6× over 6 h (slow burn) — catches both cliff-falls and slow bleeds without flapping on blips.
- Alert hygiene: multi-window confirmation, hysteresis, alert on *absence* of data too (a dead exporter looks like perfect health), runbook links in every alert, weekly review of pages (each page → fix, tune, or delete).
- **Synthetic monitoring / probers:** scripted user journeys from outside your network — catches DNS/TLS/CDN/login failures your internal metrics can't see. **Dashboards:** per-service standard layout (RED top, resources below, dependencies' health beside) so responders navigate by muscle memory.

Interview Questions

1. Design the alerts for a payments API with a 99.95% SLO. (burn-rate pages, latency-threshold SLI, synthetic probe on the full payment journey, DLQ-depth ticket)
 2. Your on-call gets 40 pages/week — triage the alert catalog.
 3. Why symptom-based paging? What's the failure mode of paging on CPU?
-

12.3 Prometheus & Grafana (and the metrics pipeline)

Core Concept & Internal Working

Prometheus: **pull-based** scraper — services expose `/metrics` (text exposition), Prometheus scrapes every 15–60 s, stores locally in a TSDB, evaluates **PromQL** alert rules, fires to **Alertmanager** (dedup, grouping, silences, routing to PagerDuty/Slack). Service discovery (K8s) auto-finds scrape targets — pull means the monitoring system *knows* when a target vanishes (vs push, where silence is ambiguous); push gateway exists for short-lived jobs.

PromQL fluency (one line each is enough in interviews):

<code>rate(http_requests_total{status=~"5.."}[5m])</code>	error rate
<code>histogram_quantile(0.99, rate(latency_bucket[5m]))</code>	p99 from histogram
<code>sum by (endpoint) (rate(http_requests_total[5m]))</code>	traffic per endpoint

Scaling story (expected at senior level): a single Prometheus is vertical-only and local-disk — federate or use **Thanos/Cortex/Mimir** (global query over many Prometheis + object-storage long-term retention + HA dedup). Histograms: buckets are pre-defined (native histograms improve this); percentiles are per-scrape-target — aggregate with care (you can't average percentiles; aggregate the buckets).

Grafana: the visualization/alerting layer over Prometheus (and Loki logs + Tempo traces — the open-source LGTM stack), with variables/templating for per-service dashboard reuse, and annotations (deploy markers — the single most useful debugging overlay: 80% of incidents correlate with a deploy).

Interview Questions

1. Pull vs push trade-offs? (target liveness knowledge, scrape control, firewall/ephemeral-job friction)
2. How do you get a *global* p99 across 200 pods? (aggregate histogram buckets, then quantile — never average per-pod p99s)
3. Prometheus retention is 15 days local — long-term + HA story? (Thanos/Mimir + object storage)

12.4 OpenTelemetry (High Level)

Core Concept

The vendor-neutral standard unifying all three signals: one **API + SDK** per language for producing metrics/logs/traces, **automatic instrumentation** for common frameworks (HTTP/gRPC/DB clients — trace context propagation for free), the **OTLP** wire protocol, and the **OTel Collector** — a pipeline component that receives, processes (batch, scrub PII, tail-sample, add metadata), and exports telemetry to any backend (Prometheus, Jaeger, Datadog, vendor X).

The architectural argument interviewers like: instrument **once** with the standard, choose/replace backends freely (no vendor lock-in at the code layer), and put governance (sampling, scrubbing, routing) in the collector instead of every service.

```

services (OTel SDK, auto-instr) → OTLP → OTel Collector → Prometheus/Mimir
                                   (batch, sample,   ↳ Tempo/Jaeger
                                   scrub, route)     ↳ Loki / vendor
  
```

Interview Question

"Your org runs three observability vendors across teams — how do you rationalize telemetry?" (OTel SDKs + collectors as the abstraction; backends become swappable)

12.5 How Engineers Actually Debug Production Issues

Why Interviewers Ask This

This is the war-story filter. The expected shape is a disciplined funnel, not tool name-dropping.

The workflow

- | | |
|-----------|--|
| 1. DETECT | page fires (SLO burn) — or worse, users/support report first |
| 2. SCOPE | dashboards: which service, endpoint, region, percentile, cohort? |

- blast radius → severity → do we need incident command?
3. CORRELATE what changed? deploy markers, feature flags, config pushes, dependency status, traffic anomalies. ~most incidents are changes: MITIGATE FIRST (rollback/flag off/failover/shed) – root cause later.
 4. NARROW traces: which span grew? errors: which exception class exploded? one bad pod (skew) or fleet-wide? upstream or downstream?
 5. INSPECT logs by trace ID for exemplar requests; DB slow log; queue lag; connection pools; GC; recent schema changes.
 6. MITIGATE rollback, scale out, breaker/flag, shed load, warm caches, failover.
 7. VERIFY SLIs recover; watch for the second wave (retry storms, cold caches).
 8. POSTMORTEM blameless; timeline; contributing factors; action items with owners – the incident's value is the prevention it buys.

Habits that mark seniority: mitigate before root-causing; check deploys/flags *first*; distrust single signals (a healthy dashboard + screaming users = your telemetry is lying — probe from outside); keep hypotheses falsifiable ("if it were the DB, we'd see X"); write down the timeline as you go.

Interview Questions

1. Checkout errors at 2%: walk me through your first 10 minutes, out loud.
2. Metrics look healthy but users are down — how is that possible? (edge/DNS/CDN failure outside your measurement, wrong SLI vantage point, telemetry pipeline itself down → synthetic probes)
3. What makes a postmortem useful vs theater? (blameless, contributing factors over single cause, tracked action items)

Module 12 Cheat Sheet

PILLARS	metrics = cheap aggregate "what/how much"; logs = per-event "exactly what" (structured + trace IDs, scrubbed, sampled INFO never ERROR); traces = "where in the call graph" (context propagation, tail sampling).
FRAMEWORKS	RED (rate/errors/duration per service), USE (util/saturation/errors per resource), golden signals. Histograms → percentiles; cardinality kills.
ALERTING	page on SYMPTOMS (SLO burn-rate multi-window); causes → tickets/dashboards. Actionable+urgent+novel. Alert on absence. Runbooks. Synthetic probes from outside.
PROMETHEUS	pull /metrics, TSDB, PromQL, Alertmanager; K8s discovery; Thanos/Mimir for HA + long retention; aggregate buckets not percentiles.
GRAFANA	dashboards + LGTM stack; deploy annotations = fastest correlation tool.
OTEL	one SDK for all signals, auto-instrumentation, OTLP → Collector (batch/scrub/sample/route) → any backend. Kills vendor lock-in.
DEBUGGING	detect → scope → correlate CHANGES (mitigate first!) → narrow via traces → logs by trace ID → mitigate → verify (second wave) → blameless postmortem with owned actions.

Top Interview Questions (Module 12)

1. Design observability for a new microservice (what you emit, what pages).
2. p99 spike: which signal first and the full funnel. 3. Cardinality explosion — cause and guardrails. 4. Burn-rate alerting mechanics. 5. Global p99 across pods.
3. Pull vs push. 7. Tail vs head sampling. 8. OTEL's architectural value.
4. "Dashboards green, users down." 10. First 10 minutes of an incident.

Common Mistakes Recap

Unstructured logs • no trace IDs • user_id metric labels • paging on CPU • alert fatigue (non-actionable pages) • averaging percentiles • 1% head sampling losing all errors • no synthetic probes • root-causing before mitigating • logs-as-metrics alerting • no deploy markers.

Mock Interview Exercise

"You inherit a 30-service platform with printf logs, no tracing, and a single 'CPU > 80%' alert. Design the observability program and the rollout order." Expected: OTEL auto-instrumentation first (traces + RED metrics nearly free) → structured logging with trace IDs → SLOs per user journey + burn-rate paging + synthetic probes → Prometheus/Mimir + Grafana standard dashboards with deploy annotations → tail-sampled tracing → alert catalog review cadence. Justify the order by debugging value per unit effort.

Module 13 — Classic System Design Problems (Part 1)

Each design follows the full checklist: functional & non-functional requirements, capacity estimation, API, data model, high-level design, detailed design (the deep dive), scaling, failure handling, caching, security, and trade-offs. Closely related prompts (URL Shortener/TinyURL, WhatsApp/Chat, Uber/Ola, Airbnb/Hotel Booking) are treated together with their differences called out — exactly how you should handle them in a real interview.

13.1 URL Shortener / TinyURL

(One design — "TinyURL" is the brand name of the same problem.)

Functional: shorten long URL → short code; redirect; optional custom alias, expiration, click analytics. **Non-functional:** redirect p99 < 50 ms, 99.99% available (redirects must survive write-path death), read:write ≈ 100:1, codes non-guessable-ish, links live for years.

Capacity: 100M new URLs/month ≈ 40 writes/s (peak ~400); reads 100× ⇒ ~4k/s (peak 40k). Storage: 100M/month × 500 B ≈ 50 GB/yr — *tiny*; this problem is about latency + ID generation, not storage. 7 chars of base62 = $62^7 \approx 3.5 \times 10^{12}$ — enough for 1,000+ years.

API:

```
POST /api/urls {long_url, custom_alias?, expires_at?} → {short_url}
GET  /{code}   → 301/302 redirect
GET  /api/urls/{code}/stats → {clicks, referrers, countries}
```

301 (permanent, browsers cache — fewer hits, no analytics) vs 302 (every click hits you — analytics possible). Say the trade-off; pick 302 if analytics matter.

Data model: `urls(code PK, long_url, owner_id, created_at, expires_at, is_custom)` — a key-value access pattern ⇒ DynamoDB/Cassandra or sharded MySQL all work; partition by `code`.

High-level design:

```
write: client → API → ID generator → DB → (warm cache)
read:  client → CDN/edge? → LB → redirect svc → Redis (hot codes, ~99% hit)
      │                                     └ miss → DB → fill cache → 302
clicks → Kafka → analytics pipeline (never on the redirect critical path)
```

Deep dive — code generation (the expected one):

- *Counter + base62*: monotonic counter → base62 encode. Simple, no collisions; but sequential codes are enumerable (privacy) and the counter is a coordination point. Fix coordination with **ranged allocation**: each app instance leases a block (e.g., 1M IDs) from a coordination store (ZooKeeper/DB row with atomic increment); hands out locally, lease a new block when exhausted. Fix enumerability with a bijective scramble of the counter.
- *Random/hash*: generate random 7-char code, INSERT with unique constraint, retry on collision (rare while sparse). Stateless, non-guessable; retries rise as space fills.
- *Pre-generated pool*: offline job fills a table of unused codes; writers pop atomically. Fast writes, extra moving part.

Scaling: redirect service is stateless → horizontal; cache absorbs the Zipf head (hot 10% of codes ≈ 90+% of traffic); DB shards by code when needed; CDN/edge workers can even serve redirects for the hottest codes.

Failure: cache down → DB with concurrency cap (Module 3.7); write path down → *redirects still work* (separate

the planes — the availability win to narrate). **Caching:** cache-aside, TTL hours–days, negative caching for missing codes (bot/scan protection). **Security:** rate-limit creation, block-list malicious targets (Safe Browsing check async), private/expiring links, don't leak sequential IDs. **Trade-offs to volunteer:** 301 vs 302; counter (coordinated, enumerable) vs random (collision retries); SQL (simple, familiar) vs KV store (scale, TTL support built-in).

13.2 Instagram Feed / Facebook News Feed / Twitter(X) Timeline

(One core design — the feed/fan-out problem — with per-product deltas.)

Functional: post content; follow users; home feed of followees' posts, ranked (FB/IG) or ~chronological (classic Twitter); like/comment counts; media. **Non-functional:** feed load p99 < 200 ms, post visible to followers in seconds, read:write extremely skewed (Twitter-scale: ~300k feed reads/s vs ~6k tweets/s), eventual consistency fine everywhere except your own actions (read-your-writes).

Capacity (Twitter-flavored): 500M tweets/day $\approx$ 6k/s avg; feed reads ~300k/s; average 200 followers $\Rightarrow$ naive fan-out $\approx$ 1.2M timeline-insertions/s at peak — this number is the design problem. Media via object storage + CDN (Module 6).

API:

```
POST /posts {text, media_ids}      GET /feed?cursor=&limit=20
POST /users/{id}/follow            GET /users/{id}/posts?cursor=
POST /posts/{id}/like
```

Cursor pagination (Module 10.6) — feeds are the canonical case.

Data model: `posts(post_id SNOWFLAKE, author_id, text, media, created_at)` sharded by author (or post_id); `follows(follower_id, followee_id)` sharded by follower (need "who do I follow" fast) with a reverse index sharded by followee ("who follows me" — needed for fan-out); `timelines` in **Redis: list/zset per user, capped ~800 entries, storing post IDs only**. Snowflake IDs give time-ordered IDs without coordination.

High-level design & the core trade-off:

FAN-OUT ON WRITE (push)	FAN-OUT ON READ (pull)
post $\rightarrow$ Kafka $\rightarrow$ fanout workers	feed request $\rightarrow$ get followees $\rightarrow$
$\rightarrow$ LPUSH post_id into each	fetch each followee's recent posts
follower's Redis timeline	$\rightarrow$ merge by time $\rightarrow$ rank $\rightarrow$ return
read = one Redis fetch (fast!)	write = one insert (cheap!)
write cost = O(followers) $\otimes$ celebrities	read cost = O(followees) per load $\otimes$

The hybrid (the expected answer): push for normal users; **don't fan out celebrities** (>~100k followers). Feed read = fetch your precomputed timeline + fetch recent posts from the handful of celebrities you follow + merge at read time. Post-visibility SLO met via Kafka-buffered async fan-out (a post reaching 5M timelines takes seconds–minutes for the tail — acceptable; your own post appears instantly to you via read-your-writes injection).

Detailed design — read path: timeline (IDs) from Redis $\rightarrow$ hydrate posts (multi-get from post cache $\rightarrow$ DB on miss) $\rightarrow$ hydrate authors (cache) $\rightarrow$ counts (approximate counters, Module 3.6) $\rightarrow$ **ranking** (FB/IG delta: score candidates with an ML model — candidate generation: your timeline + collaborative/interest sources; feature fetch; scoring service; re-rank with diversity/integrity rules; this is a service pipeline between "fetch" and "return"). Likes: sharded counters, write-back batched to DB, reconciled offline.

Scaling: everything shards cleanly (timelines by user, posts by ID, fan-out workers by partition); Redis timeline tier is the big memory bill (~800 IDs $\times$ 8 B $\times$ 500M users $\approx$ 3 TB — fine across a cluster; evict cold users' timelines and rebuild on demand via pull — regenerable state!). **Failure:** fan-out lag $\Rightarrow$ stale feeds (monitor consumer lag, alert on p99 delivery time); Redis loss $\Rightarrow$ rebuild timelines lazily via pull path (design explicitly for regeneration); dedupe fan-out with idempotent LPUSH-if-absent (at-least-once Kafka). **Caching:** post cache, user cache, timeline is a

cache, CDN for media, L1 for celebrity posts (hot keys, Module 3.6). **Security:** private accounts filtered at fan-out and read (defense in depth), block/mute filters at read, rate-limit posting/likes (bots). **Trade-offs:** push vs pull vs hybrid (recite the celebrity math); chronological (simple, transparent) vs ranked (engagement, ML infra cost); capped timelines (bounded memory) vs deep history (pull fallback beyond the cap).

13.3 WhatsApp / Chat Application

(One design: 1:1 + group messaging; WhatsApp adds E2E encryption + mobile focus.)

Functional: 1:1 and group messages; delivery + read receipts; online/last-seen presence; media; message history sync across devices; typing indicators. **Non-functional:** delivery p99 < 500 ms online; **no message loss ever** (durability trumps latency); ordering per conversation; billions of devices, mostly idle connections; E2E encryption (WhatsApp).

Capacity (WhatsApp-scale): 2B users, ~100B messages/day $\approx$ 1.2M msg/s average (peak 3–5 $\times$). Concurrent connections ~500M $\Rightarrow$ at 1M conns/server (Erlang-style tuned hosts) $\approx$ 500–1000 connection servers. Message storage: 100B $\times$ ~100 B $\approx$ 10 TB/day raw (WhatsApp historically deletes after delivery; a Slack-like keeps forever — say which model you're building).

API / protocol: WebSocket (or raw TCP) with a binary frame protocol: `send(conv_id, client_msg_id, payload)`, server ACK, `deliver`, `receipt(delivered/read)`, `presence`, `typing`; HTTPS for login, media upload (presigned, Module 6.1), history sync.

Data model: `messages(conv_id, msg_id, sender, payload, ts)` — sharded/keyed by **conversation** (Cassandra/HBase-style wide rows: partition = conv_id (+ time bucket for huge groups), clustering by msg_id $\Rightarrow$ "recent messages of a chat" is one partition read); `conversations(conv_id, members...)`; `inbox/undelivered(user_id  $\rightarrow$  pending msg refs)` per recipient; `sessions(user_id  $\rightarrow$  connection server)` in Redis. Per-conversation **sequence numbers** give ordering + gap detection for client sync.

High-level design:

```
sender ==ws==> conn-srv A -> chat service: assign seq, PERSIST message,
                                write per-recipient inbox (durable) — ACK sender
                                | lookup recipient session (Redis registry)
                                |— online: route (pub/sub or direct) -> conn-srv C ==ws==> recipient
                                |   recipient ACK -> delete from inbox -> receipt to sender
                                |— offline: stays in inbox -> push notification (APNs/FCM)
                                |   -> delivered on reconnect (client sends last_seq, server
                                |       replays the gap)
```

Deep dive — the guarantees: durability before ACK (message persisted + inbox written, *then* sender sees $\checkmark$); delivery = at-least-once + client dedupe by `client_msg_id` (idempotency, Module 7.5); ordering per conversation via the sequence assigned by the conversation's owning partition (single-writer per conv — no distributed coordination needed). **Groups:** sender uploads once; server fans out to member inboxes (small groups) — for huge groups/channels, pull-on-open instead (same push/pull trade-off as feeds). **Presence:** heartbeats $\rightarrow$ presence service with debounced pub/sub to subscribed friends (don't broadcast every blip); last-seen written lazily. **E2E encryption (WhatsApp/Signal, one paragraph):** Signal protocol — X3DH key agreement + double ratchet per conversation; server stores/forwards ciphertext only; group = sender keys; consequence: server-side search/moderation impossible, multi-device needs per-device sessions.

Scaling: connection tier stateless-ish (sacrificial sockets, Module 1.8/2.6) — scale by count; chat/storage shards by conv_id; hot conversations (huge groups) get bucketed partitions. **Failure:** conn server dies $\rightarrow$ clients reconnect with backoff+jitter $\rightarrow$ resume via last_seq replay (zero loss because inbox is durable); datacenter loss $\rightarrow$ regional inboxes replicated (user-home-region model). **Caching:** session registry, recent messages per hot conv, profile/keys cache. **Security:** E2E, TLS to edge, contact discovery privacy, spam/abuse rate limits,

registration lock. **Trade-offs:** delete-after-delivery (WhatsApp: tiny storage, privacy) vs permanent history + search (Slack: storage + index cost); push-to-inbox vs pull-for-channels; presence freshness vs fan-out cost.

13.4 YouTube / Netflix (Video Platforms)

(Shared core: upload → transcode → CDN → adaptive streaming. Delta: YouTube = UGC at massive upload scale + virality; Netflix = small curated catalog, predictable, own CDN.)

Functional: upload (YT), transcode to multi-quality, stream with adaptive bitrate, search/browse, recommendations, resume position, view counts (YT). **Non-functional:** startup < 1–2 s, no rebuffering (the #1 UX metric), 99.99% playback availability, durable originals; YT: 500+ hours uploaded/minute; Netflix: ~15–30% of national downstream internet at peak.

Capacity sketch (YT): 500 h/min uploaded = 30k min of video/min; each minute ≈ several hundred MB across renditions ⇒ exabyte-class storage, growing PB/day ⇒ erasure coding + tiering are mandatory (Module 6). Streaming egress dominated by CDN (~99% offload target).

API: POST /videos → presigned multipart upload → {video_id}; GET /videos/{id}/manifest.m3u8 (HLS/DASH manifest); segment GETs go to CDN; POST /videos/{id}/progress (resume); search/browse APIs.

Data model: videos(id, owner, title, status: uploading→processing→live, renditions[]) in a DB; originals + segments in object storage; view counts in sharded counters; watch history in a wide-column store (Netflix: Cassandra — write-heavy, per-user partition).

High-level design (the pipeline):

```
upload (presigned multipart) → object storage → event → TRANSCODING PIPELINE:
  split video into chunks → parallel transcode (ladder: 240p...4K × codecs H.264/VP9/AV1)
  → package into 2–10 s segments (HLS/DASH) + manifests → object storage → CDN prewarm
playback: client → manifest → player picks bitrate per measured bandwidth,
  fetches segments from CDN edge; ABR adapts up/down per segment ← the key idea
```

Deep dive — adaptive bitrate: video = independently decodable segments in multiple qualities; the *client* measures throughput/buffer and chooses the next segment's rendition — no server session state, plain CDN-cacheable HTTP GETs (this answer explains both scale and smooth playback). **Transcoding at scale:** chunk-level parallelism (a 2 h movie transcodes in minutes across 1,000 workers), priority queues (new-upload vs backfill), idempotent chunk jobs + reassembly (Module 10.7). **View counting (YT):** raw events → Kafka → streaming aggregation → approximate live counter + exact batch reconciliation (never count on the read path). **Netflix deltas:** small catalog ⇒ **pre-position** the entire catalog on **Open Connect** appliances inside ISPs during off-peak (predictive caching from viewing forecasts — cache hit ~100%, backbone traffic ~zero at peak); per-title encoding optimization; recommendation-heavy homepage (precomputed per user, nightly + online adjustments). **YouTube deltas:** virality ⇒ hot-video handling (CDN request collapsing, multi-tier cache), upload spam/copyright (ContentID fingerprint matching in the pipeline), long-tail catalog ⇒ real cache-miss path matters.

Scaling: every stage horizontal (uploaders, transcoders, CDN, metadata shards); metadata reads cached heavily. **Failure:** transcode job death → idempotent retry per chunk; CDN region loss → reroute to next PoP (client retry logic + multi-CDN for Netflix); origin protected by shield tier. **Caching:** CDN is 99% of the design; manifest cache; metadata cache; home-page precomputation. **Security:** DRM (Widevine/FairPlay) for premium, signed segment URLs (leech protection), upload scanning. **Trade-offs:** own CDN (Netflix: control, cost at scale) vs commercial multi-CDN (YT is Google's edge anyway); more renditions (quality per user) vs storage/compute; live streaming adds latency-vs-buffer trade (shorter segments, LL-HLS).

13.5 Google Drive / Dropbox (File Sync & Storage)

Functional: upload/download; **sync across devices** (the hard part); share with permissions; version history; offline edits reconciled. **Non-functional:** never lose a byte (11-nines durability), sync latency seconds, efficient for large files and small edits (don't re-upload 2 GB for a 1 KB change), millions of concurrent syncing clients.

Capacity: 500M users × 10 GB avg = 5 EB logical; dedup + compression cut real bytes substantially (Dropbox reported large dedup wins). Metadata: billions of files × ~KB = tens of TB — a serious sharded DB of its own.

API: presigned block upload/download; `GET /delta?cursor=` (change feed per account — the sync primitive); `POST /files/{id}/share`; long-poll/WebSocket notification channel ("something changed, call /delta").

Data model — the block design (the expected deep dive):

```
files:      (file_id, ns_id, path, current_version)
versions:   (file_id, version, block_list[hash1, hash2, ...], size, mtime)
blocks:     content-addressed: hash(SHA-256) → object storage key   (4 MB chunks)
sharing:    namespaces (ns) with ACLs; shared folder = mount of another ns
```

- **Chunking:** files split into ~4 MB blocks, addressed by content hash.
- **Delta sync:** client hashes local blocks; uploads *only new hashes*; a small edit = 1 block, not the file. (Bonus point: content-defined chunking keeps edits from shifting all subsequent block boundaries.)
- **Dedup:** identical blocks (across users!) stored once — hash-addressed storage gives it for free (mention the privacy caveat of cross-user dedup).
- **Version history:** a version = a block list; old versions share unchanged blocks (cheap snapshots, Merkle-flavored).

Sync flow:

```
device edits file → chunk + hash → ask server which blocks are new → upload those
→ commit new version (block list) to metadata svc (transactional, detects conflict:
  base_version mismatch) → change feed entry → other devices notified (long-poll)
→ they pull /delta → download missing blocks → reassemble
conflict (offline concurrent edits): DON'T merge silently – keep both:
  "report (Bob's conflicted copy)" ← classic, honest answer
```

Scaling: metadata DB sharded by namespace/user; block storage is object storage (or Dropbox's Magic Pocket); notification tier = millions of idle long-poll/WS connections (Module 2.6). **Failure:** upload resume via already-have-hashes (idempotent by content address!); metadata/block consistency via commit-after-blocks + orphan GC (Module 6.4); client crash mid-sync → journal + re-diff. **Caching:** hot metadata, block presence checks via Bloom-filter-ish caches, CDN for popular shared files. **Security:** encryption at rest per-block, TLS, ACL checks on every namespace op (IDOR!), share links with scoped tokens, virus scan pipeline on shared content. **Trade-offs:** block size (small = better dedup/delta, more metadata; big = fewer round trips); server-side merge (Google Docs — needs OT/CRDT, format-aware) vs conflicted copies (Dropbox — format-agnostic, honest); strong metadata consistency + eventual block propagation.

13.6 Uber / Ola (Ride-Hailing)

(Same design; Ola = same problem in denser, cash-heavier markets.)

Functional: rider requests ride; match nearby driver; live location tracking; ETA + dynamic pricing; trip lifecycle (request → match → pickup → ride → payment → rating). **Non-functional:** match < 5–10 s; location update pipeline handles millions of drivers @ 1 update/4 s; high availability during a trip (an ongoing trip must survive service failures); payment consistency.

Capacity: 5M active drivers $\times$ 0.25 updates/s $\approx$ 1.25M location writes/s — **the location firehose is the design center**; matching QPS is small by comparison (peaks ~thousands/s). Location data is ephemeral (latest matters) $\Rightarrow$ in-memory store, trip history to durable storage async.

API: driver: `PUT /location {lat,lng,heading}` (over an open WS/gRPC stream); rider: `POST /rides {pickup, dest}` $\rightarrow$ offer states; `GET /rides/{id}` (live tracking via WS push); pricing internal.

Deep dive — geo-indexing (the expected one): you can't query "drivers within 2 km" from a lat/lng table at this rate. Discretize the world into cells — **H3 (Uber's hexagons)**, S2 (Google), or geohash:

```
location update: cell = h3(lat,lng, res≈9)
Redis: SADD cell:{cell} driver_id (+ per-driver latest point, TTL ~30 s
      - silence self-cleans) sharded by cell across the cluster
match query: cells = k-ring(pickup_cell, 1..2)  $\rightarrow$  union members  $\rightarrow$  filter
            (available, right vehicle)  $\rightarrow$  rank by ROAD-network ETA (routing svc), not
            straight-line  $\rightarrow$  offer to best driver (accept window)  $\rightarrow$  cascade on decline
```

Hexagons: uniform neighbor distances (better ring queries than squares); resolution trades precision vs cell population; **hot cells** (airport, stadium) = classic hot key $\rightarrow$ finer resolution locally + replicated reads (Module 3.6).

Trip lifecycle & consistency: trip = a state machine (REQUESTED $\rightarrow$ MATCHED $\rightarrow$ ARRIVING $\rightarrow$ IN_RIDE $\rightarrow$ COMPLETED $\rightarrow$ PAID) persisted in a durable DB (sharded by trip/city); transitions idempotent + versioned; matching offers use short locks/leases on the driver (one offer at a time) — but final assignment is a transactional state transition (double-assignment must be impossible: unique active-trip-per-driver constraint as the arbiter, Module 11.7's "DB as final lock"). Payment via saga: auth at match/start, capture at completion (Module 7.6). **Pricing/ETA:** surge = streaming supply/demand aggregation per cell (Flink over the location/request firehose); ETA = routing service over road graph with live traffic (precomputed contraction hierarchies + live edge weights).

Scaling: shard *everything* by city/region (natural isolation — an outage in one city stays there; also data residency); location tier by cell; matching stateless per region. **Failure:** location store loss = self-healing within seconds (next updates repopulate — say this!); ongoing trips unaffected (trip state durable elsewhere); driver app offline mid-trip $\rightarrow$ GPS buffered locally, reconciled. **Caching:** driver profiles, rider payment profiles, route/ETA memoization per cell-pair. **Security:** driver identity verification, location data privacy (retention limits, access audit), fraud (GPS spoof detection — plausibility checks), payment tokenization. **Trade-offs:** hex grid resolution; push offers to one driver (fairness, latency) vs broadcast (race, but faster in sparse areas — Ola/other markets differ); exact nearest (expensive) vs cell-approximate + ETA re-rank (standard).

13.7 Airbnb / Hotel Booking / Flight Booking

(One family: search $\rightarrow$ availability $\rightarrow$ reserve without double-booking $\rightarrow$ pay. Deltas: Airbnb = 1 unit per listing + two-sided marketplace; hotels = N identical rooms per type; flights = seats + fare classes + GDS legacy + extreme search fan-out.)

Functional: search (location/dates/filters); listing/room/flight details; availability calendar; **book without double-booking**; pay; cancel; host/inventory management. **Non-functional:** search p99 < 500 ms over millions of listings; **zero double-bookings** (strong consistency exactly here); booking flow 99.99%; search freshness eventual (seconds) is fine.

Capacity (Airbnb-ish): 10M listings; searches 10k/s peak; bookings ~100/s — *search is the scale problem, booking is the consistency problem*. Say that sentence; it structures everything.

API:

```
GET /search?loc&checkin&checkout&guests&filters&cursor
GET /listings/{id}?checkin&checkout (availability + priced total)
```

```
POST /bookings {listing_id, dates, idempotency_key} → PENDING PAYMENT
POST /bookings/{id}/confirm (payment)          DELETE /bookings/{id} (cancel policy)
```

Data model: `listings` (attributes, geo) in Postgres, sharded by region as needed; **availability calendar:** `calendar(listing_id, date, status, price)` — one row per listing-day (2-year window × 10M listings = 7B rows; fine sharded by `listing_id`); `bookings(id, listing_id, dates, status state machine, idem_key UNIQUE)`; search index in Elasticsearch fed by CDC (CQRS, Module 4.7); hotels: `room_inventory(hotel_id, room_type, date, total, reserved)` — *counts*, not units; flights: `seat_inventory(flight, fare_class, remaining)` + seat map.

High-level design:

```
search: query → ES (geo + dates + filters, pre-joined availability summary)
        → candidates → pricing svc (dynamic) → rank → results [eventually consistent]
booking: → booking svc → TRANSACTIONAL availability check+reserve (source of truth,
        strongly consistent) → payment saga → confirm [strictly consistent]
```

Deep dive — preventing double booking (the guaranteed question): the calendar DB is the arbiter, not the search index and not a cache:

```
BEGIN;
SELECT * FROM calendar WHERE listing_id=? AND date BETWEEN ? FOR UPDATE; -- lock range
-- all nights 'available'? else abort with friendly error
UPDATE calendar SET status='held', hold_expires=now()+'10 min' ...;
INSERT INTO bookings(..., status='PENDING_PAYMENT');
COMMIT;
```

Two-phase UX: **hold with TTL** during payment (expired holds swept back to available), confirm on payment success (saga: capture → CONFIRMED; failure → release, Module 7.6). Idempotency key on booking creation (double-click, retry safety). Hotels: `UPDATE room_inventory SET reserved=reserved+1 WHERE ... AND reserved < total` — atomic conditional decrement of a *count* (no per-unit rows; overselling policies are deliberate business config, not bugs). Flights: same conditional decrement per fare class; legacy twist — availability often lives in external **GDS** systems ⇒ cached availability + revalidate-at-booking + graceful "fare changed" UX (a great real-world wrinkle to mention).

Scaling: search via ES shards + caching of hot geo-date queries; calendar sharded by listing (all rows of one booking in one shard ⇒ local transaction — shard key chosen *for* the consistency need, Module 4.3). **Failure:** payment timeout → hold expiry sweeps (janitor); ES lag → final availability check at booking catches phantoms ("sorry, just taken" — quantify: lag seconds × booking rate = rare); region loss → bookings DB is the recovery priority (RPO≈0 semi-sync). **Caching:** search results (short TTL), listing details, pricing memoization — **never cache the final availability check**. **Security:** payment via PSP/tokenization, IDOR on bookings, scraping rate limits, fraud screening on bookings. **Trade-offs:** pessimistic lock/hold (simple, brief lock) vs optimistic (version check at confirm — better under low contention); denormalized availability-in-ES (fast search, staleness) vs query-time join (fresh, slow); hold TTL length (conversion vs inventory lockup).

13.8 Payment Gateway / Payment System

Functional: merchant charges a card (auth/capture/refund/void); support cards + wallets + bank rails; webhooks to merchants; ledger + reconciliation with processors. **Non-functional: never double-charge, never lose money** (exact consistency); p99 auth < 2 s (human at checkout); 99.99%+; PCI-DSS compliance; full auditability.

Capacity: Stripe-order-of-magnitude: ~10k payments/s peak. Small QPS, extreme correctness — the *inverse* of feed systems; say so.

API (Stripe-shaped):

```

POST /v1/payment_intents {amount, currency, ...} Idempotency-Key: <uuid> ← REQUIRED
POST /v1/payment_intents/{id}/confirm      POST /refunds {payment_intent, amount}
GET  /v1/payment_intents/{id}              webhooks: payment_succeeded/failed (signed)

```

Data model — the two pillars:

1. **Payment state machine:** `payments(id, merchant, amount, status: CREATED→AUTHORIZED→CAPTURED→SETTLED / FAILED / REFUNDED, psp_ref, idem_key UNIQUE)` — every transition idempotent, versioned, audit-logged.
2. **Double-entry ledger:** `ledger_entries(txn_id, account, debit, credit, ...)` — **append-only, immutable**; every money movement = balanced entries (merchant receivable ↔ processor clearing ↔ fees). Balances = derived (materialized, reconciled). Corrections = new reversing entries, never UPDATES. This is what "bank-grade" means in interviews.

High-level design:

```

merchant → API (idempotency layer, Module 7.5: key+response stored in-txn)
  → payment svc (state machine, ledger) → card vault (tokenized PAN, PCI island)
  → PSP/network adapter (Visa/MC/banks) – timeout+breaker+ONE retry w/ same
    idempotency ref → async webhooks (outbox→Kafka→signed delivery w/ retries)
  → reconciliation: daily processor settlement files diffed against ledger

```

Deep dive — the double-charge question: client retry → same Idempotency-Key → stored response returned (no re-execution). Crash *after* PSP call, *before* recording? On recovery the state machine is "in flight" → **query the PSP by our reference** (or void-and-retry) — never blind-retry a charge; unknown-outcome states get a reconciliation queue. External calls carry *our* idempotent reference so the PSP dedupes too. Auth-then-capture is itself the saga's reversible pivot (Module 7.6). **Reconciliation** (the senior differentiator): settlement files vs ledger, automated matching, exception queue for humans — every real payment company runs this; mentioning it signals authenticity.

Scaling: shard by merchant (hot merchants isolated); reads from replicas; ledger appends partition cleanly. QPS is modest — scale is in event/webhook fan-out and reporting (CQRS read models). **Failure:** PSP down → queue+retry auths? No — fail fast to user, offer retry (money UX beats silent queues); PSP *slow* → timeout+breaker+fallback processor (multi-PSP routing — also a cost optimization); datacenter loss → RPO 0 (semi-sync/quorum) because "lost payment record with captured money" is the nightmare scenario. **Caching:** none on the money path (only merchant config/keys). **Security:** PCI scope minimized via tokenization/vault (Module 9.6), mTLS internal, HSM for keys, signed webhooks, velocity/fraud scoring inline (ML risk score gate before auth), full audit trail. **Trade-offs:** sync auth + async everything-else; exactly-once *effect* via idempotency (over impossible exactly-once delivery); double-entry overhead vs auditability (no real trade — always double-entry, say why).

13.9 E-commerce Platform & Inventory Management

(Amazon-style storefront; inventory is its own deep dive and the requested "Inventory Management" system.)

Functional: browse/search catalog; product pages; cart; checkout (order, payment, inventory); order tracking; reviews. Inventory: track stock per SKU per warehouse; reserve at checkout; replenish; oversell policy. **Non-functional:** catalog reads massive (100k+ QPS peak) and cacheable; **inventory decrement strongly consistent**; order flow 99.99%; flash-sale spikes 100×; search freshness seconds.

Capacity: 50M products; 100k QPS browse peak; 1k orders/s peak (flash sale: one SKU, 10k attempts/s — the hot-row problem). Read:write ≈ 1000:1 on catalog.

API: GET /products/{id}, GET /search?q&filters, POST /cart/items, POST /checkout {cart_id, idempotency_key}, GET /orders/{id}; internal: POST /inventory/reserve {order_id, sku, qty, warehouse?}.

Data model: catalog in Postgres + ES (CDC) + heavy caching; cart in Redis (TTL, session-ish) backed to DB for logged-in persistence; orders(id, status state machine, items snapshot price/title – Module 4.5); inventory(sku, warehouse, on_hand, reserved) — available = on_hand - reserved; reservations(order_id, sku, qty, expires_at, status).

High-level design:

```
browse: CDN → catalog svc → Redis (L1+L2, Module 3) → DB      [eventual, cached]
search: ES read model (CDC-fed)                                [eventual]
checkout: order svc → SAGA: reserve inventory → charge payment → confirm order
          → outbox events → fulfillment / email / analytics    [strictly consistent core]
```

Deep dive — inventory correctness + the flash sale:

```
-- reservation (atomic conditional update; the arbiter):
UPDATE inventory SET reserved = reserved + :q
WHERE sku=:s AND warehouse=:w AND on_hand - reserved >= :q;  -- 0 rows = out of stock
```

Reserve with TTL at checkout-start (cart hoarding prevention), convert to deduction on payment success, sweep expired reservations (janitor) — the same hold pattern as bookings. **Flash sale hot row:** 10k TPS on one SKU row melts row locks ⇒ (a) **queue the attempts** (Kafka per-SKU partition = single-writer serialization, natural fairness, absorbs the spike — users get position/async confirm), or (b) shard the counter into N sub-buckets summed at read (Module 3.6) with rebalancing, or (c) pre-allocate stock tokens to app servers. State (a) as default — it converts a consistency problem into an ordering problem you already solved. Overselling: policy toggle (backorder allowed?) not an accident; warehouse-level allocation at fulfillment re-optimizes later. **Multi-warehouse:** reserve against aggregate or region-preferred warehouse; reconcile with WMS (physical counts drift — cycle-count adjustments are normal; another authenticity point).

Scaling: catalog/search/cart all shard trivially; orders shard by order_id/ customer; inventory shards by SKU (hot SKUs get the queue treatment). **Failure:** payment fails → saga compensation releases reservation; ES lag → "in stock" badge stale but checkout re-verifies (same phantom pattern as bookings); cache stampede on product drop → single-flight + prewarm (Module 3.5). **Caching:** product pages (CDN + Redis, invalidated via CDC), search results short-TTL, cart in Redis, **never** the reservation check. **Security:** idempotent checkout, payment tokenization, bot defense on drops (queue + rate limit + CAPTCHA), price-integrity (server-side price, never trust client), IDOR on orders. **Trade-offs:** reserve at add-to-cart (better UX, inventory lockup) vs at checkout (standard); sync inventory check on product page (fresh, load) vs cached-with-recheck (standard); monolith checkout (simpler txn) vs saga (scale/team autonomy).

(Part 2 continues with: Chat/Notification/Autocomplete/Crawler/Distributed Cache/Rate Limiter/API Gateway/Logging/Analytics — plus the Netflix/Google Drive deltas already covered above where shared.)

Module 13 — Classic System Design Problems (Part 2)

(Chat Application: fully covered in 13.3 (WhatsApp) — a generic chat app is the same design minus E2E encryption, plus permanent history + search (Slack model): keep messages forever in the conv-partitioned store, add an ES index fed by CDC, and per-channel pull for large channels.)

13.10 Notification Service

Functional: one API for all channels (push APNs/FCM, email, SMS, in-app); templates + localization; user preferences & opt-outs; scheduling; batching/digests; delivery tracking. **Non-functional:** OTPs in seconds (high priority lane); marketing can lag minutes; **at-least-once with dedupe** (never spam duplicates); peak bursts of 10M+ (a viral event or campaign blast); channel providers are flaky third parties.

Capacity: 500M notifications/day $\approx$ 6k/s average, campaign spikes 100k+/s $\Rightarrow$ queue-centric design is forced. Provider rate limits (SMS gateways especially) are the real throughput ceiling — mention this.

API:

```
POST /notifications {user_id | segment, template_id, data, channels?, priority,
                    idempotency_key, send_at?}
GET  /notifications/{id}/status      PUT /users/{id}/preferences
```

High-level design:

```
callers → notification API (validate, dedupe by idem key)
  → Kafka, PARTITIONED LANES: critical(OTP) / transactional / marketing ◀ priority
  → workers: preference+quiet-hours check → template render (i18n) →
    rate limit per user ("max N/day") + collapse/digest →
    channel adapters: [APNs] [FCM] [SES/SMTP] [Twilio/SMS] [in-app inbox]
      each: provider rate limiter + retries + breaker + failover provider
  → delivery receipts/webhooks → status store → analytics (open/click)
  → DLQ per channel + alerting
```

Deep dives worth offering: priority isolation (separate topics/worker pools — a marketing blast must never delay OTPs: bulkhead, Module 7.4); per-user frequency capping and collapsing ("5 likes" not 5 pushes) via a short aggregation window; idempotency at two levels (API key dedupe + channel-adapter send-once records); device token lifecycle (invalid-token feedback from APNs/FCM prunes the registry); provider failover (Twilio down $\rightarrow$ backup SMS route). **Scaling:** everything is stateless workers on partitioned queues — scale consumers; segment-blast campaigns are expanded (segment $\rightarrow$ user IDs) by a fan-out stage *into* the same pipeline. **Failure:** provider outage $\rightarrow$ breaker + secondary provider + backlog in Kafka (bounded by retention — shed marketing first, keep OTP lane); duplicate webhooks from providers $\rightarrow$ idempotent status updates. **Caching:** preferences, templates, device tokens. **Security:** only trusted internal callers (this service can spam your entire user base), template injection safety, PII minimization in queues, unsubscribe compliance (CAN-SPAM/GDPR). **Trade-offs:** unified service (consistent prefs/limits, one throat to choke) vs per-team sending (autonomy, chaos); push now vs digest (engagement vs annoyance); at-least-once+dedupe vs at-most-once (OTP must never be lost; a rare duplicate OTP is harmless).

13.11 Search Autocomplete (Typeahead)

Functional: top-k suggestions per prefix as the user types; ranked by popularity (+freshness, +personalization optionally); trending queries appear within hours/minutes. **Non-functional:** p99 < 50–100 ms (it races keystrokes);

massive read QPS (every keystroke of every user); suggestion data eventually consistent (rebuilt periodically) — reads must never touch the pipeline.

Capacity: 5B searches/day, ~4 keystrokes/query with client debounce $\Rightarrow$ ~200k+ lookup QPS peak. Corpus: top ~10M queries suffice; a precomputed prefix $\rightarrow$ top-10 map $\approx$ few GB — **fits in RAM**; that observation is the design.

API: GET /suggest?q=<prefix>&limit=10 (+user/locale) — client debounces (~100–150 ms) and cancels stale requests.

Deep dive — the two-plane design (the expected answer):

```
SERVING (read) PLANE – precomputed, in-memory:
  trie where each node stores its top-10 completions cached at the node
  (or literally a hash map: prefix( $\leq n$  chars)  $\rightarrow$  [top10])  $\rightarrow$  O(1) lookup, no
  traversal at query time. Sharded by prefix range across replicas; small
  enough to replicate fully per node. CDN/edge cache for hottest prefixes.

BUILD (write) PLANE – offline/streaming aggregation:
  search logs  $\rightarrow$  Kafka  $\rightarrow$  hourly/daily batch (count, decay old counts,
  filter: spam/adult/PII)  $\rightarrow$  build trie/prefix tables  $\rightarrow$  ship immutable
  snapshot to serving nodes (atomic swap, versioned)
  + streaming fast path for trending: last-hour counts merged into results
    with a recency boost
```

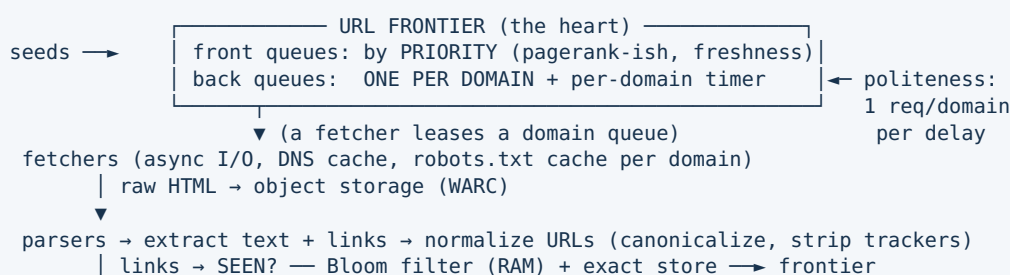
Key points to narrate: **precompute top-k at build time** (query-time ranking of thousands of completions is unaffordable at 50 ms); weight = frequency with time-decay (trending); atomic snapshot swap = trivially consistent serving + easy rollback; personalization as a thin re-rank layer over global results (client or edge merges "your recent searches"). **Scaling:** read plane scales by replication (data is small and read-only — the easiest scaling in this whole module); build plane is batch (Module 10.7). **Failure:** serving node dies $\rightarrow$ LB reroutes (read-only replicas); bad build (offensive suggestion ships) $\rightarrow$ version rollback + blocklist hotfix path. **Caching:** the serving plane is a cache; plus browser/ edge caching per prefix (short TTL). **Security/quality:** filter pipeline (profanity, PII, legal removals), per-locale corpora, abuse (query-log poisoning: rate-limit and dedupe by user before counting). **Trade-offs:** trie (memory- efficient shared prefixes, range flexibility) vs flat prefix map (simplest, more memory); update latency (rebuild cadence) vs freshness (streaming merge complexity); global vs personalized (privacy + memory per user).

13.12 Web Crawler

Functional: given seeds, download the web: fetch $\rightarrow$ parse $\rightarrow$ extract links $\rightarrow$ enqueue new URLs $\rightarrow$ store pages for indexing; re-crawl by freshness policy. **Non-functional:** billions of pages; **politeness** (per-domain rate limits, robots.txt — a crawler that DDoSes sites gets blocked/lawyerred); dedupe (URL and content); fault tolerant over weeks-long runs; prioritized (important pages first, freshness-aware re-crawl).

Capacity: 1B pages in 30 days $\approx$ 400 pages/s sustained (peaks higher); avg page ~100 KB $\Rightarrow$ ~40 MB/s ingest, 100 TB total (object storage, compressed). URL frontier: tens of billions of URLs seen $\Rightarrow$ dedupe structure must be RAM-efficient (Bloom filter territory, Module 11.2).

High-level design:



└ content fingerprint (SimHash) → near-duplicate? skip indexing
▼
downstream: indexer / freshness scheduler (re-crawl queue by change rate)

Deep dives: the **frontier** is the interview centerpiece — it must simultaneously enforce politeness (per-domain serialization + delay ⇒ back queue per domain, fetchers lease domains, timers gate re-fetch) and priority (front queues feed back queues by importance). **Dedupe at two levels:** URL-seen (Bloom filter for the fast "definitely new" check — false positives just skip a URL, tolerable — backed by an exact KV store) and content-seen (SimHash/MinHash near-dup detection — mirrors, tracking-param variants). **Traps:** spider traps (infinite calendars, session-ID URL spaces) → max depth/URL-count per domain + pattern detection; JS-rendered pages → headless-render tier for the subset worth it (10× cost — budget it). **Scaling:** partition frontier + fetchers **by domain hash** (politeness state stays local — no cross-node coordination per request); DNS resolution needs its own cache/resolver fleet (a real bottleneck at 400+ fetches/s). **Failure:** fetcher dies → domain lease expires → another picks up; frontier checkpointed (it is the crawl state); fetch failures → retry with backoff, mark dead after N (transient vs permanent, Module 5.6). **Caching:** robots.txt (per domain, TTL 24 h), DNS, conditional GETs (ETag/If-Modified-Since — save bandwidth on re-crawl). **Security/etiquette:** honor robots + crawl-delay, identify honestly in User-Agent, contactable operator, avoid login areas, malware-scan stored content (you're storing the web's worst too). **Trade-offs:** BFS-by-priority (news first) vs pure BFS; freshness (aggressive re-crawl of hot pages) vs coverage (crawl budget is finite — allocate by change-rate estimation); Bloom false positives (miss a few URLs) vs exact-only dedupe (RAM/latency).

13.13 Distributed Cache (design Redis/Memcached itself)

(Assembles Modules 3 + 11 into one system — a favorite "infra" prompt.)

Functional: GET/SET/DEL with TTL; atomic ops (INCR, CAS); cluster grows/shrinks online; optional replication.

Non-functional: sub-ms p99; millions of ops/s across the cluster; cache semantics (loss tolerable, availability + latency over durability); minimal disruption on topology change.

Capacity example: 1 TB working set, 5M ops/s ⇒ 16–32 nodes (64 GB each, 200–500k ops/s each), ×2 for replicas + headroom.

API: GET k / SET k v EX ttl / DEL k / INCR k / CAS k v version — plus MGET batching (protocol design point: pipelining matters more than fancy ops).

Design:

```
clients (smart lib: topology map, hashing, timeouts, single-flight)
  | consistent hashing w/ VIRTUAL NODES (Module 11.1) or hash-slot table
  ▼
node = single-threaded event loop (atomicity for free) + hash table
      + LRU/LFU eviction (W-TinyLFU) + TTL wheel (lazy + periodic expiry)
      + optional async replica (failover target)
cluster membership: gossip (Module 11.6) or small Raft metadata core (Module 8.4)
topology change: slot/arc migration node→node while serving (MOVED redirects)
```

Deep dives: memory management (slab classes vs jemalloc — fragmentation is the silent killer; eviction under pressure must be O(1)); threading model (single-threaded loop + I/O threads vs sharded-per-core (memcached) — atomicity vs per-node vertical scale); consistency choice — *a cache chooses AP*: async replication, failover may serve slightly stale or lose recent sets (acceptable — the DB is the source of truth; say it); hot keys (detection + client L1 + key replication, Module 3.6); client library as part of the system (timeouts ~50 ms fail-fast, backpressure, single-flight — Module 3.7). **Scaling:** add node → consistent hashing moves ~1/N keys (misses refill from DB — plan the DB headroom); scale reads via replicas. **Failure:** node death → its arc goes cold, DB absorbs refill (this cascade is the #1 thing to size for); partition → prefer availability (serve possibly-stale, no quorum reads). **Security:** network isolation, AUTH, TLS optional (latency cost), never public. **Trade-offs:**

consistent hashing (decentralized) vs slot table (operable, explicit migration — Redis chose slots); replication (fast failover, memory $\times 2$) vs none (cheaper, cold-start on failure); strong-ish consistency (WAIT/sync) vs the point of a cache.

13.14 Rate Limiter (as a system)

(Algorithms in Module 9.5 — here, the distributed service design interviewers ask for.)

Functional: enforce configurable limits (per user/API-key/IP/endpoint; rate + burst); return 429 + Retry-After + headers; runtime config changes; shadow mode. **Non-functional:** added latency $\leq 1\text{--}2$ ms p99; must not become the outage (explicit fail-open/closed policy); millions of decisions/s; approximate limits acceptable ($\pm$ small % — say this, it unlocks the good designs).

Design options (present all three, recommend per context):

A) gateway-local buckets (each node gets rate/N)	zero latency, limit accuracy $\pm N_{\text{nodes}}$ slice, skewed traffic breaks it — fine for coarse DDoS
B) central Redis counters (token bucket per key)	accurate global; +1 round trip ($\sim 1\text{ms}$); Lua for atomic check+decrement; shard by key; hot-key celebrities $\rightarrow$ local pre-allocation on top
C) local + async sync (hybrid) "token borrowing"	near-zero latency, near-accurate: nodes consume from local slice, periodically settle with a central budget — the production sweet spot

Redis token bucket (the snippet to whiteboard): store `{tokens, last_refill}` per key; Lua: `tokens = min(burst, tokens + rate $\cdot\Delta t$ )`; if `tokens $\geq 1$` then `tokens--1`; allow. Memory: 2 floats/key $\times$ 100M active keys $\approx$ GBs — fine sharded. **Config plane:** rules in a config store, pushed to enforcers (versioned, canary, shadow mode first — enforcement bugs = self-inflicted outage). **Failure:** Redis down $\rightarrow$ **fail-open for availability-tier APIs, fail-closed for abuse-sensitive endpoints (login/OTP)** — a per-rule policy, the senior nuance; local fallback limiter as middle ground. **Where it lives:** gateway middleware (Module 2.5) or sidecar; keep decision (policy) separate from counting (state). **Trade-offs:** accuracy vs latency ($A \leftrightarrow B \leftrightarrow C$); global vs per-region limits (cross-region sync is not worth it — limit per region, cap globally via async settle); token bucket (burst-friendly API semantics) vs sliding window (smoother, no burst) — Module 9.5.

13.15 API Gateway (as a system)

(Role in Modules 2.5/7.3 — here, designing the gateway itself.)

Functional: routing (host/path/header $\rightarrow$ service), authn (JWT/OIDC validation, API keys), rate limiting/quotas, TLS termination, retries/timeouts/breakers, canary + traffic splitting, request/response transforms, observability emission. **Non-functional:** it's in front of *everything*: p99 overhead ≤ 5 ms, 99.99%+, horizontal scale to full site traffic, config changes without restarts and without global blast radius.

Design:

clients $\rightarrow$ anycast/L4 (Module 2.5) $\rightarrow$ GATEWAY FLEET (stateless!)
per request: TLS $\rightarrow$ route match $\rightarrow$ authn (JWT verify LOCALLY via cached JWKS —
no auth-service call on hot path) $\rightarrow$ rate limit (13.14 hybrid) $\rightarrow$ transform $\rightarrow$
proxy (connection pools per upstream, retry budget, breaker, deadline) $\rightarrow$
emit metrics/traces/logs
CONTROL PLANE (separate!): config API/GitOps $\rightarrow$ validate $\rightarrow$ version $\rightarrow$ push (xDS-
style) to data plane; canary configs; instant rollback
state: none locally — JWKS cache, route table cache, rate-limit via 13.14

Deep dives: data plane vs control plane separation (Envoy/xDS model — config distribution is itself a distributed system; a bad config push is the gateway's

1 outage cause ⇒ validation + canary + auto-rollback on error-rate spike);

zero-downtime config/binary reload (hot restart, connection draining); per-tenant isolation (one customer's 10× spike must not eat the fleet — per-tenant concurrency caps = bulkheads); keep business logic OUT (the anti-pattern from 7.3); BFF variants per client type. **Scaling:** stateless horizontal; upstream connection pooling (HTTP/2 multiplexing to services); shard by host/SNI if config becomes huge. **Failure:** gateway node death (LB reroutes, conns re-established); auth-provider down → cached JWKS keeps validating (tokens self-contained — a JWT design win, Module 9.2); upstream down → breaker + fallback response. **Caching:** JWKS, route tables, optional response caching for public GETs. **Security:** it is the security perimeter — TLS policy, WAF integration, header sanitization (strip client-sent internal headers!), request size/time limits, mTLS to upstreams. **Trade-offs:** one shared gateway (consistency, ops leverage) vs per-domain gateways (blast-radius isolation, team autonomy); rich gateway (fewer sidecar concerns) vs thin gateway + mesh (east-west parity).

13.16 Logging System (Centralized Log Platform)

Functional: collect logs from every host/container; parse/structure; search (near-real-time, by service/time/trace-id/full-text); retention tiers; alerts on patterns; access control. **Non-functional:** ingest TB–PB/day (write-heavy, Module 1.7!); search latency seconds for recent data; **lossy is unacceptable for audit logs, acceptable-with-bounds for debug logs** (two tiers — say it); cost is a first-class constraint (logs are the biggest observability bill).

Capacity: 10k hosts × 50 GB/day ≈ 500 TB/day raw ⇒ compression (10×) + tiering + sampling are not optional. Peak ingest during incidents is 5–10× normal — exactly when you can't drop data; buffer accordingly.

Design:

```
apps → stdout → node AGENT (Fluent Bit/Vector: tail, batch, compress, spill-to-disk buffer — survives pipeline outages)
  → Kafka (the shock absorber; retention = your outage budget)
  → processors: parse → structure (JSON) → enrich (k8s metadata, geo) → PII scrub
  → route by stream:
    hot tier (7–30 d): Elasticsearch/Loki — indexed search
    warm/cold (1–7 y): object storage, Parquet/compressed, queried by batch engines (Athena-style) on demand
  → real-time matchers → alerts (error-pattern spikes)
```

Deep dives: the indexing trade-off — Elasticsearch indexes everything (fast arbitrary search, huge write amplification + storage) vs **Loki's model** (index only labels {service, level, pod}, grep the compressed chunks at query time — 10× cheaper, slower needle-searches; the modern cost answer); ordering/dedup (logs are at-least-once — consumers dedupe by (host, file, offset)); backpressure (agent disk buffers → Kafka lag → shed DEBUG before INFO before ERROR — priority shedding); multi-tenancy (per-team quotas — one team's log loop must not drown the platform: bulkhead again). **Scaling:** every stage partitions by stream/tenant; ES needs index lifecycle management (daily indices, rollover, shrink). **Failure:** ES down → Kafka buffers (size retention for your worst honest outage), agents spill locally; the meta-problem — *the logging system's own logs* (self-monitoring via metrics, not logs). **Caching:** recent-query results, field-stats. **Security:** PII scrubbing at ingest (compliance), RBAC per index/tenant, audit logs immutable (WORM object storage) + separate retention. **Trade-offs:** index-everything vs label-index (cost vs search power); retention length vs cost (tiering); sampling debug logs (10%) vs completeness (never sample errors/audit).

13.17 Analytics Platform (Events → Insights)

Functional: ingest product events (clicks, views, transactions) from clients + services; real-time dashboards (minutes) + historical/ad-hoc SQL (years); funnels/retention/aggregations; feed ML/experiments. **Non-functional:** 1M+ events/s ingest; ingestion loss < 0.1%; dashboard queries seconds over billions of rows; real-time lag ≤ 1 min; schema evolution without breakage; cost-efficient at PB scale.

Capacity: 1M events/s × 1 KB ≈ 1 GB/s ≈ 85 TB/day raw → columnar+compressed ~10–20 TB/day stored. This is the definitive write-heavy system (Module 1.7).

Design (the lambda/kappa shape, Module 10.7):

```
clients/SDKs (batch, retry, offline buffer) + services
→ collection endpoint (validate against SCHEMA REGISTRY, enrich, sessionize-id)
→ Kafka (partition by user/session for ordered per-user streams)
  | STREAM path: Flink – windowed aggregates (event-time + watermarks),
  | sessionization, funnels → real-time store (Druid/Pinot/ClickHouse)
  | → live dashboards (lag: seconds–1 min)
  | BATCH path: raw → object-storage DATA LAKE (Parquet, partitioned
  | dt/hour) → warehouse/lakehouse (Snowflake/BigQuery/Trino+Iceberg)
  | → ad-hoc SQL, experiment analysis, ML features, BACKFILLS/corrections
```

Deep dives: columnar storage is the whole trick (scan 3 columns of 300 — 10–100× less I/O; +RLE/dictionary compression; partition pruning by date) — this one paragraph answers "how do you query billions of rows in seconds"; **exactly-once-ish counting** (client dedupe IDs + Flink checkpointed state + idempotent sink upserts — and honest reconciliation: batch recount corrects stream drift nightly, the lambda reconciliation); **late events** (mobile offline hours → watermarks + allowed-lateness + corrections, Module 10.7); **schema registry** governance (typed events, compat rules — else the lake becomes a swamp); privacy (consent flags at ingest, PII vaulting/pseudonymization, GDPR deletes in an immutable lake → per-user encryption keys, crypto-shredding, Module 9.6). **Scaling:** Kafka partitions; Flink parallelism; OLAP stores shard by time+tenant; lake is object storage (infinite). **Failure:** OLAP store down → dashboards stale, ingest unaffected (Kafka buffers — decoupling win); bad deploy corrupts aggregates → **replay from Kafka/lake** (kappa's superpower — recompute is a feature, design for it). **Caching:** dashboard query cache, pre-aggregated cubes (hourly rollups) for the hot 20 dashboards. **Security:** row/column-level access in the warehouse, audit, tokenized joins for sensitive analysis. **Trade-offs:** stream+batch (lambda: fast + correct, dual code) vs kappa (one path, replay for corrections); Druid/Pinot (sub-second, ops-heavy) vs ClickHouse (simpler, powerful) vs warehouse-only (minutes latency, cheapest ops); raw-forever (rebuild anything, storage cost) vs aggregate-only (cheap, irreversible).

Module 13 Cheat Sheet — patterns that repeat across all 27 problems

SEPARATE PLANES	read path vs write path vs analytics path – scale and fail independently (shortener redirects, feeds, bookings, logs).
PRECOMPUTE	fan-out-on-write timelines, autocomplete top-k, materialized dashboards, CDN prewarm – spend write-time to buy read-time.
CELEBRITY/HOT KEY	hybrid push-pull, L1 caches, sharded counters, per-SKU queues, finer geo-cells – every domain has its celebrity.
THE ARBITER	exactly ONE strongly-consistent decision point per invariant: calendar row (booking), inventory row (stock), ledger (money), unique constraint (usernames). Everything else can be eventual.
HOLD + TTL + SWEEP IDEMPOTENCY	reservations (seats, stock, locks) expire; janitors reconcile. keys on every mutating API; dedup records in-transaction; at-least-once everywhere ⇒ consumers dedupe (chat, payments, notifications, analytics).
QUEUE AS SHOCK ABSORBER	Kafka absorbs bursts (orders, notifications, logs, events); lag is the health metric; retention is the outage budget.
STATE MACHINES	trips, orders, payments, bookings – explicit statuses, idempotent transitions, visible intermediate states.
DERIVED READ MODELS	CDC → ES/caches/warehouses; rebuildable, lag-monitored;

	final check against the arbiter catches phantoms.
GEO/DOMAIN SHARDING	city for Uber, conversation for chat, seller/listing for marketplaces, domain for crawlers – shard key = access pattern + isolation boundary.
CLIENT AS COMPONENT	debounce (autocomplete), ABR (video), resume cursors (chat), offline buffers (analytics), retry+jitter – design the client.
REGENERABLE STATE	location cells, timelines, caches, read models – losable state that rebuilds beats precious state that can't.

Mock Interview Drill (Module 13)

Pick any problem above; give yourself 35 minutes and produce, in order: requirements (5) → capacity (3) → API (3) → data model (5) → high-level diagram (5) → one deep dive chosen by "what's hardest here" (10) → failure + monitoring sweep (4). Then check: did you name the arbiter? the hot key? the queue? the idempotency story? Those four questions catch 90% of missed points.

Module 14 — Interview Strategy

Everything before this module is knowledge; this module is execution. A candidate with 80% of the knowledge and a disciplined process beats a candidate with 100% of the knowledge who rambles. Interviewers grade the *process* as much as the design.

14.1 The 45–60 Minute Structure (with time budget)

45-minute version

- 0–5 Requirements gathering (functional + non-functional + scope cuts)
- 5–8 Capacity estimation (only the numbers that change the design)
- 8–12 API design + data model sketch
- 12–20 High-level architecture (whole system, end to end, working)
- 20–35 DEEP DIVES (2–3, interviewer-steered) – where senior signal lives
- 35–40 Scaling, bottlenecks, failure handling sweep
- 40–45 Monitoring, security, trade-off recap, wrap-up
- └ 60-min version: stretch deep dives to ~25 min and add a 3rd dive ─┐

Two meta-rules: **narrate constantly** (silent whiteboarding is unassessable — think out loud, including uncertainty: "two options here, X is simpler, Y scales better; given our 10k QPS I'd start with X"), and **drive, but check in** ("I'll design the write path first, then reads — sound good?"). The interviewer's redirections are *gifts*: they're steering you toward the signal they need.

14.2 Requirement Gathering (minutes 0–5)

Never design an unscoped problem — jumping straight to boxes is the #1 rejection pattern at senior level.

Functional: "Who are the users? What are the 3–5 core operations? What's explicitly OUT of scope?" Cut scope aloud: "I'll focus on posting and feeds; search and DMs out of scope unless you want them." (Scope-cutting is a *positive* signal — it's what staff engineers do all day.)

Non-functional — extract numbers, not adjectives:

- Scale: DAU? QPS? read:write ratio? data size + growth?
- Latency: p99 targets per operation?
- Availability: how bad is downtime? (drives redundancy spend)
- Consistency: per operation — what *must* be strong? (Module 1.4 spectrum)
- Special: multi-region? compliance? spiky traffic?

If the interviewer says "you decide" — decide, state the assumption, move on ("Assuming 100M DAU and read-heavy 100:1 — correct me if you want a different scale"). Assumption-stating is graded; dithering is not.

14.3 Capacity Estimation (minutes 5–8)

Estimate **only what changes the design** — the goal is decisions, not arithmetic theater. Keep numbers round (powers of 10).

The five numbers that matter:

1. write QPS = daily writes / 86,400 $\approx$ /10⁵, then $\times 2$ –5 for peak
2. read QPS = write QPS $\times$ read ratio

```
3. storage    = items/day × size × retention (×replication)
4. bandwidth  = QPS × payload (matters for media/video only)
5. memory     = hot working set (cache sizing: ~20% of data serves ~80%+)
```

Numbers to know cold: 1M/day $\approx$ 12/s. 100M DAU $\times$ 10 actions $\approx$ 12k/s avg.
1 server $\approx$ thousands of simple QPS. Redis node $\approx$ 100k+ ops/s.
SQL node $\approx$ low thousands of writes/s. Kafka $\approx$ effectively unbounded (partition).

Then *use* them aloud: "6k writes/s — a single Postgres can't take that long-term $\Rightarrow$ shard or buffer through Kafka. 300k reads/s $\Rightarrow$ cache-first architecture." That sentence — numbers forcing decisions — is precisely what's graded.

14.4 API Design & Data Model (minutes 8–12)

- APIs: 4–6 core endpoints, REST-ish, with pagination cursors, idempotency keys on mutations, and auth noted. Don't enumerate CRUD for every entity.
- Data model: main entities, keys, and **the shard/partition key with justification** (Module 4.3 — this is the moment to align storage with access patterns). Note which store type each entity gets and why (Module 4.1).
- Flag the consistency class per entity as you go ("inventory row: strong; view counts: eventual").

14.5 High-Level Design (minutes 12–20)

Draw the complete request path(s) end to end — client $\rightarrow$ edge $\rightarrow$ services $\rightarrow$ data — for the top 2–3 operations, *before* zooming anywhere. A working whole beats a perfect fragment. Standard shape:

```
client  $\rightarrow$  DNS/CDN  $\rightarrow$  LB  $\rightarrow$  gateway (auth, rate limit)  $\rightarrow$  services  $\rightarrow$  cache  $\rightarrow$  DB
                                      $\searrow$  queue  $\rightarrow$  async workers  $\rightarrow$  derived stores
```

Then annotate: which paths are sync vs async, where the arbiter of each invariant lives (Module 13 cheat sheet), what's stateless vs stateful. Invite steering: "Where would you like to go deep — the fan-out, the storage, or the failure story?"

14.6 Deep Dives (minutes 20–35) — where offers are decided

The interviewer picks 2–3 hard spots (or you propose the hardest honestly). The expected *shape* of a senior deep-dive answer:

1. State the problem crisply ("celebrity fan-out: 100M timeline inserts for one tweet").
2. Offer 2–3 approaches with trade-offs (push / pull / hybrid).
3. **Pick one and justify with the requirements' numbers.**
4. Detail the mechanics (data structures, exact flow, edge cases).
5. Volunteer the failure modes and how you'd detect them.

Depth probes to expect, per topic: exact SQL/locking for double-booking; idempotency-record transactionality for payments; rebalancing for sharded anything; ordering for messaging; stampede/hot-key for caching. (All covered in Modules 3–13 — this is what those "Interview Questions" sections were training.)

14.7 Bottlenecks, Scaling, Failure Sweep (minutes 35–40)

Run a systematic sweep — out loud, layer by layer:

- **Bottleneck hunt:** "At 10× load, what breaks first?" (usually: DB writes → shard/buffer; then hot keys; then the queue's consumers).
- **Failure walk:** kill each box in the diagram — LB, service, cache, DB leader, a whole AZ, the queue — and say what users experience and what recovers automatically (Modules 7–8 vocabulary: timeouts, breakers, failover, fencing, DR tiers).
- **Degradation plan:** what gets shed first, what serves stale, what fails closed (Module 3.7, 9.5).

Volunteering failure analysis *unprompted* is one of the strongest staff signals.

14.8 Monitoring, Security, Final Review (minutes 40–45)

- **Monitoring:** name the 3–5 SLIs, the burn-rate paging policy, and the one metric per subsystem (consumer lag, cache hit ratio, replication lag, DLQ depth) — Module 12.
- **Security:** authn/authz placement, rate limits, encryption, the invariant-protecting bits (idempotency, tokenization) — Module 9, two minutes max.
- **Final review (do not skip):** restate requirements → confirm the design meets each, with the number ("300k reads/s ⇒ 99% cache hit ⇒ 3k DB reads/s ✓"); recap the 2–3 biggest trade-offs you consciously made; name what you'd do next with more time. This closing loop is disproportionately memorable to interviewers.

14.9 How Interviewers Evaluate You

The rubric behind the scorecard (names vary by company; the axes don't):

Axis	Mid-level looks like	Senior/Staff looks like
Problem structuring	jumps to solution	scopes, prioritizes, states assumptions
Technical depth	names components	explains internals 3 levels down on demand
Trade-off reasoning	one memorized answer	2–3 options, chooses via requirements' numbers
Quantitative sense	skips or fumbles math	numbers drive decisions
Failure thinking	waits to be asked	volunteers failure modes + detection + degradation
Communication	silent or rambling	narrated, structured, checks in, adapts to hints
Pragmatism	over-engineers (or under-)	simplest thing that meets requirements; complexity budgeted

Calibration signals per level: **senior** = owns the whole design, deep on 2–3 components, sound trade-offs; **staff** = additionally frames the problem better than given, connects to org/operational reality (migrations, on-call, cost), handles adversarial follow-ups ("your cache died at peak — now what?") without wobbling. Instant negative signals: buzzwords without mechanics ("just use Kafka"), one-size answers ("NoSQL scales"), ignoring interviewer hints twice, no numbers anywhere, and inventing requirements to fit a rehearsed design.

14.10 Common Failure Patterns (and the fix)

1. **The memorized-design dump** — answering the question you prepared, not the one asked. *Fix: requirements first; let their numbers pick the architecture.*

2. **Depth-first rabbit hole** — 25 minutes on ID generation, no end-to-end system. *Fix: breadth first, then interviewer-guided depth.*
3. **The silent artist** — beautiful diagram, no narration. *Fix: think out loud; the diagram is evidence, the reasoning is the product.*
4. **Adjective engineering** — "highly scalable, super available" with no numbers. *Fix: every claim gets a number or a mechanism.*
5. **Happy-path only** — no failure story until asked, then improvised. *Fix: the 14.7 sweep, always.*
6. **Hint-deafness** — interviewer says "interesting, what about writes?" three times. *Fix: treat every interviewer question as the syllabus.*
7. **Over-engineering** — 12 services and 4 databases for 200 QPS. *Fix: start simple, add complexity only when a stated number forces it — and say which number.*

14.11 Preparation Plan & Mock Protocol

Drill cadence (using this guide): for each of Modules 1–12, do the module's mock exercise cold, then grade against its cheat sheet. For Module 13: two problems per sitting, 35 minutes each, spoken aloud (recording yourself is brutal and effective), using the drill checklist (arbiter? hot key? queue? idempotency?).

Mock-interview protocol (with a partner or solo-recorded): 45 minutes strict; partner plays a steering interviewer (interrupts, redirects, asks "why" three times on one component); afterwards score both content (rubric 14.9) and process (time budget 14.1). Rotate problem categories: one feed-like (read-heavy fan-out), one booking-like (consistency arbiter), one pipeline-like (write-heavy streaming), one infra-like (cache/limiter/gateway) — these four archetypes cover nearly every question you'll face.

The night before: re-read the Module 13 cross-cutting cheat sheet and the latency/capacity numbers in 14.3 — those two pages are the highest-density review in this guide.

Module 14 Cheat Sheet — the one-page interview script

0–5	SCOPE	users, 3–5 core ops, OUT-of-scope aloud; numbers for scale/latency/availability/consistency; state assumptions and move.
5–8	NUMBERS	write QPS, read QPS, storage, hot set. Round. Then USE them: "6k writes/s ⇒ shard or buffer." 1M/day≈12/s.
8–12	API+DATA	4–6 endpoints (cursors, idempotency keys); entities + SHARD KEY justified; consistency class per entity.
12–20	HIGH-LEVEL	end-to-end paths for top ops; sync vs async; name the ARBITER of each invariant; stateless vs stateful; invite steering.
20–35	DEEP DIVE	problem → 2–3 options → choose via numbers → mechanics → failure modes. Expect: locking, idempotency, rebalancing, ordering, stampede/hot-key.
35–40	SWEEP	10× bottleneck hunt; kill every box (user impact + recovery); degradation order (shed, stale, fail-closed list).
40–45	CLOSE	SLIs + paging; security in 2 min; RESTATE requirements vs design with numbers; recap top 3 trade-offs; next steps.
ALWAYS narrate; follow hints; numbers over adjectives; simplest thing that meets requirements; volunteer failures before being asked.		

Final Mock Interview Exercise

Full-dress rehearsal: "Design a global event-ticketing platform (Ticketmaster): 100M users, ticket drops sell 50k seats to 2M concurrent buyers in minutes, zero double-selling, fans see live seat maps, payments must be exact." 45 minutes, recorded, strict time budget. It combines every archetype: spike load (queue + waiting room, Module 1.7), the arbiter (seat rows + holds with TTL, 13.7), hot keys (one event = one shard's nightmare, 3.6), payment

sagas + idempotency (7.5–7.6, 13.8), real-time push (2.6), and degradation under load (14.7). Grade yourself with 14.9's rubric — then go book the real interview.